

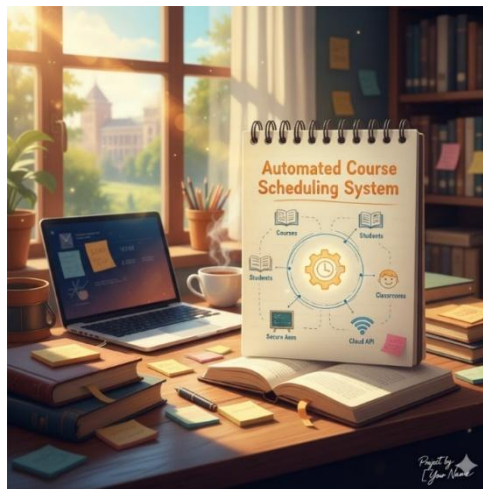


ספר פרויקט לעבודת גמר

י"ד הנדסאי תוכנה (שאלון 714918)

בנושא

מערכת לשיבוץ קורסים אקדמיים ע"פ אילוצים



שם הסטודנט: כפיר יוסף יעקובוב

ת"ז: 216232470

מנחה: אילן פרץ

תאריך הגשה: י"ח סיוון תשפ"ו 03/06/2026

שם המכללה: כנפי רוח קרית נוער ירושלים (סמל מוסד: 140129)

תוכן עניינים

3	1. הצעת הפרויקט שאושרה ע"י משרד החינוך
4	2. מבוא
4	2.1 הרקע לפרויקט
4	2.2 תהליך המחקר
5	2.3 רקע תיאורטי וסקירת ספרות מקצועית
6	2.4 אתגרים מרכזיים במחקר
7	3. מטרות ויעדים לפיתוח המערכת
8	4. אתגרים בבניית המערכת
9	5. הגדרת מדדי הצלחה למערכת
9	6. תיאור פתרונות קיימים לבעיה וניתוח חלופות
10	7. תיאור החלופה הנבחרת ונימוקים לבחירתה
11	8. אפיון המערכת המוצעת
11	8.1 ניתוח הדרישות מהמערכת
12	8.2 מודלי המערכת
13	8.3 אפיון פונקציונלי וביצועים עיקריים
15	8.4 אילוצי המערכת
15	9. תיאור הארכיטקטורה של המערכת
15	9.1 ארכיטקטורת המערכת
17	9.2 מודל שרת לקוח
18	9.3 תהליכים של מע' ההפעלה
18	9.4 תיאור פרוטוקולי תקשורת
19	9.5 שירותים חיצוניים
20	10. ניתוח תרחישים וזרימת המידע במערכת המוצעת
20	10.1 תרשימי תרחיש Use case Diagram
22	10.2 תרשימי רצף Sequence Diagram
28	11. תיאור מסכים וממשק משתמש
28	11.1 תרשימי היררכיית המסכים
29	11.2 תיאור המסכים
52	12. תיאור התוכנה
52	12.1 סביבת הפיתוח של התוכנה
54	12.2 תיאור מבנה הפרויקט והמחלקות
82	12.4 מבני נתונים בהם נעשה שימוש
84	12.5 קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים
89	13. תיאור מסד הנתונים
95	14. מדריך למשתמש
97	15. בדיקות והערכה
97	16. מסקנות
98	17. פיתוחים עתידיים
98	18. ביבליוגרפיה

1. הצעת הפרויקט שאושרה ע"י משרד החינוך

תחום שיבוץ המערכות (Scheduling) במוסדות אקדמיים מהווה אתגר לוגיסטי וחשובי מורכב, המשתייך למשפחת הבעיות של "סיפוק אילוצים" (Constraint Satisfaction Problems - CSP). במוסדות השכלה גבוהה, יצירת לוח זמנים סמסטריאלי אינה מסתכמת רק בשיבוץ טכני של קורסים, אלא דורשת פתרון אופטימיזציה רב-ממדי שנועד לסנכרן בין משאבים מוגבלים לבין דרישות סותרות. האתגרים והבעיות הקיימים בתחום: התהליך הקיים של בניית מערכת שעות מתבצע לעיתים קרובות באופן ידני או חצי-אוטומטי, מה שמוביל למספר כשלים מרכזיים:

- טעויות אנוש והתנגשויות: קושי במעקב אחר מאות אילוצים בו-זמנית גורם לשיבוצים כפולים (לדוגמה התנגשויות מרצים) או ליצירת מערכות שאינן בנות-ביצוע עבור הסטודנטים.
 - חוסר יעילות וניצול משאבים: ללא אלגוריתם אופטימיזציה, קשה לנצל את חדרי הלימוד ואת שעות ההוראה בצורה מיטבית, מה שמוביל לבזבז משאבים מוסדיים.
 - מורכבות חישובית (NP-Hard): הבעיה מאופיינת בגידול מעריכי (אקספוננציאלי) של מרחב הפתרונות ככל שמספר הקורסים והאילוצים עולה. לכן, מציאת הפתרון "המושלם" בזמן סביר היא משימה כמעט בלתי אפשרית ללא שימוש בהיוריסטיקות מתקדמות.
 - ריבוי אילוצים סותרים: הצורך לאזן בין אילוצים "קשיחים" (שאסור להפר, כגון מרצה שלא יכול ללמד בשני מקומות במקביל) לבין אילוצים "רכים" (העדפות נוחות של סטודנטים ומרצים) יוצר קונפליקטים שקשה לפתור ללא כלים ממוחשבים חכמים.
- המטרה בתחום זה היא לפתח מנגנונים שיאפשרו מעבר משיבוץ ידני ומסורבל לשיבוץ דינמי, הממקסם את שביעות הרצון של כלל הגורמים (סטודנטים, מרצים והנהלה) תוך עמידה קפדנית בכל אילוצי המערכת.

תיאור הנושא הפרויקט

מטרת המערכת ואופן פעולתה היא לבצע שיבוץ אוטומטי, חכם ואופטימלי של קורסים במוסד אקדמי, תוך התחשבות בזמינות מרצים, קבוצות לימוד ואילוצי מערכת שונים. המטרה היא לספק פתרון טכנולוגי מתקדם שיחליף את התהליך הידני והמורכב של בניית מערכת שעות, ויאפשר ניהול יעיל, גמיש ודינמי של מערך הלימודים בכל סמסטר.

המערכת מאפשרת למנהלי המערכת להזין נתונים, לנהל קורסים, מרצים וסטודנטים, ולהפעיל אלגוריתם שיבוץ אוטומטי אשר מחשב מערכת שעות מאוזנת וללא התנגשויות בין קורסים, מרצים או קבוצות לימוד. האלגוריתם שואף לבצע אופטימיזציה של מערכת השעות תוך ניצול נכון של משאבי המוסד האקדמי ושמירה על תקינות השיבוץ.

חזונית המשתמש והמערכת פותחה כיישום אינטרנט הנגיש מכל דפדפן, על מנת לאפשר זמינות מלאה ונוחות מרבית ללא צורך בהתקנה מקומית. המערכת מספקת ממשק פשוט וברור עבור כל סוג משתמש, ומאפשרת צפייה וניהול של נתוני מערכת השעות בצורה נוחה וידידותית. לאחר יצירת השיבוץ, כל משתמש יכול לצפות במערכת השעות האישית שלו בהתאם להרשאותיו במערכת.

הגדרת הבעיה האלגוריתמית

הבעיה המרכזית שהמערכת נועדה לפתור היא בעיית שיבוץ ואופטימיזציה של קורסים במוסד אקדמי. מדובר בבעיה קומבינטורית מורכבת שבה יש לשבץ קבוצה גדולה של משאבים - קורסים, מרצים וקבוצות לימוד - על פני ציר זמן מוגדר, תחת מערכת רחבה של אילוצים אקדמיים ומערכתיים. הקושי המרכזי אינו מסתכם רק ביצירת שיבוץ "חוקי" שמונע התנגשויות, אלא ביצירת שיבוץ יעיל ומאוזן המאפשר ניהול תקין של מערכת השעות עבור כלל המשתמשים במערכת.

2. מבוא

2.1 הרקע לפרויקט

ניהול וארגון של תוכניות לימודים במוסדות להשכלה גבוהה מהווה תשתית לוגיסטית קריטית עבור פעילותו השוטפת של המוסד האקדמי. בעוד שההוראה עצמה נמצאת בחזית, הרי שמאחורי הקלעים נדרש תכנון קפדני ומקדים של מערך הקורסים הסמסטריאלי והשנתי. באופן מסורתי, מלאכת הרכבת לוח הזמנים והשיבוץ של מאות קורסים מול סגלי הוראה מגוונים התבססה על פתרונות ידניים, הנסמכים על ניסיון עבר, לוחות אלקטרוניים פשוטים ותהליכים חצי-אוטומטיים. שיטות עבודה אלו, אשר התאימו אולי לעשורים עברו, מתגלות כבלתי יעילות לחלוטין בעידן הנוכחי, שבו חלה קפיצת מדרגה משמעותית בכמות מסלולי הלימוד, בהיקף הקורסים ובתלויית האקדמיות ביניהם. ההסתמכות על הגורם האנושי בניהול מערכות רחבות היקף אלו גורמת לבזבז משאבי זמן יקרים של מזכירות אקדמיות וסגלי ניהול, ומייצרת פתרונות חלקיים בלבד כאשר נכללים ביניהם כשלים מבניים חמורים. לכן, פרויקט זה נולד מתוך הצורך ההולך וגובר להחליף את דפוסי העבודה המיושנים הללו ולהעמיד חלופה טכנולוגית חכמה המסוגלת לעבד נתונים עמוסים ולהפיק לוח זמנים יציב, מאוזן ותקני.

2.2 תהליך המחקר

הליך המחקר והפיתוח במסגרת פרויקט זה התקדם בצורה מאורגנת ומסודרת, החל משלב אפיון המערכת ועד להערכת התוצרים החישוביים. בשלב הראשוני בוצע ניתוח דרישות מעמיק, שמטרתו הייתה הבנה והגדרה מדויקת של חוקי העסק האקדמיים והמרתם לשפה ברורה ומובנת.

במסגרת זו נחקרו יחסי התלות המבניים בין הקורסים השונים (דרישות קדם אקדמיות) ומופּוּ מגבלות הזמינות של סגל המרצים. בשלב השני, תורגמה בעיית השיבוץ למודל מתמטי-גרפי המבוסס על תורת הגרפים, שבו נבחנו מודלים שונים לייצוג קשרים ותלויות. השלב השלישי הוקדש לסקירה, ניתוח והשוואה של גישות אלגוריתמיות מגוונות בעולם האופטימיזציה הדיסקרטית. בשלב זה נבחנו ונפסלו אלגוריתמים חמדניים, אלגוריתמים גנטיים ושיטות חישול מדומה, מתוך הבנה שהם אינם מסוגלים להבטיח את רמת הדיוק הולגית הנדרשת לבעיה זו. לבסוף, גובש מודל ייחודי המשלב מיון היררכי מקדים יחד עם אלגוריתם חיפוש רקורסיבי בעל מנגנוני סינון מתקדמים, אשר הוטמע ונבדק תחת תרחישי קיצון מוסדיים.

2.3 רקע תיאורטי וסקירת ספרות מקצועית

בספרות המקצועית של מדעי המחשב וחקר הביצועים, בעיית שיבוץ הקורסים (Course Scheduling) מוגדרת כאחת הבעיות הקלאסיות והמורכבות ביותר של "סיפוק אילוצים" (Constraint Satisfaction Problems - CSP). מאפייניה המרכזיים של בעיה זו מקנים לה מעמד חישובי של NP-Hard, שמשמעותו היא שלא קיים אלגוריתם דטרמיניסטי המסוגל למצוא פתרון אופטימלי גלובלי בזמן ריצה פולינומי, ומרחב החיפוש גדל בצורה אקספוננציאלית (מעריכית) ככל שנוספים משתנים למערכת. הספרות מחלקת את מרכיבי הבעיה לשתי קטגוריות של אילוצים לוגיים:

- אילוצים קשיחים (Hard Constraints): תנאים מתמטיים מוחלטים שחובה לספק על מנת שהפתרון ייחשב לחוקי ותקף. בקטגוריה זו נכללים חוקים פיזיים ולוגיים, כגון מניעת שיבוץ של מרצה מסוים בשני קורסים מקבילים בציר הזמן, והבטחה שקורסי חובה השייכים לאותו סמסטר ומסלול לא יחפפו זה את זה, כדי לאפשר רצף לימודי תקין.
- אילוצים רכים (Soft Constraints): מדדים המגדירים את איכות הפתרון, הנוחות שלו ומידת האופטימיזציה שלו. בניגוד לאילוצים הקשיחים, הפרה של אילוץ רך אינה פוסלת את חוקיות המערכת, אך היא גוררת "קנס" בפונקציית המטרה. היעדים המרכזיים תחת קטגוריה זו הם התחשבות בהעדפות ימי העבודה הנוחים למרצים, וכן שאיפה מערכתית לצמצום חלונות זמן (שעות מתות) בטבלאות התזמון הכלליות של המוסד.

המחקר האקדמי מראה כי גישות פשוטות כגון אלגוריתמים חמדניים (Greedy Algorithms) נכשלות בפתרון בעיות CSP מורכבות, שכן הן מבצעות בחירות מקומיות אופטימליות לטווח הקצר מבלי להתחשב בהשלכות העתידיות, מה שמוביל במהירות למבוי סתום במרחב החיפוש. מנגד, שיטות מטה-היוריסטיות כמו אלגוריתמים גנטיים (Genetic Algorithms) או חישול מדומה (Simulated Annealing) סובלות מזמני התכנסות ארוכים במיוחד ואינן מסוגלות להבטיח עמידה מוחלטת ב- 100% מהאילוצים הקשיחים, דבר המייצר מערכות שעות שאינן ניתנות לביצוע בפועל.

על כן, הגישה המקובלת והיציבה ביותר נשענת על שילוב של סריקה רקורסיבית מבוקרת יחד עם גיזום מתמטי של תתי-עצים לא רלוונטיים במרחב המצבים.

2.4 אתגרים מרכזיים במחקר

פיתוחה של מערכת ממוחשבת לשיבוץ ואופטימיזציה של קורסים אקדמיים מעמיד בפנינו שורה של אתגרים מחקרניים והנדסיים מורכבים. הפיכת תהליך השיבוץ הידני והמסורבל למנגנון אוטומטי לחלוטין אינה משימה פשוטה, שכן היא דורשת פתרון מדויק לבעיה מתמטית קומבינטורית קשה. האתגרים המרכזיים במחקר זה מתחלקים לשלושה היבטים משלימים: מציאת פתרון טכנולוגי ואלגוריתמי שיועד להתמודד עם הגידול הקיצוני בכמות האפשרויות מבלי לגרום או לנעול את משאבי המערכת, הבנת המוטיבציה והצורך הממשי בהחלפת השיטות הישנות שקיימות כיום בשטח, והגדרת הצרכים המערכתיים שעליהם התוכנה צריכה לענות כדי לייעל את ניצול משאבי המוסד האקדמי. בסעיפים הבאים נפרט בהרחבה על האתגרים הללו, על דרכי ההתמודדות איתם ועל המטרות שהנחו את שלבי הפיתוח של הפרויקט.

2.4.1 התמודדות עם הבעיה

ההתמודדות עם הסיבוכיות החישובית של בעיית השיבוץ בפרויקט זה מבוססת על גישת פתרון דו-שלבית חכמה, המשלבת מבני נתונים מתקדמים מתורת הגרפים יחד עם היוריסטיקות חיפוש רקורסיביות. בשלב הראשון, המערכת מייצגת את כלל הקורסים ותנאי הקדם שלהם כגרף מכוון ללא מעגלים (Directed Acyclic Graph - DAG). על מבנה זה מופעל אלגוריתם מיון טופולוגי (Topological Sort) מבוסס חיפוש עומק (DFS). אלגוריתם זה מפיק סידור לוגי ליניארי של הקורסים, המבטיח כי כל קורס ישובץ רק לאחר שכל תנאי הקדם שלו סופקו במלואם (לדוגמה, הבטחה מבנית שקורס מבוא ישובץ בסמסטר מוקדם יותר מקורס מתקדם התלוי בו).

בשלב השני, בהתבסס על השלד ההיררכי שנבנה, המערכת מפעילה מנוע חיפוש מסוג נסיגה לאחור (Backtracking) לצורך הצבת הקורסים בפועל על פני משבצות הזמן. כדי למנוע את הגידול האקספוננציאלי בזמן הריצה, משולבים מנגנוני גיזום (Pruning) חכמים המבצעים בדיקות תקפות בזמן אמת. ברגע שהאלגוריתם מזהה כי הצבה מסוימת מפרה אילוץ קשיח (כגון חפיפת זמנים של מרצה), הוא קוטע (גוזם) מיד את המשך חקירת ענף החיפוש הנוכחי וחוזר לאחור (Backtrack). תהליך זה מבטיח כי האלגוריתם יתקדם אך ורק במסלולים חוקיים מבחינה מבנית, עד למציאת מערכת שעות שלמה, חוקית ואופטימלית העונה בצורה המיטבית על דרישות המוסד.

2.4.2 הסיבות והמוטיבציה לבחירת הנושא

המוטיבציה המרכזית לבחירת נושא זה נובעת מהפער הניכר הקיים בין ההתקדמות הטכנולוגית המדעית לבין היישום שלה בפועל בניהול הלוגיסטי של מוסדות אקדמיים רבים.

חווית הלימוד האישית חושפת לעיתים קרובות את הכשלים המובנים של מערכות שעות שנבנו בצורה חובבנית או ידנית: חלונות זמן ארוכים המאלצים סטודנטים ומרצים לבזבז שעות יקרות בקמפוס, או חפיפות בלתי הגיוניות בין קורסי חובה המונעות התקדמות אקדמית תקינה וגוררות גרירת תארים ועיכובים מנהלתיים. מן ההיבט המקצועי, אתגר זה מייצג מקרה בוחן מרתק שבו תיאוריות אלגוריתמיות מופשטות ומורכבות כמו תורת הגרפים, סיבוכיות חישובית ובעיות CPS אינן נשארות על הנייר, אלא מתורגמות לכלי תוכנה מעשי ודינמי הפותר בעיה מוחשית בעולם האמיתי.

2.4.3 על איזה צורך הפרויקט עונה ?

הפרויקט עונה על צורך מוסדי ומערכתי כפול, המקיף הן את הצד התפעולי והן את הצד הפדגוגי של המוסד האקדמי:

- במישור המוסדי-ניהולי: המערכת פותרת את התלות המוחלטת בגורם האנושי ובמאות שעות עבודה ידניות המושקעות מדי סמסטר על ידי מחלקות הרישום והמזכירות בניסיונות לפתור קונפליקטים נקודתיים. היא מספקת פלטפורמה ריכוזית ואוטומטית המאפשרת למנהל המערכת להפיק טיוטת שיבוץ שלמה ומאוזנת בלחיצת כפתור אחת, תוך הבטחה מתמטית לאפס התנגשויות מבניות.
- במישור איכות ההוראה והמשאבים: המערכת עונה על הצורך בניצול אופטימלי ומתוכנן היטב של שעות ההוראה של המרצים ומונעת שחיקה הנגרמת מלוחות זמנים מקוטעים. על ידי בניית לוח זמנים מהודק, נטול חפיפות לוגיות ובעל מינימום חלונות זמן מתים, המערכת מייעלת את השימוש במשאבי המוסד, משפרת את האטרקטיביות שלו, ומבטיחה לסטודנטים מסלול לימודים רציף, עקבי ותקין העומד בכל הדרישות האקדמיות שהוגדרו.

3. מטרות ויעדים לפיתוח המערכת

מטרת העל של פרויקט זה היא להציב חלופה טכנולוגית מתקדמת, אמינה וריכוזית לתהליכי התכנון הידיניים המקובלים במוסדות אקדמיים, באמצעות פיתוח יישום אינטרנט חכם לשיבוץ אוטומטי ואופטימלי של קורסים. המערכת שואפת להוות כלי ניהולי אסטרטגי עבור הנהלת המוסד והמזכירות האקדמית, אשר יחליף את מאות שעות העבודה המושקעות מדי סמסטר בניסיונות ידיניים לפתרון קונפליקטים מורכבים. מן ההיבט המערכתי, הפרויקט מכון ליצירת סביבה אקדמית רציפה ויציבה, הממקסמת את היעילות התפעולית של המוסד, מונעת שחיקה של סגל ההוראה כתוצאה מלוחות זמנים לקויים, ומבטיחה לסטודנטים מסלול לימודים תקיני, עקבי ונטול חסמים מבניים.

כדי לתרגם את מטרות העל הללו למונחים מעשיים והנדסיים, הוגדרו עבור המערכת מספר יעדים מרכזיים:

- אוטומציה מלאה של תהליך התזמון: פיתוח מנוע חישוב עצמאי המסוגל לעבד את נתוני התשתית של המוסד ולהפיק, בלחיצת כפתור, לוח זמנים סמסטריאלי שלם עבור כלל מסלולי הלימוד.
- עמידה מוחלטת באילוצים קשיחים: הבטחה מתמטית של 100% לחוקיות המערכת, הכוללת מניעה מוחלטת של כפילויות בזמני המרצים (מרצה אינו יכול ללמד שני קורסים במקביל) ומניעת חפיפות זמנים בין קורסי חובה השייכים לאותו סמסטר ומסלול.
- אופטימיזציה של אילוצים רכים: הטמעת פונקציית מטרה משוקללת השואפת למקסם את איכות הפתרון, באמצעות התחשבות מרבית בהעדפות ימי העבודה שהוגדרו על ידי המרצים, לצד שאיפה גורפת לצמצום חלונות זמן מתים במערכת השעות הכללית.
- אכיפת תלויות אקדמיות מובנית: שילוב מנגנון טופולוגי המבטיח כי סדר הצעת הקורסים בציר הזמן יכבד באופן מוחלט את יחסי הקדם האקדמיים שנקבעו בתוכנית הלימודים.
- מתן ממשק ניהול אינטואיטיבי: אספקת סביבת עבודה מודרנית ונגישה המאפשרת למנהלי המערכת להזין, לעדכן ולבחון נתונים בקלות, ולצפות בתוצרי האלגוריתם בצורה ברורה, בהירה ומוגנת מפני שגיאות לוגיות.

4. אתגרים בבניית המערכת

פיתוחה של מערכת ממוחשבת לשיבוץ ואופטימיזציה של קורסים אקדמיים מעמיד שורה של אתגרים מורכבים. אתגרים אלו אינם מצטמצמים רק לכתיבת קוד תקין, אלא נוגעים בליבת מדעי המחשב: החל מניהול סיבוכיות חישובית תחת משאבים מוגבלים ועד לעיצוב ארכיטקטורה לוגית חסינת שגיאות.

האתגר המרכזי והראשון במעלה הוא ההתמודדות עם בעיית הגידול המעריכי (האקספוננציאלי) של מרחב החיפוש. מאחר שבעיית סיפוק האילוצים (CSP) הנוכחית מוגדרת כ-NP-Hard, כל הוספה של קורס יחיד למאגר הנתונים מכפילה ומשלשת את כמות השילובים האפשריים בציר הזמן. ללא הגדרת גבולות גזרה ברורים ומנגנוני סינון קשוחים, זמן הריצה של האלגוריתם עלול להגיע לערכים בלתי סבירים שיגרמו לקריסת השרת או לנעילת זימון המערכת (Out of Memory). כמענה הנדסי ישיר לאתגר זה, ועל מנת להבטיח את יציבות המערכת ויכולתה להתכנס לפתרון תקף בזמן אמת, הוטמעה במערכת מגבלה מובנית המונעת הזנה ושיבוץ של מעל 20 קורסים בו-זמנית בסמסטר נתון. מגבלה זו מהווה חסם עליון (Upper Bound) הכרחי, אשר יחד עם מנגנוני הגיזום (Pruning), מאפשר לשלוט על גודל עץ החיפוש הרקורסיבי ולשמור על יעילות חישובית מקסימלית.

אתגר מהותי נוסף טמון בסינכרון הדינמי והמבוקר בין האילוצים הקשיחים לאילוצים הרכים. המערכת נדרשת לבצע הצלבה מתמטית מורכבת בין שלל נתונים: קביעת רצף לימודי קשיח המבוסס על גרף תלויות (תנאי קדם), הימנעות מוחלטת מחפיפה בין קורסי חובה, מניעת שיבוץ כפול של מרצים, ומנגד - שאיפה מתמדת לריצוי העדפות הזמינות של המרצים וצמצום חלונות זמן מתים.

האתגר ההנדסי פה הוא בניית לוגיקת בדיקה וסינון יעילה בתוך אלגוריתם הנסיגה לאחור, אשר תדע לזהות חוסר אופטימיזציה ולבצע בחירות חכמות בין מסלולים חוקיים, מבלי לפסול פתרונות שהם תקפים לחלוטין מבחינה מבנית, ותוך הבטחה מוחלטת שאפס קונפליקטים לוגיים יתרחשו בתוצר הסופי.

5. הגדרת מדדי הצלחה למערכת

הערכת הצלחתו של הפרויקט והוכחת יעילותו ההנדסית נשענות על קביעת מדדי הצלחה ברורים ומדידים, המצליבים בין נכונות לוגית לבין ביצועים חישוביים בתנאי אמת. המדד הראשי והעליון להצלחת המערכת הוא הבטחה מתמטית של 100% לחוקיות הפתרון המופק (Zero Conflicts). המערכת תיחשב כמי שחוללה שיבוץ מוצלח אך ורק אם לוח הזמנים הסופי עומד לחלוטין בכל האילוצים הקשיחים שהוגדרו: אפס כפילויות בלוחות הזמנים של סגל המרצים, ואפס חפיפות בציר הזמן בין קורסי חובה השייכים לאותו סמסטר ומסלול לימודים. כל חריגה לוגית, ולו הקטנה ביותר, שתביא להתנגשות בין מועדי הוראה או לפגיעה ברצף הלימודי של מסלול מסוים, תגדיר את פעולת המערכת ככישלון.

מדד קריטי נוסף להצלחה נוגע ליעילות החישובית וזמני התגובה של אלגוריתם הנסיגה לאחור (Backtracking). המערכת נדרשת להתכנס לפתרון שלם וחוקי, או לחילופין להודיע למשתמש על אי-ייתכנות מתמטית לשיבוץ (במקרה של סתירה מובנית באילוצים), תוך זמן ריצה סביר שאינו עולה על שניות בודדות עבור קלט מקסימלי של עד 20 קורסים לסמסטר. עמידה בחסם זמן זה, הודות למנגנוני הגיזום (Pruning) הלוגיים המונעים את קריסת הזיכרון, תהווה הוכחה ליציבותו של המנוע האלגוריתמי ותאימותו לעבודה דינמית מול משתמש קצה.

מעבר לתקינות הלוגית הגורפת, הצלחת המערכת תימדד באמצעות מדדי אופטימיזציה של איכות הפתרון (האילוצים הרכים). הישג זה ייבחן על ידי פונקציית הערכה המודדת את אחוז ריצוי בקשות הזמינות של המרצים ואת רמת הדחיסות של מערכת השעות הכללית. המערכת תשאף למקסם את שיבוץ המרצים בימים המועדפים עליהם ולמזער ככל הניתן חלונות זמן מתים בתוכנית הסמסטריאלית. לבסוף, הפרויקט יימדד גם במישור הממשקי, ויחייב הצגת חוויית משתמש חלקה, חסינת טעויות ואינטואיטיבית, המאפשרת למנהל המערכת להזין נתונים אקדמיים מורכבים ולצפות בתוצרי השיבוץ בצורה בהירה, ארגונומית ומאורגנת היטב.

6. תיאור פתרונות קיימים לבעיה וניתוח חלופות

כשבאים לבחון כיצד מוסדות אקדמיים מתמודדים כיום עם האתגר הלוגיסטי של שיבוץ קורסים, ניתן לחלק את הפתרונות הקיימים בשוק ובספרות המקצועית לשתי חלופות מרכזיות:

פתרונות מסחריים רחבי היקף, או שימוש במודלים אלגוריתמיים מבוססי אקראיות והסתברות (כמו אלגוריתמים גנטיים או שיטות חישול מדומה). ניתוח מעמיק של החלופות האלו חושף פערים משמעותיים בין לבין הצרכים ההנדסיים והנקודתיים של הפרויקט הנוכחי, ומסביר מדוע פיתוח של פתרון עצמאי וממוקד הוא הצעד הנכון ביותר.

החלופה הראשונה הנפוצה בשוק כוללת מערכות תוכנה מסחריות גדולות לניהול אקדמי. מדובר במערכות ענק (מערכות ERP מוסדיות) שמנסות לנהל את כל משאבי הקמפוס תחת קורת גג אחת - החל מניהול כספי של שכר לימוד, דרך מעקב ציונים קפדני, ועד לשיבוץ של כיתות לימוד פיזיות, מעבדות, ומערכות שעות אישיות המותאמות לכל סטודנט וסטודנט. למרות שהמערכות האלו כוללות כלים חזקים, הן סובלות מכמה חסרונות משמעותיים עבור מוסדות רבים: הן יקרות מאוד לרכישה ולתחזוקה, דורשות תהליכי הטמעה ארוכים ומסורבלים, ומאחר שהן בנויות כ"קופסה שחורה", קשה מאוד לשנות או להתאים את החוקים הפנימיים שלהן בקלות. מעבר לכך, המערכות האלו פועלות מתוך הנחה קשיחה שיש צורך לשבץ חללים פיזיים (כיתות) ולטפל באילוצים אישיים של אלפי תלמידים בודדים. במערכת שלנו דרישות אלו אינן קיימות ואף יוצרות עומס חישוב מיותר, כיוון שהאפיון שלנו ממוקד כולו בבניית לוח זמנים תקין לקורסים עצמם ובהצלבה ישירה מול זמינות המרצים. לכן, שימוש במערכת מדף כזו הוא פתרון כבד, מסורבל ולא יעיל עבור הצרכים הספציפיים של הפרויקט.

החלופה השנייה, שנפוצה מאוד במחקר ובספרות של מדעי המחשב, היא שימוש באלגוריתמים מבוססי היוריסטיקה (כמו אלגוריתם גנטי או חישול מדומה). אלגוריתמים אלו מנסים להתמודד עם מרחבי חיפוש גדולים על ידי שיטות של ניסוי וטעייה, הסתברות וחיקוי של תהליכים מהטבע, במטרה להגיע לפתרון "מספיק טוב" בזמן סביר. היתרון שלהם הוא היכולת לרוץ על כמויות נתונים ללא הגבלה, אך החיסרון המכריע שלהם בבעיה של סיפוק אילוצים (CSP) הוא חוסר היכולת להבטיח דיוק של 100%. בבעיית שיבוץ אקדמית, אם האלגוריתם מוצא פתרון שהוא "כמעט מושלם" אבל משבץ מרצה מסוים לשני קורסים שונים במקביל, או יוצר חפיפה בין שני קורסי חובה באותו מסלול - לוח הזמנים כולו נפסל והופך לבלתי שמיש לחלוטין בפועל. בגלל שהמערכת שלנו דורשת אפס סובלנות לטעויות ולוגיות ודיוק מוחלט, השימוש בשיטות אקראיות אלו נפסל ולא יכול להוות בסיס למנוע שיבוץ אמין.

7. תיאור החלופה הנבחרת ונימוקים לבחירתה

כחלופה לפתרונות הקיימים שנבחנו ומופו בסעיף הקודם, הפרויקט הנוכחי מציע ומיישם גישה הנדסית עצמאית, ממוקדת והיברידית. הבחירה המרכזית בארכיטקטורת המערכת היא שילוב בין שני אלגוריתמים דטרמיניסטיים קלאסיים: מיון טופולוגי (Topological Sort) לצורך סידור היררכי מוקדם של תנאי הקדם, ואלגוריתם נסיגה לאחור (Backtracking) יחד עם מנגנון גיזום (Pruning) לטובת שיבוץ בפועל על גבי לוח הזמנים המבוקש.

החלופה הזו נבחרה ונשענת על מספר נימוקים מקצועיים והנדסיים קריטיים, המותאמים בצורה מושלמת למגבלות המערכת:

- הבטחת דיוק של 100% בחוקיות לוח הזמנים: בניגוד לחלופות ההיסטוריות והאקראיות, אלגוריתם נסיגה לאחור בשילוב עם מיון טופולוגי אינו מנחש או מסתמך על מזל והסתברות. המערכת בונה גרף מכון ללא מעגלים (DAG) שמייצג את כל הקורסים, ומסדרת אותם ליניארית כך שקורס מבוא תמיד ישובץ במערכת לפני קורס מתקדם התלוי בו. לאחר מכן, מנוע החיפוש הרקורסיבי מציב את הקורסים בלוח הזמנים רק לאחר בדיקה קפדנית של האילוצים הקשיחים. אם מתגלה התנגשות בזמנים המרצה או חפיפה של קורסי חובה באותו מסלול, מנגנון הגיזום חותך מיד את המסלול וחוזר לאחור (Backtrack). צורת עבודה זו מבטיחה שאף מערכת שעות לא תיווצר עם שגיאות לוגיות, דבר שהוא קריטי להפעלה מעשית במוסד אקדמי.
- ליעילות החישובית תחת מגבלות משאבי המערכת: פתרונות קיימים נוטים להאט או לקרוס בשל כמות משתנים עצומה. בפרויקט זה הוגדר חסם עליון קשיח של עד 20 קורסים בסך הכל לכלל המערכת. הגבלה הגיונית זו מקטינה באופן דרסטי את עץ האפשרויות של האלגוריתם ומביאה אותו לגודל נשלט. בזכות כך, זמני הריצה והחישוב של המערכת קצרים במיוחד ואורכים שניות בודדות, מה שמאפשר לבצע את השיבוץ בזמן אמת ומספק חוויית משתמש מהירה מבלי להעמיס על שרת הנתונים או לגרום לנעילת זיכרון.
- פשטות היישום והתאמה מדויקת לדרישות: המערכת לא "סוחבת" איתה קוד מיותר ורעש חישובי שמאפיין מערכות מדף גדולות, כמו ניהול חללים פיזיים (כיתות) או טיפול באילוצים אישיים ופרטניים של כל סטודנט. על ידי התמקדות נקודתית בשלד הקורסים, חוקי הקדם האקדמיים וזמנים המרצים, הצלחנו לבנות מערכת ברורה, יעילה, שקל מאוד לתחזק ולעדכן אותה, ושתואמת בדיוק מוחלט את הגדרת הבעיה המקורית של הפרויקט.

8. אפיון המערכת המוצעת

בפרק זה נגדיר ונאפיין את הדרישות, המודלים והביצועים של המערכת המוצעת. האפיון נועד להעניק תמונה ברורה של יכולות התוכנה, המגבלות שלה והדרך שבה היא ממפה את העולם האקדמי לכדי פתרון תוכנתי.

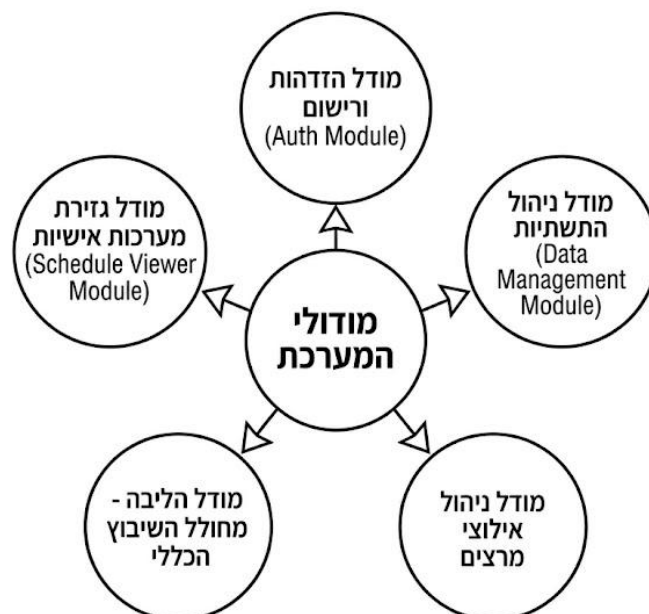
8.1 ניתוח הדרישות מהמערכת

כדי שהמערכת תיתן מענה אמיתי ויעיל לבעיית שיבוץ הקורסים, הוגדרו עבורה מספר דרישות מרכזיות שחובה לעמוד בהן, הנוגעות למשתמשים, לנתונים ולביצועים:

- ריבוי משתמשים והרשאות: המערכת נדרשת לתמוך בשלושה סוגי משתמשים עיקריים שפועלים בה במקביל: מנהל המערכת (Admin), מרצים וסטודנטים. לכל סוג משתמש יש הרשאות שונות לחלוטין. מנהל המערכת מקבל שליטה מלאה על הזנת הקורסים והפעלת אלגוריתם השיבוץ. המרצים מורשים להזין את מגבלות הזמינות שלהם ולצפות במערכת שלהם. והסטודנטים מורשים רק לצפות במערכת השעות הסופית הרלוונטית אליהם.
- אבטחה ומניעת כפילויות: כדי להבטיח את אמינות הנתונים ולמנוע כפילויות משתמשים במסד הנתונים, דרישת חובה של המערכת היא שהרישום וההזדהות (Login) יתבצעו אך ורק באמצעות כתובת אימייל ייחודית לכל משתמש, ולא באמצעות שם משתמש (Username) כללי.
- שמירת נתונים ועבודה בענן: המערכת נדרשת לפעול כאפליקציית Web הזמינה מכל מקום, ולכן כלל הנתונים (משתמשים, קורסים, זמינות מרצים ומערכות שעות) חייבים להישמר במסד נתונים מרכזי בענן (MongoDB) המסתנכר בזמן אמת.
- יעילות חישובית ואמינות: דרישת הליבה של המערכת היא לייצר מערכת שעות חוקית ב - 100%, ללא שום התנגשויות של מרצים (מרצה לא ילמד שני קורסים במקביל) וללא הפרה של תנאי קדם. המערכת נדרשת לבצע את כל החישובים המסובכים האלו בתוך שניות בודדות, מבלי לקרוס ומבלי לנעול את המחשב.

8.2 מודלי המערכת

המערכת בנויה ממספר "מודלים" (רכיבים לוגיים ופונקציונליים) מרכזיים, שכל אחד מהם אחראי על חלק אחר בתהליך:



- מודל הזדהות ורישום (Auth Module): המודל שאחראי על שער הכניסה למערכת. הוא מנהל את הרישום, את ההתחברות (באמצעות אימייל וסיסמה) ואת ניתוב המשתמש למסך המתאים לו לפי סוג ההרשאה שלו (מנהל, מרצה או סטודנט).
- מודל ניהול התשתיות (Data Management Module): מודל המשמש את מנהל המערכת בלבד. באמצעותו ניתן להוסיף קורסים חדשים, להגדיר מי המרצה המלמד, מהם תנאי הקדם של כל קורס ולאיזה סמסטר הוא מיועד.
- מודל ניהול אילוצי מרצים: מודל המאפשר למנהל המערכת להזין עבור המרצה למערכת את חלונות הזמן שבהם הם אינם יכולים ללמד (למשל, ימים חופשיים או שעות ספציפיות). הנתונים האלו נשמרים ומועברים ישירות למנוע השיבוץ.
- מודל הליבה - מחולל השיבוץ הכללי: זהו המוח של המערכת. מודל זה מופעל על ידי המנהל, והוא לוקח את כל הקורסים ואת זמינות המרצים, ומפעיל עליהם את האלגוריתמים (מיון טופולוגי לתנאי הקדם ונסיגה לאחור - Backtracking למציאת המשבצות הפנויות בלוח הזמנים). המודל מייצר מערכת שעות אחת כללית חוקית ללא קונפליקטים.
- מודל גזירת מערכות אישיות (Schedule Viewer Module): לאחר שמודל הליבה סיים לבנות את המערכת הכללית, מודל זה לוקח את התוצאה ו"גוזר" מתוכה מערכות שעות מותאמות אישית. כך שסטודנט רואה רק את הקורסים שרלוונטיים למסלול שלו, ומרצה רואה רק את לוח זמני ההוראה האישי שלו.

8.3 אפיון פונקציונלי וביצועים עיקריים

מודל הזדהות ורישום (Auth Module):

היבט פונקציונלי: מודל זה משמש כמאבטח בכניסה למערכת ומסדיר את תהליך הרישום וההתחברות. כל משתמש נדרש להזין את כתובת האימייל הייחודית שלו ואת הסיסמה שבחר. המערכת מזהה אוטומטית מי המשתמש שהתחבר ומנתבת אותו ישירות למסך המותאם לו: מנהל המערכת יגיע למסכי השליטה וההפקה, המרצה יראה את חלונות הזמן שלו, והסטודנט יועבר ישירות לצפייה במערכת השעות שלו.

ביצועים עיקריים: תהליך הזיהוי והניתוח מתבצע בצורה מיידי. בזכות השימוש באימייל ייחודי כשדה חובה, המערכת מונעת לחלוטין טעויות אנוש של כפילויות משתמשים או הרשמות כפולות, ומעבירה את המשתמש בין המסכים בצורה חלקה וללא זמני המתנה.

מודל ניהול התשתיות (Data Management Module):

היבט פונקציונלי: מודל זה פתוח לשימוש מנהל המערכת בלבד ומהווה את מרכז הזנת הנתונים האקדמיים.

דרך ממשק זה המנהל יכול להקים קורסים חדשים, לשייך לכל קורס את המרצה שאמור ללמד אותו, להגדיר בצורה ברורה אילו קורסים מהווים תנאי קדם (קורסי חובה שחייבים ללמוד לפני קורס מתקדם) ולקבוע לאיזה סמסטר הקורס מיועד.

ביצועים עיקריים: הזנת הנתונים ועדכוןם נעשים בצורה דינמית ופשוטה. הממשק מעוצב בצורה ארגונומית המאפשרת למנהל לראות את כל תמונת המצב של הקורסים מול עיניו, כאשר כל שינוי או הוספה של קורס נשמרים ומעודכנים במערכת באופן מיידי ומאובטח.

מודל ניהול אילוצי מרצים:

היבט פונקציונלי: רכיב זה מיועד לאיסוף ומעקב אחר זמני הזמינות של סגל ההוראה. המודל מאפשר למנהל המערכת להזין באופן מרוכז עבור כל מרצה ומרצה את חלונות הזמן שבהם הוא אינו יכול ללמד (כמו ימים חופשיים מוגדרים או שעות ספציפיות במהלך היום שבהן המרצה אינו פנוי).

ביצועים עיקריים: המערכת קולטת את מגבלות הזמן האלו ומעדכנת את פרופיל המרצה בזמן אמת. המידע נשמר בצורה מאורגנת ומדויקת, כך שברגע שהמנהל ירצה להפעיל את מחולל השיבוץ, הנתונים יהיו מוכנים ושמורים מראש ללא צורך בתיאומים ידניים מורכבים מול המרצים.

מודל הליבה - מחולל השיבוץ הכללי:

היבט פונקציונלי: זהו המוח המרכזי של האפליקציה, המופעל על ידי מנהל המערכת בלבד באמצעות לחיצת כפתור פשוטה ("הפק שיבוץ"). המודל לוקח באופן אוטומטי את כל הקורסים שהוזנו ואת אילוצי הזמינות של המרצים, ומבצע חישוב חכם: הוא דואג שקורסי המבוא יופיעו תמיד לפני קורסים מתקדמים התלויים בהם, ובמקביל מוצא משבצות זמן פנויות שבהן אין שום התנגשות (מרצה לא ישובץ לשני קורסים במקביל, וקורסי חובה של אותו סמסטר לא יחפפו). התוצר הוא לוח זמנים כללי, תקני וחוקי ב-100%.

ביצועים עיקריים: הביצועים של מודל זה מהירים בצורה יוצאת דופן. מאחר שהמערכת מוגבלת לשיבוץ של עד 20 קורסים בכללי, ומאחר שהאלגוריתם יודע "לגזום" ולדלג מיד על אפשרויות לא חוקיות (כמו שעות שבהן המרצה חסום), החישוב כולו מסתיים בתוך שניות בודדות בלבד, חוסך שבועות של עבודה ידנית ומונע לחלוטין כשלים לוגיים במערכת השעות הסופית.

מודל גזירת מערכות אישיות (Schedule Viewer Module):

היבט פונקציונלי: לאחר שמודל הליבה סיים לחשב את לוח הזמנים הכללי, המודל הנוכחי נכנס לפעולה ותפקידו לפשט את התוצאה עבור המשתמשים. במקום להציג לוח זמנים ענק ומסורבל, המודל מבצע "סינון" וגוזר מערכות שעות ממוקדות: הסטודנט שנכנס למערכת רואה אך ורק את הקורסים שרלוונטיים למסלול הלימודים שלו, והמרצה רואה ביומן שלו אך ורק את הימים והשעות המדויקים שבהם הוא נדרש להגיע ללמד.

ביצועים עיקריים: הפעולה מתבצעת באופן אוטומטי ואינסטנטנטיבי ברגע הטעינה. המערכות האישיות מוצגות למשתמשי הקצה על גבי טבלה שבועית ויזואלית, נקייה, וברורה מאוד לקריאה, המתעדכנת בצורה דינמית ללא צורך ברענון של דפדפן האינטרנט.

8.4 אילוצי המערכת

אילוצי המערכת מתייחסים למגבלות הטכניות והלוגיות שבהן התוכנה פועלת:

- מגבלת נפח הנתונים (עד 20 קורסים): האילוץ ההנדסי והמערכתי החשוב ביותר. כדי להבטיח ביצועים מהירים ולמנוע את קריסת הזיכרון של השרת בשל הגידול האקספוננציאלי של האפשרויות, המערכת מוגבלת לחישוב ושיבוץ של עד 20 קורסים בסך הכל. חסם זה הכרחי לשמירה על יציבות.
- תלות בחיבור אינטרנט: מאחר שהמערכת מבוססת Web (שרת-לקוח) ומסד הנתונים יושב בענן, קיימת תלות מוחלטת בחיבור אינטרנט רציף ופעיל לצורך התחברות למערכת, קריאה או עדכון של נתונים.
- תלות ברצף לוגי תקין: המערכת לא תוכל להפיק שיבוץ אם מנהל המערכת הזין נתונים שיוצרים סתירה מתמטית בלתי פתירה מלכתחילה (למשל, אם שני קורסי חובה באותו סמסטר דורשים את אותו המרצה בדיוק, והמרצה פנוי רק לשעתיים בשבוע). במקרה כזה, אילוץ המערכת הוא לעצור את החישוב ולהחזיר הודעת שגיאה על חוסר היתכנות לוגית, ולא לייצר שיבוץ פגום.

9. תיאור הארכיטקטורה של המערכת

9.1 ארכיטקטורת המערכת

כדי להבטיח שהמערכת תהיה מודולרית, מאובטחת וקלה לתחזוקה ולשדרוג עתידי, היא תוכננה ופותחה על בסיס מודל "ארכיטקטורת שלוש השכבות" (Three-Tier Architecture). מודל זה מחלק את הקוד והפונקציונליות של התוכנה לשלושה חלקים נפרדים ועצמאיים, כאשר כל שכבה אחראית על תפקיד מוגדר ומתקשרת אך ורק עם השכבה שסמוכה אליה.

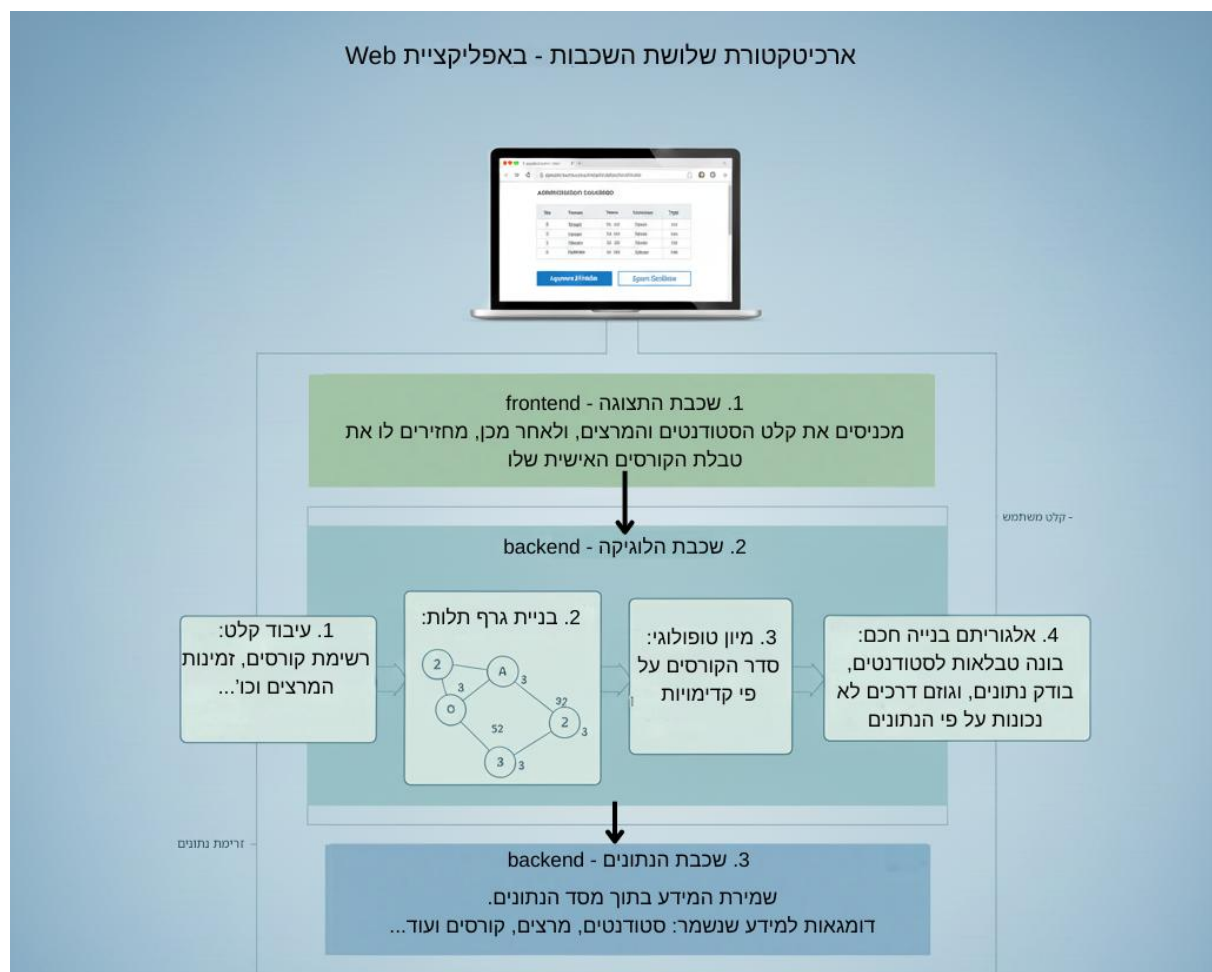
להלן פירוט נרחב על כל אחת מהשכבות, תפקידיה וזרימת המידע ביניהן:

1. שכבת התצוגה (Frontend - ממשק המשתמש): זוהי השכבה החיצונית ביותר, והיא למעשה הפנים של המערכת. כל מה שהמשתמש (מנהל המערכת, המרצה או הסטודנט) רואה מולו בדפדפן האינטרנט נמצא בשכבה זו. תפקידה הוא הצגת המידע בצורה ויזואלית (טבלאות, כפתורים, טפסים) וקליטת פקודות מהמשתמש. שכבה זו "טיפשה" מבחינה חישובית – היא אינה מחשבת את השיבוץ ואינה שומרת נתונים, אלא רק מציגה אותם.

2. שכבת הלוגיקה (Backend - ליבת המערכת): שכבה זו יושבת בשרת המרוחק והיא ה"מוח" המרכזי של האפליקציה. היא מפרידה לחלוטין בין המשתמש לבין הנתונים הרגישים. תפקידה הוא קבלת הבקשות משכבת התצוגה, בדיקת התקינות שלהן, והפעלת חוקי העסק (Business Logic) של המוסד האקדמי.

3. שכבת הנתונים (Database): השכבה התחתונה ביותר במערכת, המוקדשת כולה לשמירה ואחסון של המידע לטווח ארוך. תפקידה הוא שמירת כלל הישויות של המערכת (קורסים, מרצים, משתמשים ושיבוצים) בצורה בטוחה, יעילה ונגישה. משתמשי הקצה (שכבת התצוגה) לעולם אינם ניגשים לשכבה זו ישירות. רק ה-Backend מורשה "לדבר" עם מסד הנתונים, מה שמונע פריצות אבטחה או שינוי נתונים זדוני.

זרימת המידע במערכת: כאשר מנהל לוחץ על כפתור "הפק שיבוץ" ב-Frontend, נשלחת בקשה מסודרת ל-Backend. ה-Backend מיד פונה ל-Database ומבקש ממנו את רשימת הקורסים המעודכנת (עד 20 קורסים) ואת שעות הזמינות של המרצים. לאחר שה-Database מחזיר את הנתונים, ה-Backend מריץ את אלגוריתם האופטימיזציה ומייצר לוח זמנים חוקי ללא כפילויות. בסיום, ה-Backend שומר את לוח הזמנים החדש ב-Database, ושולח הודעת אישור ל-Frontend, שמעדכן את המסך ומציג את הטבלה למשתמש.



בתרשים אפשר לראות שארכיטקטורת שלוש השכבות (Three-Tier Architecture) ממומשת באפליקציית Web, והיא מפרידה בבירור בין האחריות והתפקוד של כל שכבה במערכת.

השכבה הראשונה היא שכבת התצוגה, שכבה זו מיוצגת תחת הספרה 1 בתרשים והיא ממשק המשתמש. היא מיוצגת על ידי ממשק ה- Web בדפדפן, והיא מטפלת בקלט המשתמש (כגון הכנסת אילוצים) ומציגה את הנתונים, כמו טבלת הקורסים האישית. השכבה השנייה היא שכבת הלוגיקה היא מיוצגת תחת הספרה 2 בתרשים. היא ממוקמת בbackend, והיא לב המערכת. שכבה זו אחראית על עיבוד נתוני הקלט וכוללת לוגיקה מורכבת כמו בניית גרף תלויות, מיון לפי קדימויות ויישום אלגוריתם חכם לשיבוץ כדי לייצר את מערכת השעות הסופית. השכבה השלישית והאחרונה שמופיעה תחת הספרה 3 היא שכבת הנתונים ממוקמת בתחתית, והיא אחראית אך ורק על אחסון ושליפת הנתונים (כגון נתוני סטודנטים, מרצים, קורסים) עבור שכבת הלוגיקה העסקית.

התרשים מדגים כיצד המידע זורם בין השכבות, כאשר כל שכבה מטפלת בתחום האחריות שלה בלבד, מה שמבטיח מודולריות וקלות תחזוקה.

9.2 מודל שרת לקוח

המערכת פועלת על בסיס מודל שרת-לקוח מובהק, אשר מחלק את העומס החישובי ומפריד בין התוכנה שמציגה את המידע לבין התוכנה שמעבדת אותו. חלוקה זו קריטית במיוחד בפרויקט העוסק בבעיית NP-Hard (סיבוכיות זמן אקספוננציאלית), שכן היא מונעת קריסה של המחשב האישי של המשתמש.

הלקוח (Client): הלקוח מיוצג על ידי דפדפן האינטרנט של המשתמש קצה (בין אם זה בסמארטפון או במחשב הנייד). הלקוח מתפקד כ"לקוח רזה" (Thin Client) – כלומר, אין עליו עומס עיבוד בכלל. תפקידו הוא רק לטעון את ממשק המשתמש ולשלוח בקשות לשרת. כאשר משתמש נכנס למערכת באמצעות האימייל שלו, הלקוח אוסף את פרטי ההתחברות, שולח אותם דרך רשת האינטרנט וממתין לפקודות כיצד לצייר את המסך הבא.

השרת (Server): השרת הוא מחשב מרוחק ועוצמתי (במקרה שלנו, שרת Spring Boot) הפועל באופן רציף (24/7) וממתין לפניית מהלקוחות. השרת פועל כ"נקודת האמת המרכזית" של המערכת. כל הלוגיקה והמתמטיקה הכבדה של אלגוריתם הנסיגה לאחור מתרחשת עליו.

היתרונות המובהקים של מודל זה בפרויקט הנוכחי:

- ביצועים ויציבות: גם אם המחשב של מנהל המערכת חלש או איטי, תהליך הפקת השיבוץ ירוץ במהירות שיא (שניות בודדות), משום שהעבודה הקשה מתבצעת על מעבדי השרת החזקים ולא בדפדפן.

- ריכוזיות וסנכרון: ברגע שמנהל עורך קורס או שמרצה חוסם יום ביומן, הנתון נשלח מיד לשרת המרכזי. כל משתמש אחר שייכנס למערכת באותו רגע יקבל את המידע המעודכן ביותר, ללא חשש להתנגשויות גרסאות או מידע חסר.
- אבטחה מקסימלית: האלגוריתם, חוקי הקדם של הקורסים וסיסמאות המשתמשים לא נשמרים אצל הלקוח. פריצה למחשב הלקוח לא תחשוף את קוד המקור או את הנתונים של שאר המשתמשים במערכת.

9.3 תהליכים של מע' ההפעלה

כדי שהמערכת תפעל, מערכת ההפעלה מקצה משאבים ומנהלת מספר תהליכים וחוסים המאפשרים תקשורת ועיבוד מקבילי. סעיף זה מתאר את נקודת המבט של מערכת ההפעלה על הפרויקט.

- ברמת מערכת ההפעלה, האפליקציה פועלת כתהליך יחיד (Process) של סביבת הריצה של ג'אווה. מערכת ההפעלה מקצה לתהליך זה זיכרון (RAM) וזמן עיבוד (CPU).
- Thread Pool של שרת היישומים (Tomcat): השרת המובנה ב-Spring Boot מייצר מראש "בריכת חוסים" ברמת מע' ההפעלה. כאשר בקשת HTTP מגיעה מהרשת, Tomcat מקצה לה חוט מערכת הפעלה נפרד. הדבר מאפשר ללוגיקת הרישום או ההתחברות של משתמש א' לרוץ במקביל ללוגיקה של משתמש ב', מבלי שהם יפריעו זה לזה.

תקשורת, פורטים וכרטיסי רשת (Networking & Ports)

פורט 8080 (Http/Application Port): זהו הפורט אליו מאזין שרת Tomcat.

9.4 תיאור פרוטוקולי תקשורת

כדי שארכיטקטורת שלוש השכבות ומודל השרת-לקוח יעבדו בסנכרון מלא, חלקי המערכת חייבים "לדבר" באותה שפה. המערכת עושה שימוש בפרוטוקולי התקשורת הסטנדרטיים והמתקדמים ביותר בשוק כדי להבטיח זרימת מידע אמינה, מהירה ומאובטחת המערכת משתמשת ב:

פרוטוקול HTTP / HTTPS (HyperText Transfer Protocol Secure): זהו פרוטוקול התקשורת המרכזי של רשת האינטרנט (שכבת היישום). המערכת משתמשת בו כדי לנהל את ה"שיחה" הבסיסית בין הדפדפן לשרת באמצעות בקשות כמו GET (למשיכת טבלת קורסים) או POST (לשליחת טופס התחברות עם אימייל וסיסמה). המערכת תשאף להשתמש בגרסה המאובטחת והמוצפנת (HTTPS), כדי להבטיח שכל מידע שעובר ברשת (כמו נתוני זמינות של סגל ההוראה) יהיה מוצפן ובלתי קריא לגורמים זדוניים.

פרוטוקול TCP/IP (Transmission Control Protocol / Internet Protocol): אלו הם פרוטוקולי התשתית (שכבות הרשת והתעבורה).

כאשר השרת סיים לחשב את מערכת השעות המורכבת, עליו לשלוח אותה בחזרה ללקוח. פרוטוקול IP דואג לנתב את המידע לכתובת הנכונה של המשתמש, בעוד פרוטוקול TCP חותך את המידע לחבילות (Packets) ומוודא באמינות מוחלטת שכל החבילות יגיעו בשלמותן, ללא שגיאות ובסדר הנכון. אם חבילת מידע "נפלת" בדרך, TCP דואג לשלוח אותה מחדש באופן אוטומטי, מה שמבטיח שמערכת השעות תוצג במלואה.

פורמט JSON (JavaScript Object Notation): אף על פי שאין זה פרוטוקול רשת מובהק, JSON הוא סטנדרט ייצוג הנתונים המרכזי שבו משתמשת המערכת. כאשר השרת מתקשר עם מסד הנתונים MongoDB בענן, כלל הישיויות (כגון קורס בעל שם, שעות ומזהה ייחודי) מומרות לאובייקטים בפורמט JSON. זהו טקסט קריא, קליל ומובנה, המאפשר העברה מהירה מאוד של מידע כבד בין ה-Backend למסד הנתונים, וחוסך משאבי רוחב פס (Bandwidth).

9.5 שירותים חיצוניים

במסגרת פיתוח המערכת האקדמית, הושם דגש על יצירת חוויית משתמש (UX) אישית, עשירה ורלוונטית לסטודנט. לשם כך, המערכת מבצעת אינטגרציה עם שירות חיצוני לאספקת נתוני מזג אוויר בזמן אמת, המשולב במסך הבית ובסרגל הניווט (NavBar) של אזור הסטודנט האישי.

שם השירות: OpenWeatherMap API

סוג השירות: REST API ענן.

1. מטרת השילוב (Use Case): המערכת דוגמת את מזג האוויר הנוכחי בעיר שבה ממוקם המוסד האקדמי (למשל, תל אביב), ועל בסיס נתונים אלו מציגה לסטודנט עם התחברותו למערכת חיווי טמפרטורה מדויק ו/או המלצות הגעה מותאמות אישית (כגון חובת הצטיידות בבגדים חמים בימים קרים, או בקבוק מים בימי שרב). פיצ'ר זה הופך את מערכת הניהול האקדמית ממערכת פונקציונלית ו"יבשה" למערכת אינטראקטיבית שאכפת לה מרווחת הסטודנט.

1. מימוש טכני וארכיטקטורה:

פרוטוקול תקשורת: התקשורת מול שרתי OpenWeatherMap מתבצעת בפרוטוקול HTTP מאובטח, תוך שימוש במתודת GET אל ה-Endpoint הייעודי של השירות.

צד השרת (Backend): בתוך סביבת Spring Boot, נבנה שירות ייעודי (WeatherService). השירות עושה שימוש במחלקת RestTemplate המובנית של Spring על מנת לשלוח את בקשות הרשת בצורה סינכרונית ולקרוא את התגובה.

עיבוד נתונים (Parsing): שרת מזג האוויר מחזיר את הנתונים במבנה נתונים מסוג JSON. כדי לחלץ את הטמפרטורה המדויקת מתוך העץ המורכב של התגובה, המערכת עושה שימוש בספריית

Jackson (באמצעות ObjectMapper ו-JsonNode), המאפשרת ניווט יעיל ובטוח אל השדה הרלוונטי.

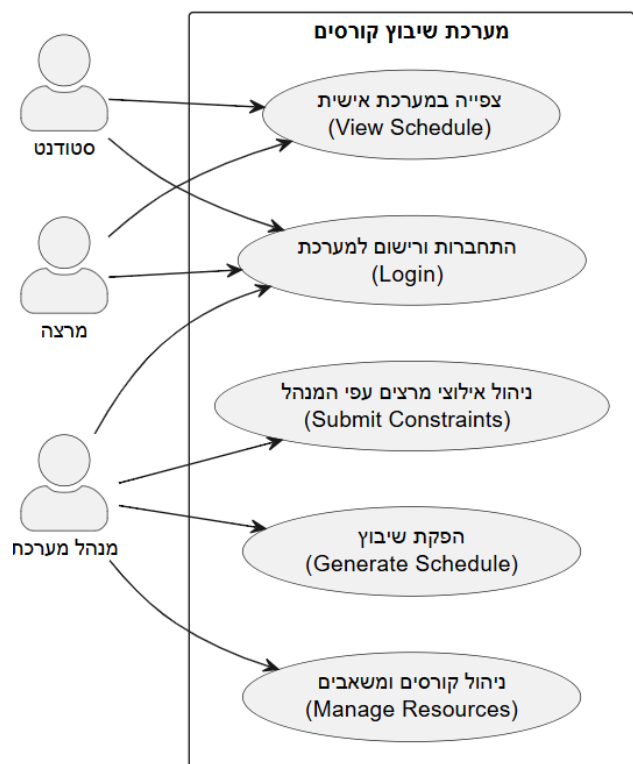
אימות ואבטחה (Authentication): הגישה ל-API מאובטחת ומנוהלת באמצעות מפתח גישה אישי (API Key) ייחודי לפרויקט, אשר משורשר כפרמטר בבקשת הרשת ומאמת את זהות המערכת מול שרתי הספק החיצוני.

- עמידות, יציבות וטיפול בשגיאות (Fault Tolerance & Fallback): מכיוון שהמערכת מסתמכת על צד שלישי שזמינותו אינה בשליטתנו, הוטמע מנגנון הגנה קריטי בקוד. כל קריאת הרשת עטופה במנגנון טיפול בשגיאות (try-catch). במקרה של כשל תקשורת, קריסת שרתי מזג האוויר, או בעיית הרשאות – המערכת תתפוס את החריגה (Exception) באופן סמוי ותחזיר "טמפרטורה ברירת מחדל" (Fallback value של 22 מעלות).
לוגיקה זו מבטיחה שרצף הפעולה של המערכת לא ייקטע, תהליך עליית מסך הסטודנט יושלם בהצלחה, והאפליקציה תשמור על יציבות (High Availability) ללא תלות מלאה בשירות החיצוני

10. ניתוח תרחישים וזרימת המידע במערכת המוצעת

10.1 תרשימי תרחיש Use case Diagram

להלן תרשימי use case של המערכת:



בתרשים אפשר לראות שיש כמה מודולים במערכת והם:

התחברות ורישום למערכת (Login): תהליך זה מאפשר לכלל המשתמשים להזדהות בצורה מאובטחת ולקבל גישה למערכת השיבוץ. ההתחברות מבטיחה שכל משתמש - סטודנט, מרצה או מנהל - יקבל הרשאות מתאימות וייחשף אך ורק למידע ולפעולות הרלוונטיים לתפקידו.

צפייה במערכת אישית (View Schedule): פונקציה המשמשת את הסטודנטים והמרצים כאחד לצורך צפייה במערכת השעות הסופית והמעודכנת שלהם. התצוגה מרכזת באופן ברור את זמני השיעורים והקורסים אליהם שובץ כל משתמש באופן אישי.

ניהול אילוצי מרצים עפ"י המנהל (Submit Constraints): תהליך המאפשר למנהל להזין את שעות הזמינות והעדפות המרצים. איסוף מדויק של האילוצים מהווה תשתית קריטית למנגנון השיבוץ על מנת למנוע התנגשויות זמנים.

הפקת שיבוץ (Generate Schedule): פעולת הליבה של המערכת, המבוצעת באופן בלעדי על ידי מנהל המערכת, ובה מיוצרת מערכת השעות המלאה של המוסד האקדמי. התהליך משקלל בצורה חכמה את אילוצי המרצים ודרישות הקורסים כדי לייצר שיבוץ אופטימלי ונטול חפיפות.

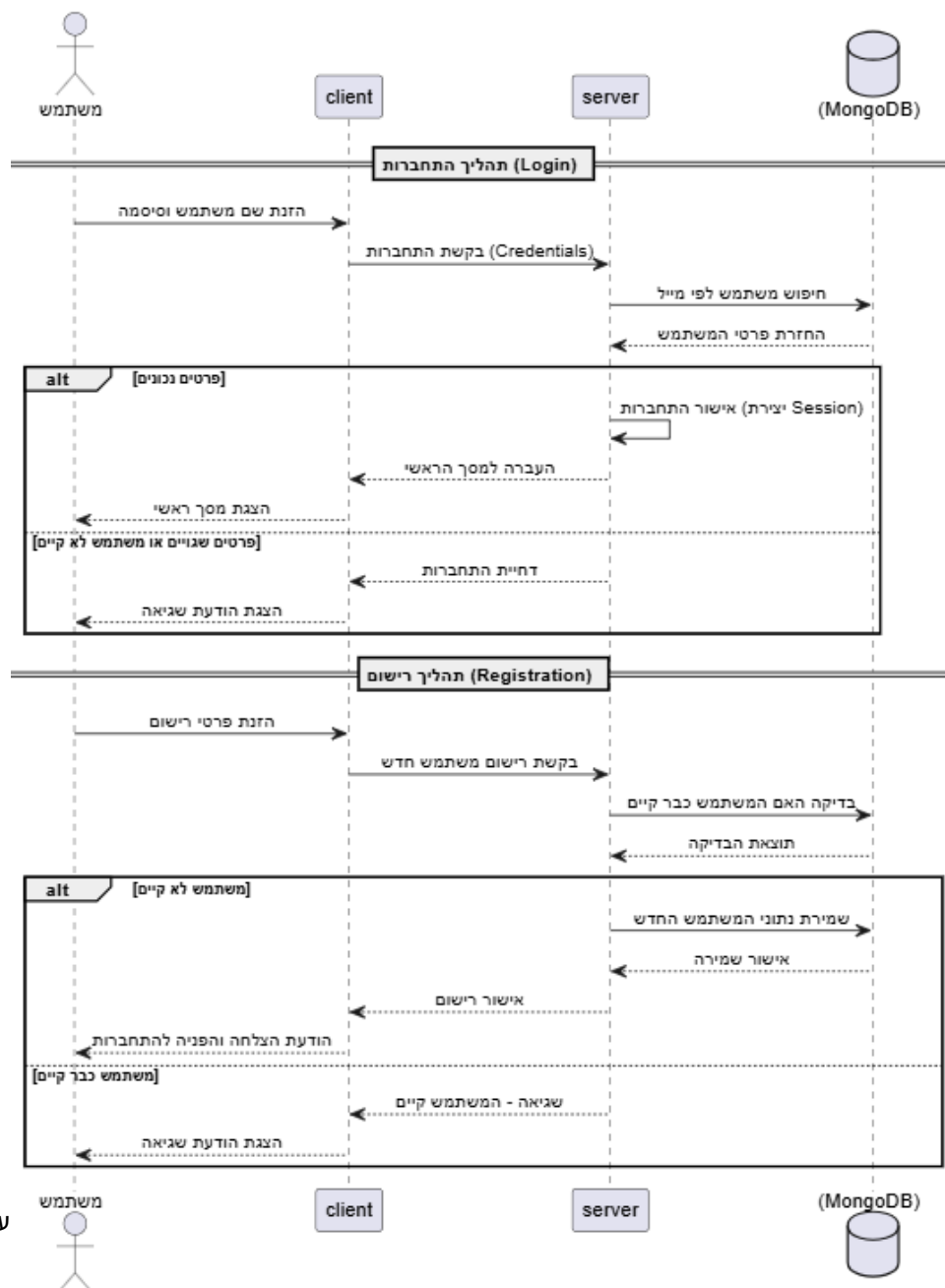
ניהול קורסים ומשאבים (Manage Resources): פעולת תשתית השמורה למנהל המערכת בלבד, הכוללת הוספה, עריכה או מחיקה של נתונים בסיסיים כגון קורסים ומשאבים פיזיים. הגדרת משאבים אלו במערכת הכרחית כדי לספק לאלגוריתם השיבוץ את בסיס הנתונים הנדרש לפעולתו.

10.2 תרשימי רצף Sequence Diagram

תרשים login & register

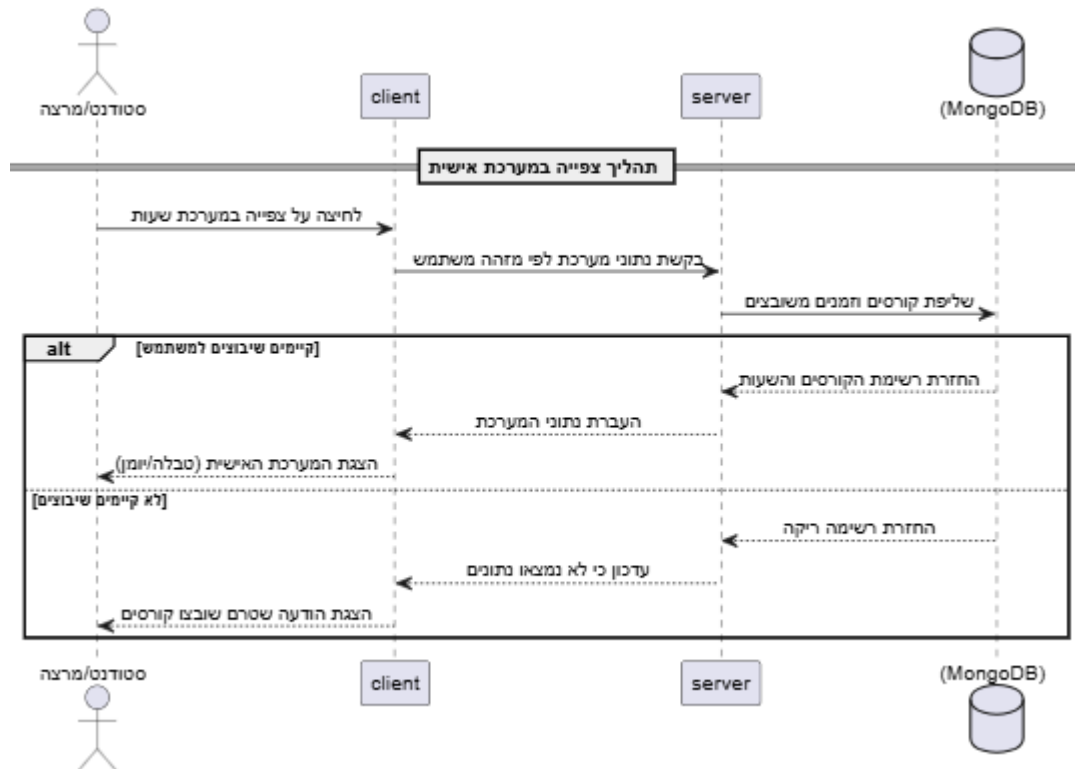
התרשים המוצג הוא תרשים שמתאר התחברות ורישום שמבוסס על שני חלקים: החלק הראשון מציג את תהליך ההתחברות למערכת. התהליך מתחיל כאשר המשתמש מזין את פרטי ההזדהות שלו אל תוך ממשק הלקוח. צד הלקוח מעביר את הבקשה אל השרת, אשר פונה למסד הנתונים כדי לחפש את המשתמש על פי המייל שלו ולשלוח את פרטיו. בשלב זה המערכת מתפצלת לשני תרחישים אפשריים באמצעות בלוק תנאי. בתרחיש של הזנת פרטים נכונים, השרת מאמת את הנתונים, יוצר פגישת משתמש מאובטחת, ומנחה את הלקוח להציג למשתמש את המסך הראשי. לעומת זאת, אם הפרטים שהוזנו שגויים או שהמשתמש אינו קיים, השרת דוחה את בקשת ההתחברות ומורה ללקוח להציג הודעת שגיאה מתאימה למשתמש.

החלק השני מפרט את תהליך הרישום של משתמש חדש. המשתמש מזין את פרטיו האישיים בממשק הלקוח, אשר מעביר בקשת רישום אל השרת. השרת פונה תחילה למסד הנתונים כדי לוודא



שמשמש עם נתונים אלו אינו רשום כבר במערכת. גם כאן קיימת התפצלות לשני מצבים התלויים בתוצאת הבדיקה. במידה והמשתמש אינו קיים, השרת שומר את הנתונים החדשים במסד הנתונים. לאחר קבלת אישור על השמירה, השרת שולח אישור רישום ללקוח, המציג למשתמש הודעת הצלחה ומפנה אותו חזרה למסך ההתחברות. אם המשתמש כבר קיים במערכת, תהליך השמירה נעצר, השרת מחזיר שגיאה ללקוח, וזה מציג למשתמש הודעה המיידעת אותו על הכפילות.

תרשים צפייה במערכת אישית:



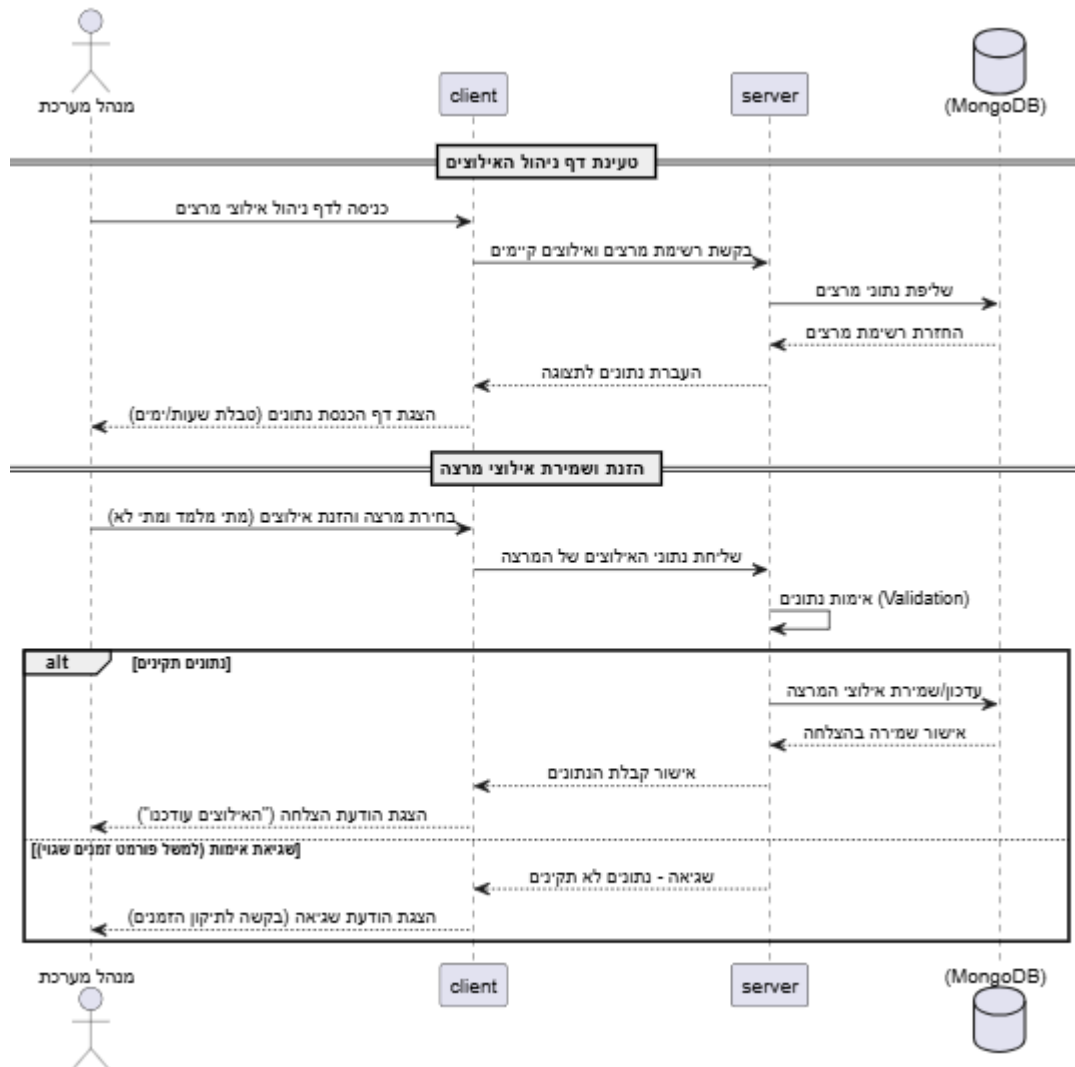
בתרשים זה אפשר לראות שהתהליך מתחיל כאשר הסטודנט או המרצה מבצעים לחיצה על אפשרות הצפייה במערכת השעות דרך ממשק ה-client. בעקבות הפעולה, ה-client יוצר בקשה לקבלת נתוני המערכת ושולח אותה אל ה-server, תוך שימוש במזהה משתמש ספציפי כדי למקד את הבקשה. ה-server מקבל את פניית הלקוח וניגש ישירות אל מסד הנתונים MongoDB במטרה לשלוף משם את כלל הקורסים והזמנים המשובצים התואמים לאותו מזהה.

בשלב קבלת התשובה ממסד הנתונים, המערכת מתפצלת לשני תרחישים אפשריים המיוצגים באמצעות בלוק תנאי. התרחיש הראשון מתאר מצב בו אכן קיימים שיבוצים למשתמש. במקרה זה, ה-MongoDB מחזיר ל-server את רשימת הקורסים והשעות המלאה. לאחר מכן, ה-server מעביר את נתוני המערכת המעובדים בחזרה אל ה-client, אשר מציג לסטודנט או למרצה את המערכת האישית שלהם בצורה חזותית ונוחה, כגון טבלה או יומן.

התרחיש השני והחלופי מציג מצב שבו לא קיימים כלל שיבוצים עבור אותו משתמש. במקרה כזה, מסד הנתונים מחזיר רשימה ריקה אל ה-server, אשר מבין כי אין נתונים להציג ומעביר עדכון מתאים אל ה-client.

ה-client מקבל את העדכון ומציג לסטודנט או למרצה הודעה ברורה המיידעת אותם שטרם שובצו עבורם קורסים, ובכך משלים את זרימת המידע ללא הצגת מערכת ריקה.

ניהול אילוצי מרצים עפ"י מנהל:



תרשים רצף זה מתאר את תהליך העבודה במערכת בעת ניהול והגדרת אילוצי זמן של מרצים. התרשים מציג את האינטראקציה הרציפה בין מנהל המערכת, ממשק צד הלקוח, שרת המערכת ומסד הנתונים. התהליך כולו מחולק לשני שלבים מרכזיים: טעינת דף ניהול האילוצים, ולאחר מכן הזנת הנתונים ושמירתם במערכת.

בשלב הראשון, המערכת מכינה את התצוגה עבור מנהל המערכת. התהליך מתחיל כאשר מנהל המערכת יוזם כניסה לדף ניהול אילוצי מרצים דרך ממשק הלקוח. בעקבות זאת, צד הלקוח שולח בקשה לשרת לקבלת רשימת המרצים והאילוצים הקיימים. השרת פונה אל מסד הנתונים כדי לשלוף את המידע הרלוונטי. לאחר שהנתונים מוחזרים ממסד הנתונים לשרת, הם מועברים חזרה לצד הלקוח, אשר מעבד אותם ומציג למנהל המערכת טופס מובנה להזנת הנתונים הכולל טבלת שעות וימים.

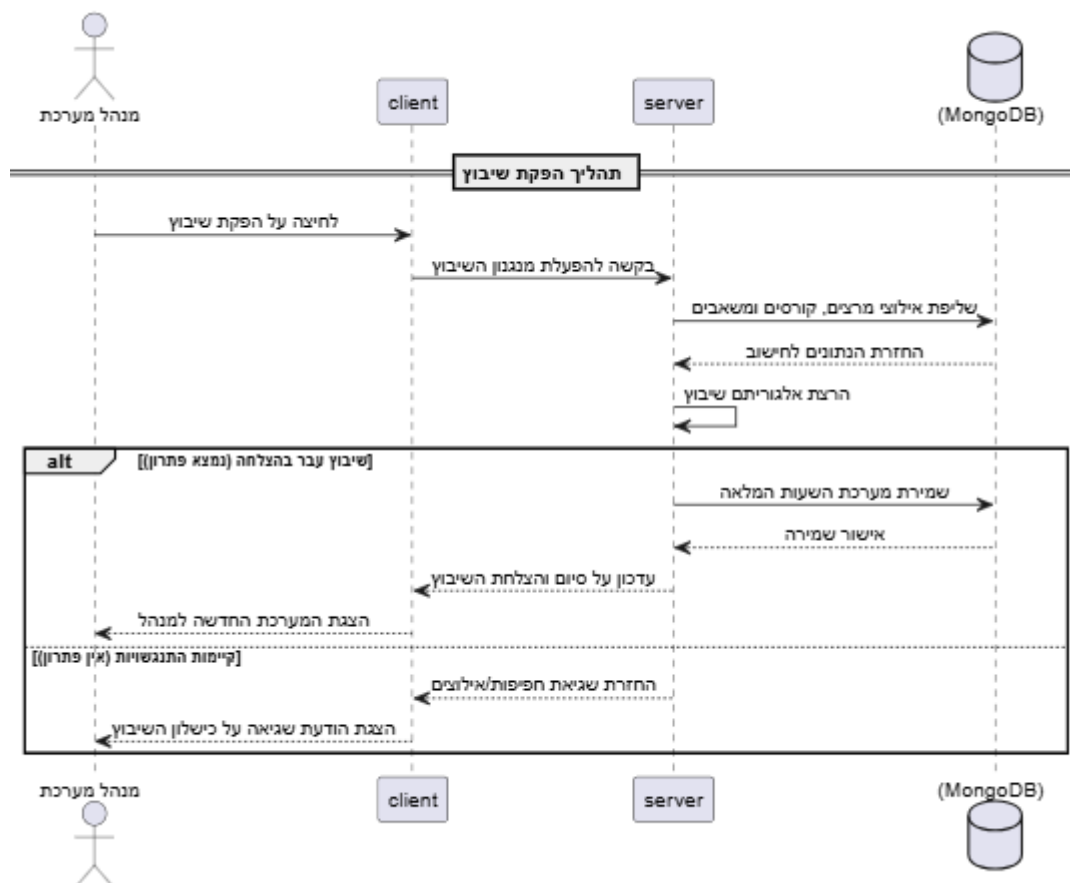
בשלב השני, מנהל המערכת בוחר מרצה מתוך הרשימה שהוצגה ומזין את אילוצי הזמן שלו, כלומר באילו שעות וימים הוא זמין או אינו זמין ללמד. לאחר ההזנה, נתוני האילוצים נשלחים מצד הלקוח אל

השרת לשם עיבוד. בנקודה זו, השרת מבצע פעולה פנימית של אימות נתונים על מנת לוודא שהזמנים שהוזנו תקינים, הגיוניים, ואינם חופפים או שגויים.

התרשים מציג פיצול בתהליך בהתאם לתוצאת אימות הנתונים. בתרחיש שבו הנתונים נמצאו תקינים, השרת שולח פקודה למסד הנתונים לעדכן ולשמור את אילוצי המרצה. לאחר קבלת אישור ממסד הנתונים על הצלחת השמירה, השרת מעביר את האישור לצד הלקוח, וזה מציג למנהל המערכת הודעת הצלחה המאשרת שהאילוצים עודכנו.

לעומת זאת, בתרחיש שבו מתגלית שגיאת אימות, כגון פורמט זמנים שגוי, השרת עוצר את התהליך ולא פונה למסד הנתונים. במקרה כזה, השרת מחזיר הודעת שגיאה לצד הלקוח, אשר מציג למנהל המערכת הודעה מתאימה בליווי בקשה לתקן את הזמנים שהוזנו בטרם ניסיון שמירה נוסף. תהליך זה מבטיח חוויית משתמש רציפה ושמירה קפדנית על שלמות הנתונים.

הפקת שיבוץ:



תרשים הרצף מתאר את השלבים המרכזיים בתהליך הפקת שיבוץ מערכת השעות. התרשים מציג את זרימת המידע והפעולות בין מנהל המערכת, השרת ומסד הנתונים, במטרה לייצר מערכת שעות תקינה העונה על כלל האילוצים. התהליך מתחיל בפעולה יזומה של מנהל המערכת, אשר לוחץ על כפתור הפקת השיבוץ דרך ממשק הלקוח.

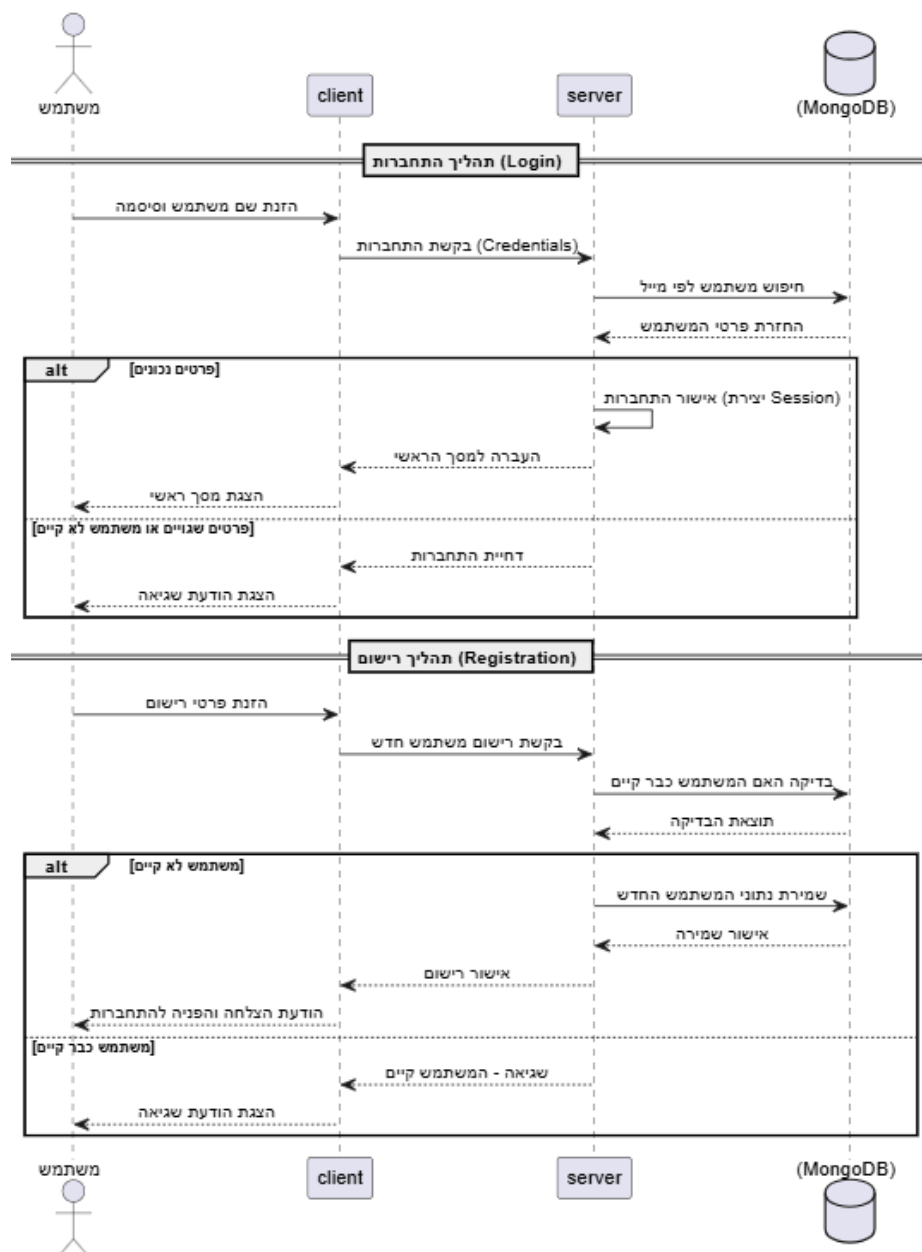
בעקבות לחיצה זו, צד הלקוח שולח בקשה לשרת להפעלת מנגנון השיבוץ. השרת פונה אל מסד הנתונים כדי לשלוף את כלל הנתונים ההכרחיים לחישוב, הכוללים את אילוצי המרצים, הקורסים והמשאבים

הקיימים. לאחר שמסד הנתונים מחזיר את הנתונים הנדרשים, השרת מתחיל להריץ באופן פנימי את אלגוריתם השיבוץ על מנת למצוא פתרון שיבוץ תקין.

בשלב זה, התרשים מתפצל לשני תרחישים אפשריים בהתאם לתוצאות ריצת האלגוריתם. בתרחיש הראשון, בו השיבוץ עבר בהצלחה ונמצא פתרון מתאים, השרת שולח פקודה למסד הנתונים לשמור את מערכת השעות המלאה שנוצרה. לאחר קבלת אישור השמירה ממסד הנתונים, השרת מעדכן את צד הלקוח על סיום והצלחת תהליך השיבוץ. לבסוף, צד הלקוח מציג למנהל המערכת את מערכת השעות החדשה.

בתרחיש השני, בו קיימות התנגשויות והאלגוריתם לא הצליח למצוא פתרון, השרת מחזיר אל צד הלקוח הודעת שגיאה המפרטת את החפיפות או האילוצים הבעייתיים. בעקבות זאת, צד הלקוח מציג למנהל המערכת הודעת שגיאה על כישלון השיבוץ, מה שמאפשר לו להבין את הבעיה ולבצע את ההתאמות הנדרשות לפני ניסיון הפקה נוסף של המערכת.

ניהול קורסים ומשאבים:



תרשים הרצף מתאר את תהליך העבודה במערכת בעת ניהול קורסים ומשאבים. התהליך כולו מחולק לשני שלבים עיקריים: טעינת עמוד הניהול, ולאחר מכן הוספה או עדכון של קורס או משאב. בשלב הראשון, מנהל המערכת יוזם כניסה לעמוד ניהול קורסים ומשאבים דרך צד הלקוח. בעקבות זאת, צד הלקוח שולח בקשה לשרת לקבלת רשימת הקורסים, הכיתות והמשאבים הקיימים.

השרת פונה אל מסד הנתונים כדי לשלוף את המידע. לאחר שהנתונים מוחזרים ממסד הנתונים לשרת, הם מועברים חזרה לצד הלקוח, אשר מציג למנהל המערכת את טבלת הקורסים והמשאבים הקיימים במערכת.

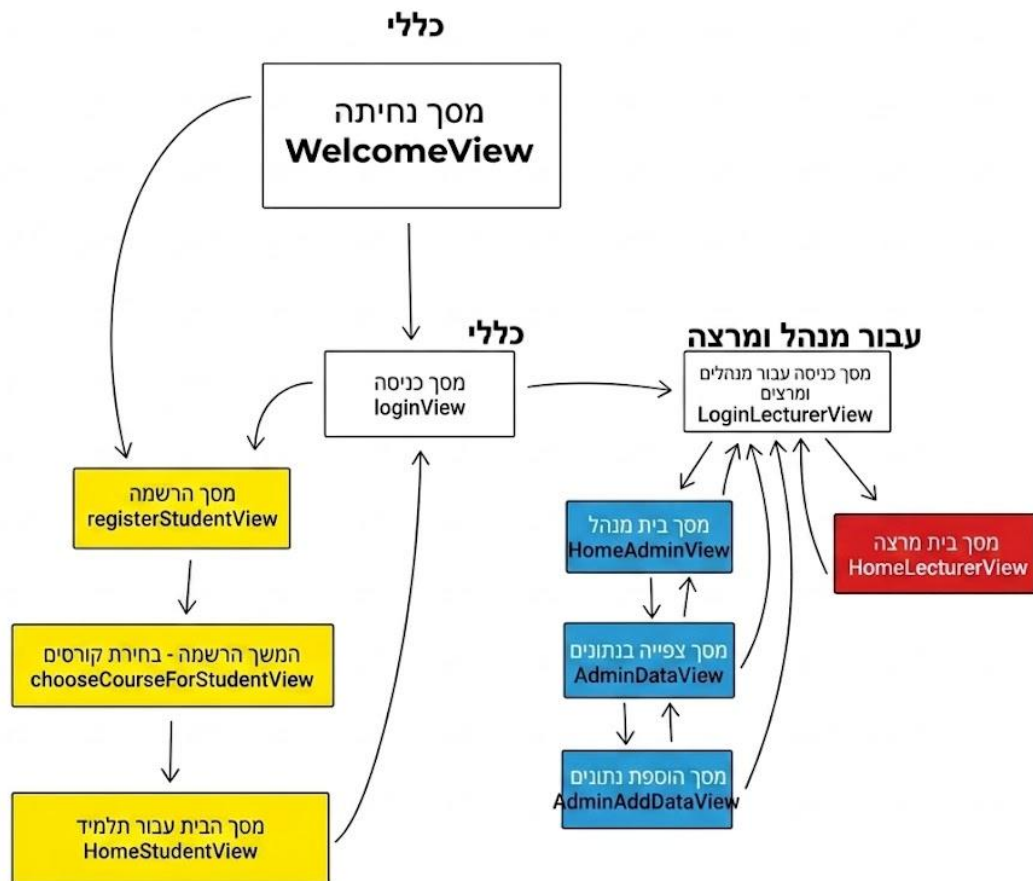
בשלב השני, מתבצע תהליך של הוספת קורס או משאב חדש. מנהל המערכת מזין את פרטי הקורס או המשאב, כגון שם הקורס ודרישות הכיתה, דרך הממשק. צד הלקוח שולח את הנתונים החדשים אל השרת לשם עיבוד. בנקודה זו, השרת מבצע פעולה פנימית של אימות נתונים, שנועדה לבדוק, בין היתר, האם קוד הקורס שהוזן כבר קיים במערכת כדי למנוע כפילויות.

מכאן, התרשים מציג פיצול בתהליך בהתאם לתוצאת אימות הנתונים. בתרחיש שבו הנתונים נמצאו תקינים ואין כפילויות, השרת שולח פקודה למסד הנתונים לשמור או לעדכן את נתוני הקורס או המשאב. לאחר קבלת אישור ממסד הנתונים על הצלחת השמירה, השרת מעביר את האישור לצד הלקוח, המציג למנהל המערכת הודעת הצלחה ומרענן את הרשימה המוצגת. לעומת זאת, בתרחיש שבו מתגלית שגיאת אימות או מזהה כפול, השרת עוצר את התהליך ומחזיר הודעת שגיאה לצד הלקוח, המציינת כי הקורס כבר קיים או שחסרים נתונים. בעקבות זאת, צד הלקוח מציג למנהל המערכת הודעת שגיאה, מה שמאפשר לו לתקן את הפרטים שהוזנו.

11. תיאור מסכים וממשק משתמש

11.1 תרשים היררכיית המסכים

להלן תרשים המתאר את מעבר המסכים בפרויקט:



תרשים זה מציג את ארכיטקטורת הניווט (Navigation Flow) של המערכת, ומתאר את מסלולי המעבר בין המסכים השונים בהתאם לסוג המשתמש: תלמיד שמופיע בצהוב, מרצה שמופיע באדום או מנהל שמופיע בכחול.

הכניסה למערכת מתבצעת דרך מסך הכניסה הראשי (loginView), ממנו מתפצלת זרימת המשתמשים לשני נתיבים עיקריים: נתיב המיועד לתלמידים ונתיב המיועד לסגל המערכת.

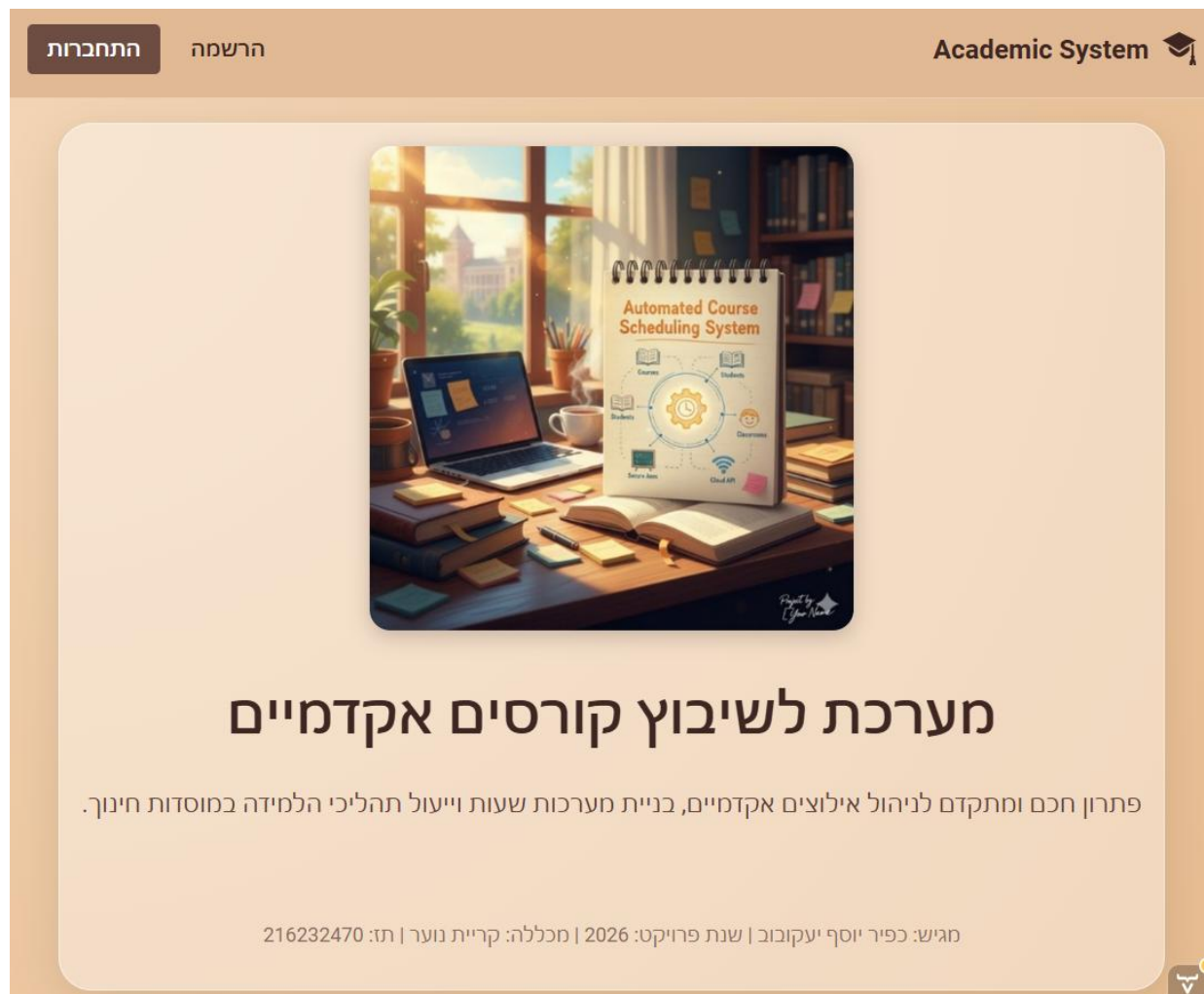
בנתיב התלמידים, תלמיד חדש מופנה למסך ההרשמה (registerStudentView), ומיד לאחריו למסך המשך ההרשמה שבו מתבצעת בחירת הקורסים (chooseCourseForStudentView). בסיום תהליך הקליטה, התלמיד מגיע למסך הבית הייעודי עבורו (HomeStudentView). ממסך זה קיימת אפשרות חזרה (התנתקות) למסך הכניסה הראשי.

בנתיב הסגל, אנשי צוות מופנים ממסך הכניסה הראשי למסך הזדהות ייעודי עבור מנהלים ומרצים (LoginLecturerView). לאחר ההזדהות, המערכת מנתבת את המשתמשים על פי הרשאותיהם: מרצים מועברים ישירות למסך הבית למרצים (HomeLecturerView), בעוד שמנהלים מועברים למסך הבית למנהל (HomeAdminView).

עבור מנהלי המערכת תוכנן ניווט פנימי המאפשר ניהול מידע שוטף. ממסך הבית של המנהל ניתן לעבור למסך צפיית הנתונים (AdminDataView), וממנו להמשיך למסך הוספת נתונים (AdminAddDataView). המעברים בין מסכי צפיית הנתונים, הוספת הנתונים ומסך הבית של המנהל הם דו-כיווניים, מה שמאפשר עבודה רציפה. כמו כן, מכל אחד ממסכי הניהול הללו קיימת אפשרות חזרה ישירה אל מסך הכניסה של הסגל.

11.2 תיאור המסכים

מסך נחיתה:



- מטרת המסך (תפקידו):

מסך זה הוא דף הנחיתה (Landing Page) הראשוני של המערכת. הוא מהווה את נקודת המפגש הראשונה של המשתמשים עם המערכת ומטרתו להציג אותה, להסביר את תפקידה

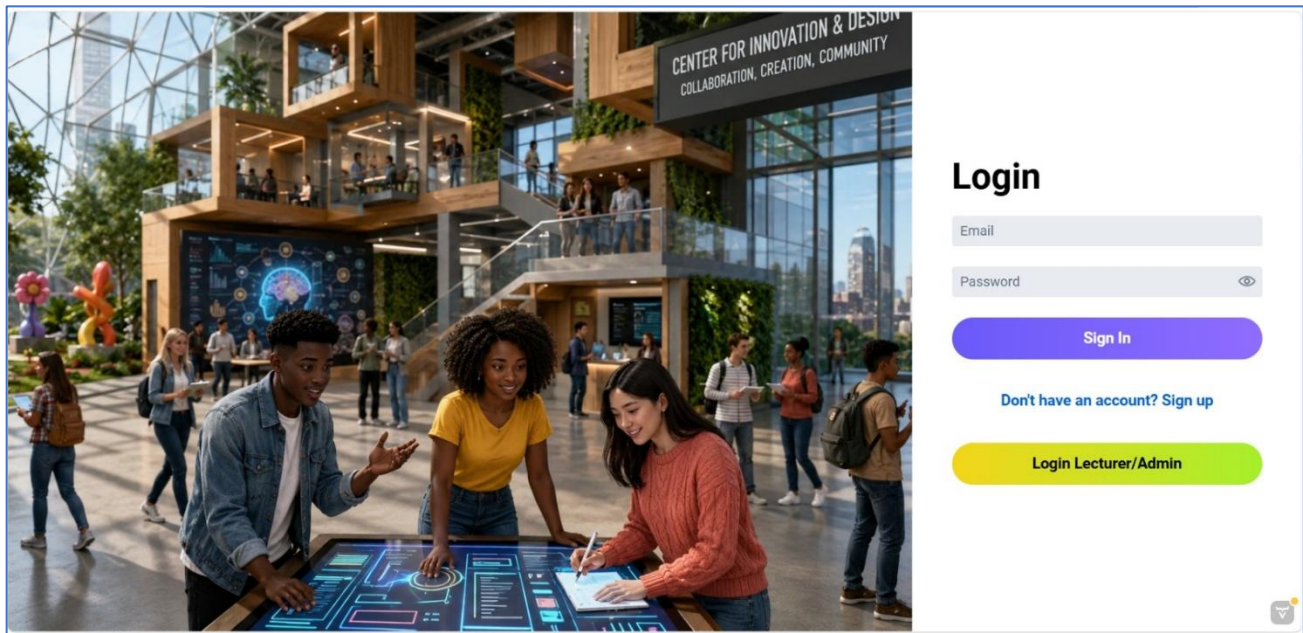
בקצרה, ולספק שתי אפשרויות קריאה לפעולה מרכזיות: כניסה למשתמשים רשומים או הרשמה למשתמשים חדשים. העיצוב משדר חדשנות, ידידותיות ומקצועיות, תוך שימוש בתמונת קמפוס מודרנית ובממשק משתמש (UI) נקי ומזמין על גבי פאנל מרכזי שקוף-למחצה.

- רכיבי ממשק משתמש (UI Elements) ותפקידם:
 - לוגו וכותרת: אייקון הלוגו של המערכת והכיתוב "Academic System". דברים אלו מציגים את זהות המערכת למשתמש.
 - כותרת כללית במרכז המסך: מגדירה בבירור את מהות המערכת עבור המשתמש.
 - טקסט הסבר: פסקה קצרה המסבירה בעברית את מטרת המערכת.
 - תמונה: תמונה באמצע המסך התורמת למראה המודרני וה"שמח" של המערכת ומשלימה את החוויה הויזואלית.
 - כפתור "כניסה / Log in": כפתור קריאה לפעולה ראשית, לחיצה עליו תעביר את המשתמש למסך ההתחברות (Login) הסטנדרטי.
 - כפתור "הירשם / Sign up": כפתור ראשי נוסף. לחיצה עליו תעביר את המשתמש למסך ההרשמה למערכת.

נאב בר (סרגל ניווט) עבור כניסה ראשונית (מסך נחיתה):



- מטרת סרגל הניווט (תפקידו):
 - משמש כסרגל הניווט העליון של מסך הבית/קבלת הפנים של המערכת (Welcome Page).
 - מטרתה להציג את מיתוג המערכת (לוגו) ולהכווין משתמשים שאינם מחוברים לפעולות המרכזיות של כניסה למערכת - התחברות (Login) או הרשמה (Register).
- רכיבי ממשק משתמש (UI Elements) ותפקידם:
 - תמונה של כובע אקדמאי הממוקמת לצד שם המערכת.
 - רכיב טקסט המציג את שם המערכת - "Academic System" מודגש.
 - שני כפתורי ניווט - כפתור "התחברות" (מנווט למסך "login") וכפתור "הרשמה" (מנווט למסך "register") בעיצובים שונים.

מסך login - התחברות:

- מטרת המסך (תפקידו):

מסך זה מהווה את שער הכניסה הראשי למערכת האקדמית עבור הסטודנטים. תפקידו לאמת את זהות המשתמשים (Authentication) על בסיס כתובת אימייל (אשר משמשת כמזהה ייחודי במערכת למניעת כפילויות) וסיסמה, ולאפשר להם גישה מאובטחת לאזור האישי שלהם. בנוסף, המסך משמש כצומת ניווט המפנה משתמשים חדשים להרשמה, ואנשי סגל למסך כניסה ייעודי. העיצוב משלב תמונה צבעונית ומוארת של סטודנטים בסביבת קמפוס מודרנית וחדשנית, במטרה להעניק לממשק המשתמש תחושה חיובית, מזמינה ומעוררת השראה.

- רכיבי ממשק משתמש (UI Elements) ותפקידם:

➤ תמונת אווירה (Image): תמונה של מרכז חדשנות בקמפוס אקדמי התופסת את החצי השמאלי של המסך. נועדה לשפר את חוויית המשתמש (UX) ולהעניק הקשר ויזואלי התואם לאופי המערכת.

➤ כותרת Login - (Heading): טקסט המגדיר למשתמש את מטרתו של הדף הנוכחי.

➤ תיבת טקסט Email - (Text Field): שדה הזנה בו המשתמש נדרש להקליד את כתובת הדוא"ל שלו.

➤ תיבת סיסמה Password - (Password Field): שדה הזנה מאובטח (בו התווים מוסתרים) עבור סיסמת המשתמש. שדה זה כולל אייקון של עין (Toggle Visibility) המאפשר למשתמש להציג או להסתיר את תווי הסיסמה שהוקלדו לשם נוחות ווידוא הקלדה נכונה.

➤ כפתור פעולה ראשי Sign In - (Primary Button): כפתור (בצבע סגול) שלחיצה עליו מפעילה את פונקציית ההתחברות. הכפתור שולח את נתוני ההזנה לשרת לצורך אימות מול מסד הנתונים. במידה והפרטים מאומתים בהצלחה, המשתמש מועבר למסך הבית.

➤ כפתור Sign up: טקסט לחיץ המפנה משתמשים שטרם נרשמו למערכת אל מסך יצירת חשבון חדש.

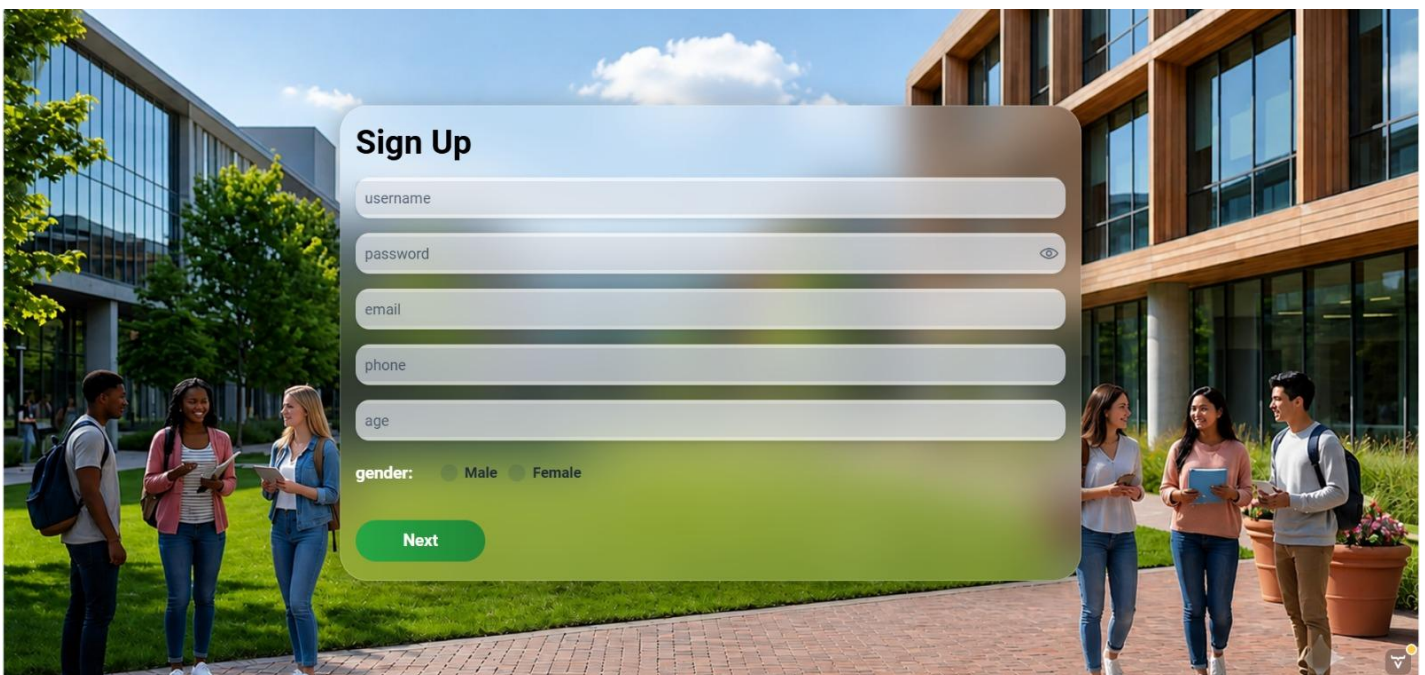
➤ כפתור ניווט Login Lecturer/Admin - (Navigation Button): כפתור (בצבע צהוב) המיועד לאנשי סגל האקדמי ומנהלי המערכת. לחיצה עליו לא מבצעת התחברות במסך זה, אלא מפנה את המשתמש למסך התחברות נפרד המותאם להרשאות הניהול וההוראה.

- הודעות ודיאלוגים (Messages & Dialogs):

➤ הודעת שגיאה על פרטים שגויים (Invalid Credentials): במידה והמשתמש מזין אימייל או סיסמה שאינם קיימים במסד הנתונים או שאינם תואמים, תוקפץ למשתמש הודעה (Notification/Toast) המיידעת אותו שהפרטים שגויים ומבקשת ממנו לנסות שנית.

➤ הודעת ולידציה על שדות ריקים (Validation Error): במידה והמשתמש לוחץ על כפתור "Sign In" מבלי להזין את שני השדות, תופיע הודעה או שגיאה תחת שדה ההזנה הרלוונטי, המנחה אותו למלא את כל שדות החובה טרם שליחת הטופס

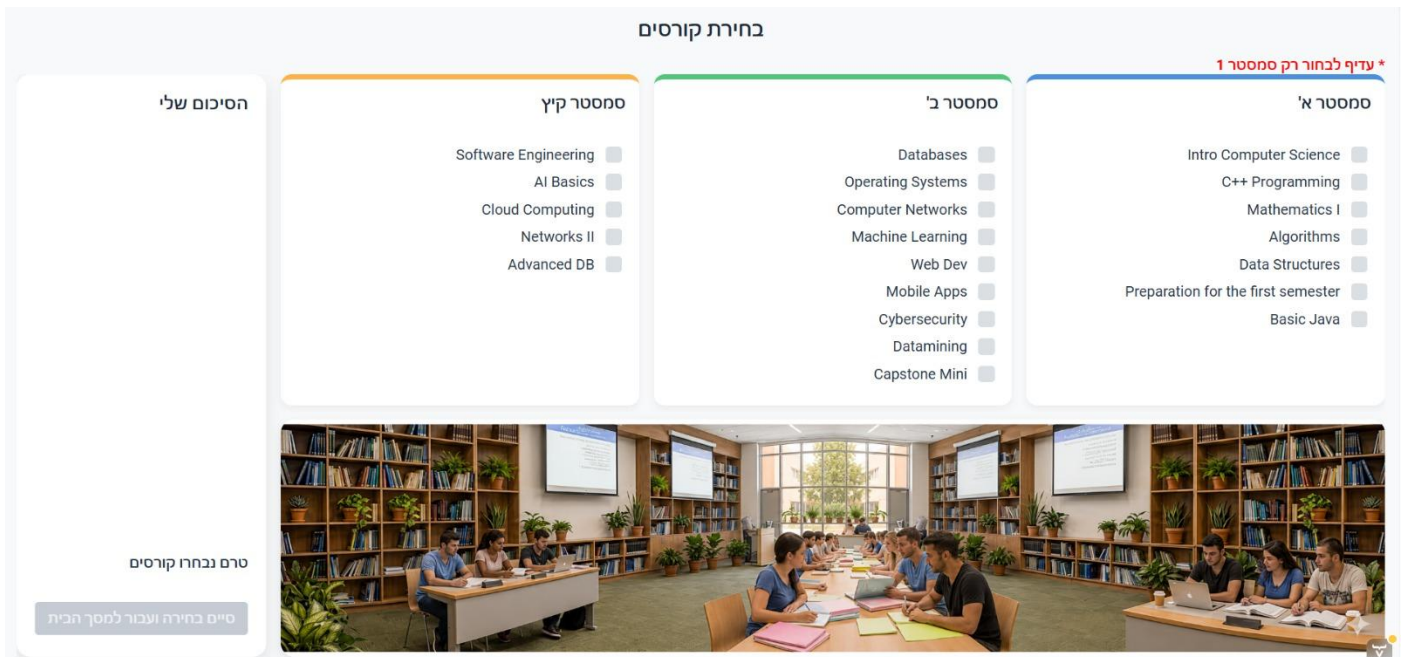
מסך register - רישום:



- מטרת המסך (תפקידו):

מסך זה מיועד לרישום סטודנטים חדשים למערכת האקדמית וליצירת פרופיל משתמש. המסך נפתח לאחר לחיצה על קישור ההרשמה (Sign up) במסך ההתחברות. מטרתו היא לאסוף את נתוני הבסיס של הסטודנט, כגון פרטי התקשרות ונתונים דמוגרפיים, בתהליך הזנה פשוט וברור. העיצוב ממשיך את הקו החזותי של המערכת, עם רקע חי של קמפוס מואר באור יום וסטודנטים, ומציג את טופס ההרשמה.

- רכיבי ממשק משתמש (UI Elements) ותפקידם:
 - כותרת Sign Up - (Heading): כותרת ראשית בראש החלונית המבהירה למשתמש שהוא נמצא בתהליך יצירת חשבון חדש.
 - תיבת טקסט username - (Text Field): שדה להזנת שם המשתמש.
 - תיבת סיסמה password - (Password Field): שדה מאובטח להזנת הסיסמה הרצויה.
 - שדה זה כולל אייקון של עין (Toggle Visibility) המאפשר למשתמש לראות את התווים שהקליד כדי למנוע שגיאות הקלדה בשלב יצירת הסיסמה.
 - תיבת טקסט email - (Text Field): שדה להזנת כתובת הדואר האלקטרוני של הסטודנט.
 - שדה זה קריטי במיוחד, שכן כתובת האימייל תשמש כמזהה הייחודי של המשתמש לצורך התחברות עתידית למערכת (במקום שימוש בשם משתמש שיכול להיות כפול).
 - תיבת טקסט phone - (Text Field): שדה להזנת מספר הטלפון הנייד של הסטודנט ליצירת קשר.
 - תיבת טקסט age - (Text Field): שדה מספרי להזנת גיל הסטודנט, אשר נשמר כחלק ממודל הנתונים של הסטודנט במערכת.
 - כפתורי רדיו gender - (Radio Buttons): רכיב בחירה המאפשר לסטודנט לסמן את המגדר שלו (Male / Female). רכיב זה מאפשר בחירה של אפשרות אחת בלבד בכל פעם.
 - כפתור פעולה ראשי Next - (Primary Button): כפתור (בצבע ירוק) לאישור הנתונים שהוזנו במסך זה. מכיוון שהכפתור נקרא "Next", הוא מעיד על כך שתהליך ההרשמה מורכב ממספר שלבים, ולחיצה עליו (לאחר ולידציה) תעביר את המשתמש למסך הבא להשלמת קליטתו במערכת.
- הודעות ודיאלוגים (Messages & Dialogs):
 - הודעת שגיאה על אימייל קיים (Duplicate Email Error): במידה והסטודנט מזין כתובת אימייל שכבר קיימת במסד הנתונים של המערכת, תקפוץ הודעה המתריעה כי המשתמש כבר רשום ותציע לו לעבור למסך ההתחברות (Login).
 - הודעות ולידציה על תקינות נתונים (Format Validation): בעת לחיצה על "Next", המערכת בודקת את תקינות השדות. באם ישנן חריגות (למשל: כתובת אימייל ללא @, סיסמה קצרה מדי, או הזנת אותיות בשדה הגיל/טלפון), תופיע הודעת שגיאה נקודתית מתחת לשדה הרלוונטי או כהודעה קופצת (Toast), המבקשת לתקן את הקלט.
 - הודעה על שדות חסרים (Missing Required Fields): במידה והושארו שדות חובה ריקים, המערכת לא תאפשר מעבר לשלב הבא ותסמן למשתמש באדום את השדות שעליו למלא.

מסך בחירת הקורסים - המשך רישום:

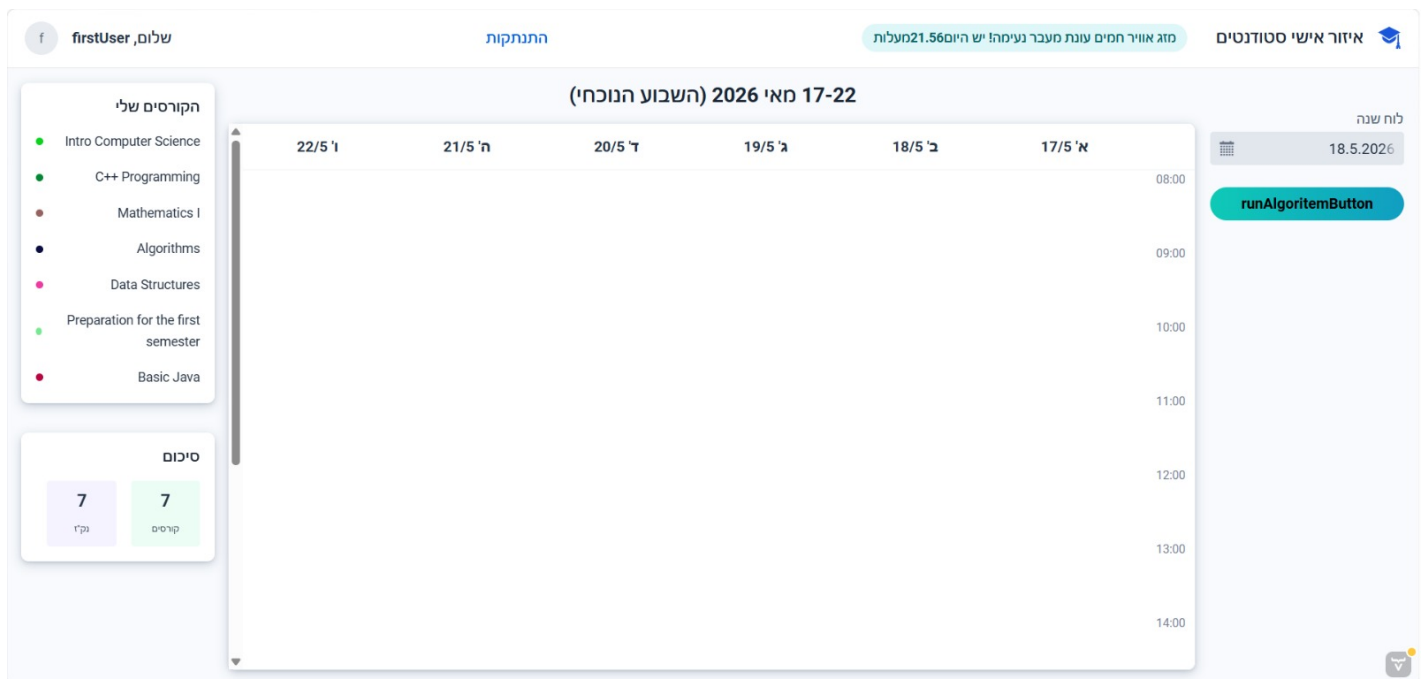
- מטרת המסך (תפקידו):

מסך זה מוצג לסטודנט מיד לאחר השלמת תהליך ההרשמה הראשוני (או עם כניסתו הראשונה למערכת), ומטרתו לאפשר לו להרכיב את מערכת השעות ולבחור את הקורסים אליהם הוא מעוניין להירשם. המסך מציג את כלל הקורסים הזמינים למחזור הלימודים, בחלוקה ברורה לסמסטרים (א', ב' וק"ץ). תצוגת המסך מתוכננת ומותאמת באופן מלא לתמיכה בכיווניות ימין-לשמאל (RTL), כך שהסמסטר הראשון מופיע בצד ימין, ופאנל הסיכום מוקם כרכיב האחרון בצד השמאלי ביותר, מה שיוצר זרימת קריאה ועבודה טבעית למשתמש בעברית. עיצוב המסך ממשיך את הקו החזותי הבהיר, וכולל בתחתיתו תמונת אווירה רחבה של סביבת למידה משותפת.

- רכיבי ממשק משתמש (UI Elements) ותפקידם:

- כותרת עליונה (Main Heading): "בחירת קורסים" - מגדירה בבירור את הפעולה הנדרשת במסך זה.
- טקסט הכוונה/המלצה (Guidance Text): כיתוב באדום בפינה הימנית העליונה ("עדיף לבחור רק סמסטר 1"). רכיב זה משמש להנחיית משתמשים חדשים למנוע עומס או שגיאות עתידיות בבניית המערכת.
- פאנלים של סמסטרים (Semester Cards/Panels): שלוש עמודות המייצגות את סמסטר א', ב' וק"ץ. כל עמודה מתפקדת כמכולה (Container) המאגדת בתוכה את רשימת הקורסים המוצעים באותו סמסטר.

- תיבות סימון (Checkboxes): מופיעות לצד כל שם של קורס ומשמשות לבחירה מרובה (Multiple Selection). סימון התיבה (V) בוחר את הקורס, והסרת הסימון מבטלת את הבחירה.
- פאנל "הסיכום שלי" (Summary Panel): אזור הממוקם בצדו השמאלי של המסך. זהו רכיב דינמי שתפקידו לרכז ולהציג למשתמש בזמן אמת את כל הקורסים שהוא סימן בתיבות הבחירה, על מנת שיוכל לסקור את הרשימה המלאה במקום אחד.
- טקסט מצב (Status Text): הכיתוב "טרם נבחרו קורסים" המופיע בפאנל הסיכום כברירת מחדל, כל עוד המשתמש לא סימן אף קורס. ברגע שתבצע בחירה, טקסט זה יספור את מספר הקורסים שנוספו לרשימה.
- כפתור פעולה ראשי - "סיים בחירה ועבור למסך הבית": כפתור המאשר את סיום תהליך הבחירה. בצילום המסך הנוכחי הכפתור במצב מושבת (Disabled - צבוע באפור) מכיוון שהמערכת מזהה שטרם נבחרו קורסים. עם בחירת קורס אחד לפחות, הכפתור יופעל (Enabled) ולחיצה עליו תשמור את הנתונים במסד הנתונים ותנתב את הסטודנט למסך הבית.
- תמונת אווירה רחבה (Banner Image): תמונה פנורמית בתחתית המסך התורמת למראה המודרני של המערכת ומשלימה את החוויה הויזואלית.
- הודעות ודיאלוגים (Messages & Dialogs):
 - דיאלוג אישור סיום בחירה: אחרי שהסטודנט בוחר קורסים ועובר למסלך הבית (עליו ממש אסביר עכשיו) קופץ הודעה של "ההרשמה בוצעה בהצלחה"

דף הבית / אזור אישי סטודנטים:

- מטרת המסך (תפקידו):

מסך זה מהווה את מרכז הבקרה הראשי של הסטודנט במערכת. הוא מוצג לאחר התחברות מוצלחת או לאחר סיום שלב בחירת הקורסים. מטרתו המרכזית היא להציג לסטודנט תמונת מצב מקיפה על הסטטוס האקדמי שלו: הקורסים שבחר, סיכום נקודות הזכות, ומערכת השעות השבועית.

בנוסף, המסך מכיל את פונקציונליות הליבה של המערכת - הפעלת אלגוריתם השיבוץ החכם שבונה את מערכת השעות בפועל. עיצוב המסך נקי, מרווח, וכולל אלמנטים דינמיים התורמים לחווית משתמש (UX) מודרנית ונעימה.

- רכיבי ממשק משתמש (UI Elements) ותפקידם:

➤ סרגל ניווט עליון (Top Navigation Bar):

- לוגו וכותרת: הכיתוב "איזור אישי סטודנטים" מלווה באייקון של כובע אקדמי, המציין באיזה מודול של המערכת המשתמש נמצא כעת.
- טקסט מזג אוויר: רכיב דינמי המציג טקסט ידידותי (למשל: "מזג אוויר חמים עונת מעבר נעימה!") ואת הטמפרטורה הנוכחית. תפקידו להעניק למערכת תחושה חיה, אינטראקטיבית ומסבירת פנים.
- כפתור התנתקות (Logout Link): קישור המאפשר לסטודנט לסיים את הגישה המאובטחת למערכת ולחזור למסך ה-Login.
- פרטי משתמש (User Profile): תצוגה של שם הסטודנט (למשל "firstUser") מלווה באייקון פרופיל אישי.

➤ פאנל ימני - אזור פעולות (Control Panel):

- בחירת תאריך (Date Picker / "לוח שנה"): רכיב המאפשר לסטודנט לבחור תאריך ספציפי.

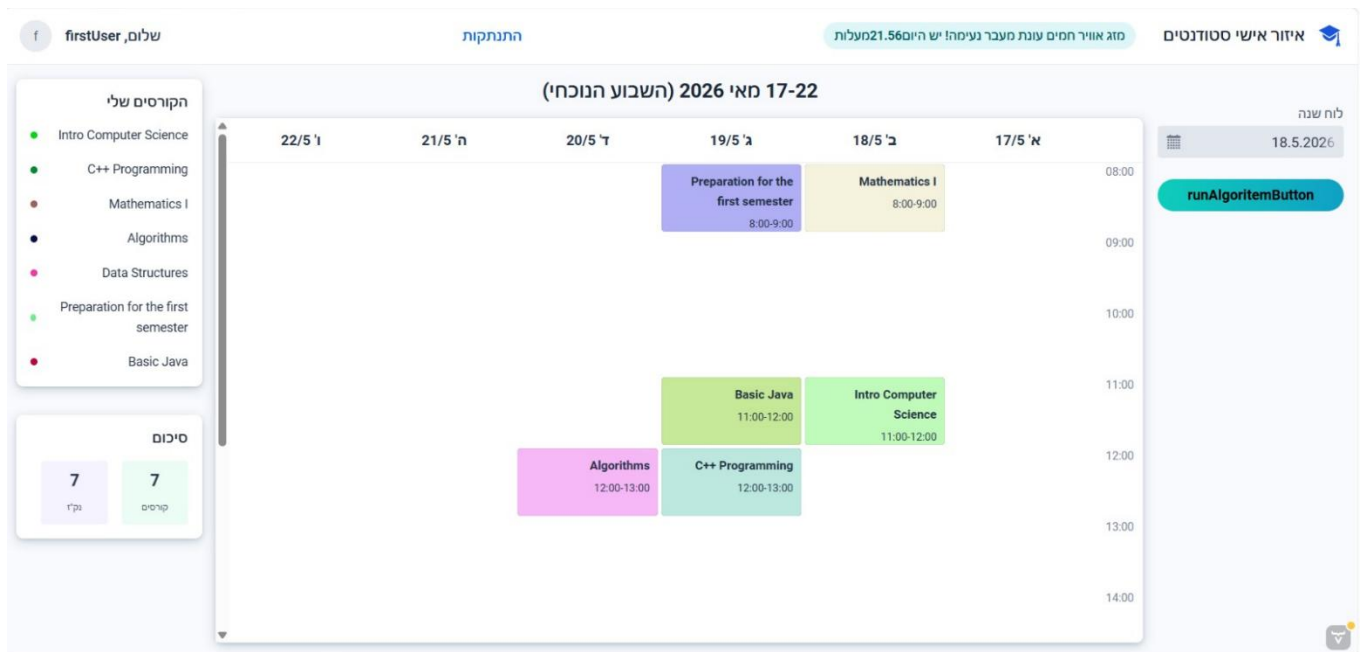
- כפתור הפעלת אלגוריתם (runAlgoritemButton): כפתור פעולה קריטי בולט (בצבע טורקיז). לחיצה על כפתור זה מפעילה מאחורי הקלעים את אלגוריתם השיבוץ המורכב של המערכת (המשתמש בלוגיקת מיון טופולוגי ואילוצים כדי לחשב את סדר הקורסים האופטימלי). תוצרי חישוב זה יוצגו על גבי הטבלה המרכזית.

➤ פאנל שמאלי - מידע וסיכום (Info & Summary Panels):

- רשימת "הקורסים שלי" (My Courses): מציגה את כלל הקורסים שהסטודנט בחר במסך הקודם או רשום אליהם. לצד כל קורס מופיע חיווי צבעוני (Color Dot) אשר צפוי לשמש כמקרא ויזואלי לבלוקים שיוצגו בתוך מערכת השעות, לצורך זיהוי מהיר.
- חלונית "סיכום" (Summary Widget): מציגה נתונים מספריים מרוכזים וברורים: סך כל הקורסים שנבחרו (למשל 7 קורסים) וסך נקודות הזכות (נק"ז) שהם מקנים (למשל 7 נק"ז). רכיב זה מסייע לסטודנט לעקוב אחר העומס האקדמי שלו.

➤ רכיב מרכזי - טבלת מערכת השעות (Central Calendar Component): טבלת יומן

- שבועית התופסת את רוב שטח המסך. הטבלה מציגה את ימי השבוע (א'-ו') בחלקה העליון ושעות פעילות בצירה האנכי. (אציין שרכיב זה ותפקידו המלא יורחבו בהמשך תחת תיאור התמונה הבאה).

דף הבית/אזור אישי סטודנטים - חלק ב (לאחר הפעלת האלגוריתם השיבוץ):

- מטרת המסך (תפקידו): מסך זה הוא הלכה למעשה התוצר הסופי והמרכזי של המערכת עבור הסטודנט. הוא מציג את דף הבית מיד לאחר שהמשתמש לחץ על כפתור הפעלת האלגוריתם (runAlgorithmButton). מטרת המסך כעת היא להציג בצורה חזותית, ברורה ונוחה את מערכת השעות האופטימלית שחושבה עבור הסטודנט, בהתבסס על הקורסים שבחר קודם לכן. אלגוריתם השיבוץ (אשר רץ מאחורי הקלעים ומתחשב באילוצי מערכת, תלויות ודרישות קדם) סיים את עבודתו, והמיר את רשימת הקורסים ללוח זמנים שבועי משובץ.
- רכיבי ממשק משתמש (UI Elements) ותפקידם: (הערה: הרכיבים ההיקפיים - סרגל עליון, מזג אוויר, פרטי משתמש, סיכום נק"ז וכפתורים נותרו זהים למסך הקודם. התיאור להלן מתמקד בשינוי המרכזי במסך זה).
 - טבלת יומן שבועית (Weekly Calendar Grid): הרכיב המרכזי במסך. הטבלה בנויה במבנה של רשת (Grid) המציגה את ימי השבוע לאורך ציר ה-X ושעות היום לאורך ציר ה-Y.
 - תמיכה בכיווניות (RTL Layout): הטבלה מיושרת לימין (Right-to-Left), כך שהעמודה הראשונה מימין היא יום ראשון (א' 17/5), והימים מתקדמים שמאלה. עיצוב זה הכרחי לחוויית משתמש טבעית ותקנית בממשקים בשפה העברית.
 - אירועי מערכת / משבצות קורסים (Course Events/Blocks): רכיבים גרפיים המייצגים את השיבוץ בפועל.

- כל בלוק ממוקם ביום ובשעה המדויקים שנקבעו לו על ידי האלגוריתם.
- כל בלוק מכיל טקסט המציג את שם הקורס (למשל: "Intro Computer Science") ואת שעות הפעילות שלו (למשל: "11:00-12:00").
- כל בלוק מקבל צבע יחודי עבורו כדי להבדיל בין הקורסים השונים.
- פס גלילה אנכי (Vertical Scrollbar): ממוקם בצדה השמאלי של הטבלה. מאפשר לסטודנט לגלול מטה ולצפות בשעות לימוד מאוחרות יותר שאינן נכנסות בתצוגה הראשונית של המסך (למשל אחרי שעה 14:00).
- הודעות ודיאלוגים (Messages & Dialogs):
 - הודעת הצלחה (Success Toast/Notification): מיד עם סיום ריצת האלגוריתם והופעת הקורסים על המסך, תוצג הודעה קופצת קטנה (למשל למטה או בפינה העליונה) המאשרת למשתמש: "מערכת השעות שובצה בהצלחה!".

מסך התחברות לסגל ומנהלי מערכת (Lecturer/Admin Login):

- מטרת המסך (תפקידו):

מסך זה מהווה את שער הכניסה המאובטח עבור אנשי הסגל האקדמי (מרצים) ומנהלי המערכת. למסך זה מגיעים לאחר לחיצה על כפתור הניווט הייעודי (Login Lecturer/Admin) שהופיע במסך ההתחברות הראשי של הסטודנטים. מטרתו היא לבצע אימות זהות (Authentication) נפרד לבעלי הרשאות ניהול והוראה, תוך הפרדה לוגית וויזואלית מממשק הסטודנטים. העיצוב תומך בהפרדה זו על ידי שימוש בתמונה ייצוגית ומקצועית של ישיבת צוות בחדר ישיבות אקדמי, המותאמת לאופי ולסטטוס של קהל היעד בעמוד זה.

- רכיבי ממשק משתמש (UI Elements) ותפקידם:

- כותרת Login for lecturers/admin - (Heading): כותרת בולטת המגדירה בבירור למי מיועד המסך ומונעת בלבול של סטודנטים שאולי הגיעו לעמוד בטעות.
- תיבת טקסט ID - (Text Field): שדה הזנה שבו איש הסגל או המנהל נדרש להקליד את מספר הזיהוי שלו (תעודת זהות או מספר עובד). בניגוד לסטודנטים המזדהים באמצעות אימייל, עבור סגל נבחר השימוש ב-ID כמזהה הייחודי.
- תיבת סיסמה Password - (Password Field): שדה הזנה מאובטח להזנת סיסמת הגישה. השדה כולל אייקון של עין (Toggle Visibility) המאפשר להציג או להסתיר את תווי הסיסמה שהוקלדו, לשם וידוא תקינות ההקלדה.
- תיבת סימון (Checkbox) - התחבר כמנהל מערכת: רכיב בחירה המאפשר למשתמש להגדיר את תצורת ההתחברות שלו. סימון תיבה זו מנחה את המערכת לאמת את המשתמש מול הרשאות הניהול (Admin) ולאפשר לו גישה למסכים ולפונקציות מערכתיות מתקדמות

(כגון ניהול משתמשים, עריכת קורסים וכדומה), בהנחה שאכן יש לו את ההרשאות הנדרשות במסד הנתונים.

- כפתור פעולה ראשי login: כפתור אישור (בצבע ירוק-צהוב) השולח את נתוני ה-ID, הסיסמה והסטטוס (מרצה/מנהל) לשרת לצורך אימות. התחברות מוצלחת תנתב את המשתמש לאזור האישי המותאם לתפקידו.
- תמונת אווירה (Image): תמונה של צוות אקדמי בחדר ישיבות, התופסת את החצי הימני של המסך.

• הודעות ודיאלוגים (Messages & Dialogs):

- הודעת שגיאת הרשאות או פרטים שגויים (Invalid Credentials / Access Denied): במידה והוזנו ID או סיסמה שאינם תואמים, או במקרה שבו מרצה רגיל סימן את התיבה "התחבר כמנהל מערכת" אך אין לו הרשאות מתאימות לכך, המערכת תקפיץ הודעת שגיאה המיידעת אותו על גישה נדחית או פרטים שגויים.
- הודעת ולידציה על שדות ריקים (Validation Error): בדומה למסכים הקודמים, ניסיון לחיצה על כפתור ה-login מבלי למלא את שדות החובה (ID או סיסמה) יציג הודעת שגיאה מקומית המבקשת להשלים את הנתונים החסרים.

מסך הבית עבור המנהל:



• מטרת המסך (תפקידו):

מסך זה הוא מרכז הבקרה הראשי של מנהל המערכת האקדמית. בניגוד למסך הסטודנט המציג מערכת אישית, מסך זה מציג את תמונת המצב המערכתית הכוללת - "הלו"ז הכללי". מטרתו לאפשר למנהל להשקיף על כלל הקורסים המשובצים, המרצים המלמדים, ופריסת השעות השבועית במוסד. בנוסף, המסך מכיל את הפונקציה המרכזית והקריטית ביותר לניהול האקדמי: היכולת להריץ מחדש את אלגוריתם השיבוץ המרכזי כדי ליצור לוח זמנים אופטימלי חדש עבור כלל הסטודנטים והסגל.

- רכיבי ממשק משתמש (UI Elements) ותפקידם:
 - (הערה: סרגל הניווט העליון יתואר בנפרד בסעיף מאוחר יותר).
 - כותרת הלוח (Calendar Header): טקסט דינמי המציין את התאריכים המוצגים כעת (למשל "17-22 מאי 2026") ומבהיר למשתמש כי זהו הלוח הכללי ("לוח כללי").
 - פאנל ימני - פקדים ופעולות מערכת (Right Control Panel):
 - בחירת תאריך (Date Picker / "לוח שנה"): רכיב בחירת תאריכים המאפשר למנהל לנווט קדימה ואחורה בין שבועות שונים.
 - כפתור פעולה ראשי - "חשב מערכת מחדש" (Recalculate Schedule Button):
 - כפתור בולט בצבע אדום, שנועד לסמן פעולה קריטית ובעלת השפעה רחבה (Destructive/Major Action). לחיצה עליו מפעילה מאחורי הקלעים את אלגוריתם האופטימיזציה המורכב של המערכת (המתחשב בתלויות הקורסים, זמינות מרצים ואילוצים נוספים), במטרה להרכיב ולשבץ מחדש את לוח הזמנים המלא של המוסד.
 - פאנל שמאלי - "קורסים במערכת" (Left Panel - System Courses):
 - מונה קורסים (Total Courses Counter): תצוגה מספרית (למשל: סה"כ: 21 קורסים) המעניקה למנהל נתון סטטיסטי מהיר על נפח הפעילות האקדמית.
 - רשימת קורסים נגללת (Scrollable Course List): רשימה המציגה את כלל הקורסים הפעילים.
 - כל פריט ברשימה כולל את שם הקורס, קוד קורס ייחודי (למשל: C1CS1) המסייע בזיהוי חד-חד-ערכי, ונקודה צבעונית (Color Dot) המשמשת למקרא צבעים מול הטבלה המרכזית. הרשימה מלווה בפס גלילה (Scrollbar) המאפשר ניווט נוח גם כשיש עשרות קורסים.
 - רכיב מרכזי - טבלת מערכת השעות הכללית (Master Calendar Grid): יומן שבועי המיושר במלואו לימין (יום א' הוא העמודה הראשונה מימין), מה שמבטיח חוויית משתמש חלקה וטבעית לממשק בשפה העברית.
 - משבצות קורסים מורחבות (Extended Course Blocks): בלוקים המייצגים את השיבוץ בפועל. בניגוד למסך הסטודנט, המשבצות במסך המנהל עשירות יותר במידע ומכילות את: שם הקורס, הקוד שלו בסוגריים, שעות מדויקות, ושם המרצה המשובץ להעביר את הקורס (למשל: David Gerber או Yotam Eli). הפריסה מאפשרת למנהל לזהות חפיפות, חלונות זמן ריקים או עומס על מרצים.
 - הודעות ודיאלוגים (Messages & Dialogs):
 - הודעת שגיאה על חוסר היתכנות (Unsolvable Constraints Error): במידה והאלגוריתם נכשל בבניית המערכת (למשל, אין מספיק שעות פנויות למרצה מסוים או שקיימת התנגשות

קשיחה בדרישות), תקפוץ הודעת שגיאה מפורטת למנהל שתסביר אילו קורסים או אילוצים מנעו את השלמת השיבוץ, כדי שיוכל לטפל בבעיה ידנית במסד הנתונים.

דף הוספת הנתונים - עבור מנהל:

The screenshot displays a web interface titled "הוספת נתונים למערכת" (Add Data to System). It features four distinct panels, each with a colored header and a set of form fields:

- הוספת מנהל (Manager):** Fields for "שם המנהל" (Manager Name), "תעודת זהות / משתמש" (ID / User), "סיסמה" (Password), and a blue "שמור מנהל" (Save Manager) button.
- הוספת קורס (Course):** Fields for "שם הקורס" (Course Name), "מזהה (ID)" (ID), "משך הקורס (סמסטר פתיחה)" (Course Duration (Semester Start)), a checkbox for "קורס חובה?" (Required Course?), "מרחב" (Room), and "קורסי קדימות" (Prerequisite Courses).
- הוספת מרצה (Lecturer):** Fields for "שם המרצה" (Lecturer Name), "תעודת זהות" (ID), "סיסמה" (Password), "זמני הוראה" (Lecture Time), "יום" (Day), and "שעת התחלה" (Start Time).
- הוספת סטודנט (Student):** Fields for "שם משתמש" (Username), "סיסמה" (Password), "אימייל" (Email), "טלפון" (Phone), and "גיל" (Age).

Each panel includes a small eye icon for password visibility and a dropdown arrow for the "מרחב" (Room) field. A "שיוך קורסים" (Assign Courses) button is located at the bottom right of the interface.

• מטרת המסך (תפקידו):

מסך זה משמש כלוח הבקרה המרכזי (Back-Office) להזנת נתונים אל תוך מסד הנתונים של המערכת. הגישה למסך זה מוגבלת למנהלי מערכת (Admins) בלבד, והוא חסום לחלוטין בפני סטודנטים או מרצים רגילים. המסך מאפשר למנהל ליצור ולנהל את ישויות הליבה של המוסד האקדמי: סטודנטים, מרצים, קורסים ומנהלים נוספים. העיצוב מבוסס על פריסת עמודות (Cards Layout) המותאמת לכיווניות ימין-לשמאל (RTL), כך שכל עמודה מוקדשת ליישות אחרת במערכת וכוללת את הטופס הרלוונטי עבורה.

• רכיבי ממשק משתמש (UI Elements) ותפקידם:

➤ כותרת ראשית: "הוספת נתונים למערכת" - מגדירה את מטרת העל של המסך.

➤ עמודת "הוספת סטודנט" (פאנל ימני - כחול):

○ שדות טקסט (Text Fields): שם משתמש, סיסמה (עם אייקון עין להצגה/הסתרה), אימייל, טלפון, וגיל (שדה מספרי השומר את נתוני הגיל במודל הסטודנט). שדה האימייל מהווה מרכיב קריטי כאן, שכן הוא משמש כמזהה הייחודי (Unique Identifier) של הסטודנט בעת ההתחברות למערכת, על מנת למנוע כפילויות של שמות משתמש.

○ רשימה נפתחת (Dropdown): שיוך קורסים - מאפשר למנהל לרשום את הסטודנט לקורסים קיימים במערכת באופן ידני.

○ כפתור פעולה "שמור סטודנט" (בצבע כחול) - שולח את נתוני הטופס הספציפי לשרת ושומר אותם ב-DB.

➤ עמודת "הוספת מרצה" (פאנל מרכזי-ימני - ירוק):

- שדות טקסט: שם המרצה, תעודת זהות וסיסמה.
- הגדרת זמני הוראה (Teaching Constraints): אוסף של שדות בחירה (רשימות נפתחות עבור יום בשבוע ושעת התחלה). רכיבים אלו קריטיים, שכן הם מגדירים את חלונות הזמן בהם המרצה זמין ללמד, נתונים שישמשו את אלגוריתם השיבוץ לאחר מכן כדי למנוע התנגשויות.
- רשימת שיוך קורסים דינמית (Dynamic Select List): רכיב המאפשר לבחור אילו קורסים המרצה מוסמך ללמד. רשימה זו אינה סטטית, אלא נטענת בזמן אמת (Fetched) מתוך מסד הנתונים ומציגה את כלל הקורסים הקיימים.
- כפתור פעולה "שמור מרצה" (בצבע כחול) - שולח את נתוני הטופס הספציפי לשרת ושומר אותם ב-DB.

➤ עמודת "הוספת קורס" (פאנל מרכזי-שמאלי - צהוב):

- שדות טקסט: שם הקורס, מזהה (ID - קוד קורס ייחודי).
- משך הקורס / סמסטר פתיחה: שדה טקסט/בחירה המגדיר בעוד כמה סמסטרים הקורס נפתח. נתון זה הכרחי לבניית גרף התלויות של המערכת (משמש כמשקלי הקשתות בגרף התכנון).
- תיבת סימון (Checkbox): קורס חובה? - מאפשר לסמן האם הקורס הוא בגדר חובת לימוד או בחירה.
- רשימה נפתחת דינמית - קורסי קדימות (Prerequisites): רכיב חיוני המאפשר למנהל לבחור מתוך מסד הנתונים אילו קורסים חייב הסטודנט לסיים לפני שיוכל להירשם לקורס הנוכחי. מידע זה קריטי להפעלת המיון הטופולוגי (Topological Sort) באלגוריתם השיבוץ.
- כפתור פעולה "שמור קורס" (בצבע כחול) - שולח את נתוני הטופס הספציפי לשרת ושומר אותם ב-DB.

➤ עמודת "הוספת מנהל" (פאנל שמאלי - סגול):

- שדות טקסט: שדות בסיסיים ליצירת מנהל חדש (שם, ת"ז/משתמש, סיסמה).
- כפתור פעולה "שמור מנהל" (בצבע כחול) - שולח את נתוני הטופס הספציפי לשרת ושומר אותם ב-DB.

• הודעות ודיאלוגים (Messages & Dialogs):

- הודעת שמירה בהצלחה (Success Toast/Alert): בעת לחיצה על כפתור השמירה באחת העמודות (והזנה תקינה של הנתונים), תקפוץ הודעה חיובית קצרה (למשל: "הקורס נוסף בהצלחה למסד הנתונים") והטופס יתנקה לקראת הזנה חדשה.

- הודעת שגיאה על מזהה כפול (Duplicate Entry Error): במידה והמנהל מנסה להזין סטודנט עם אימייל שכבר קיים, או קורס עם ID שכבר שמור במערכת, תופיע הודעת שגיאה המונעת את השמירה ומבקשת לשנות את המזהה.
- הודעת שגיאת ולידציה (Form Validation Error): במידה וחסרים שדות חובה (כגון אי הזנת שעות זמינות למרצה, או השארת שם קורס ריק), השדות הרלוונטיים יסומנו באדום ותופיע הודעה מתאימה הדורשת השלמת פרטים בטרם השמירה.

דף הצגת הנתונים עבור המנהל:

תצוגת נתוני המערכת				
בחר נתונים להצגה				
קורסים				
מזהה	שם הקורס	סמסטר פתיחה	מרצה	קדימויות
C1CS1	Intro Computer Science	1	David Gerber	
C1C++2	C++ Programming	1	Yotam Eli	C1CS1
C1MATH1	Mathematics I	1	David Gerber	
C1ALG4	Algorithms	1	Yotam Eli	C1C++2, C1MATH1
C1DS5	Data Structures	1	David Gerber	C1C++2, C1MATH1
C1DB6	Databases	2	David Gerber	C1C++2
C1SE7	Software Engineering	3	Yotam Eli	C1ALG4, C1DS5
C1OS8	Operating Systems	2	David Gerber	C1DS5
C1CN9	Computer Networks	2	David Gerber	C1DS5, C1DB6
C11110	Preparation for the first semester	1	Yotam Eli	

• מטרת המסך (תפקידו):

מסך זה משמש ככלי בקרה ושקיפות עבור מנהלי המערכת, ומאפשר להם לצפות בזמן אמת בכלל הנתונים השמורים במסד הנתונים (DB) של המוסד האקדמי. בניגוד למסך ההוספה, מסך זה מיועד לקריאה בלבד (Read-Only) ומטרתו להציג בצורה מרוכזת, טבלאית וברורה את הרשומות הקיימות. המסך דינמי לחלוטין ומאפשר למנהל לעבור בקלות בין תצוגות של ישויות שונות במערכת (סטודנטים, מרצים, מנהלים, קורסים, ושיבוצים) ללא צורך בניווט למסכים נפרדים.

• רכיבי ממשק משתמש (UI Elements) ותפקידם:

- כותרת ראשית (Main Heading): "תצוגת נתוני המערכת" - מגדירה למשתמש את מהות העמוד.
- רשימה נפתחת לסינון נתונים (Dropdown Menu / Data Selector): רכיב מפתח תחת הכותרת "בחר נתונים להצגה". לחיצה עליו פותחת תפריט המאפשר למנהל לבחור איזו טבלת נתונים לשלוף מהשרת. האפשרויות הקיימות הן: סטודנטים, מרצים, מנהלים, קורסים, ושיבוץ. בחירה באחת מהאפשרויות מרעננת את הטבלה המרכזית בהתאם.

- טבלת נתונים דינמית (Dynamic Data Grid): הרכיב המרכזי במסך, אשר משנה את העמודות שלו בהתאם לבחירה ברשימה הנפתחת. בדוגמה המוצגת בצילום המסך (תצוגת "קורסים"), הטבלה מכילה את העמודות הבאות:
 - מזהה (ID): קוד הקורס הייחודי במערכת.
 - שם הקורס (Course Name): השם המלא של הקורס.
 - סמסטר פתיחה (Opening Semester): נתון מספרי המייצג בעוד כמה סמסטרים הקורס ייפתח. (נתון זה קריטי מאחורי הקלעים, שכן הוא מתורגם למשקלי הקשתות בגרף השיבוץ).
 - מרצה (Lecturer): איש הסגל המשוך להעברת הקורס.
 - קדימויות (Prerequisites): רשימת מזהים של קורסי קדם הנדרשים כחובה. (נתון זה חיוני עבור אלגוריתם המיון הטופולוגי המרכיב את מערכת השעות).
 - (חשוב לציין שלכל טבלה שמוצגת יש נתונים שונים. הנתונים שיוצגו הם הנתונים שהמנהל הכניס בעמוד "הוספת הנתונים" שהוצג מקודם)
- פס גלילה אנכי (Vertical Scrollbar): ממוקם בצד השמאלי של הטבלה ומאפשר ניווט נוח ומהיר במידה וכמות הרשומות במסד הנתונים חורגת מהגובה הנראה של המסך.
- הודעות ודיאלוגים (Messages & Dialogs):
 - הודעת שגיאת תקשורת (Fetch Error Toast): אם מתרחשת תקלה בשליפת הנתונים מהשרת או ניתוק ממסד הנתונים, תקפוץ התראה (באדום) המיידעת את המנהל על השגיאה ומציעה לו לרענן את העמוד.

נאב בר (סרגל ניווט) עבור המנהל:

- מטרת הרכיב (תפקידו): סרגל הניווט העליון (Nav Bar) הוא רכיב גלובלי (Global Component) המוקרן בחלקו העליון של כל מסך באזור הניהול (Back-Office). תפקידו המרכזי הוא לספק למנהל המערכת התמצאות במרחב הווירטואלי, ולאפשר לו גישה מהירה, חלקה וישירה לכלל הפונקציות המרכזיות של המערכת, מבלי לאבד את ההקשר או להזדקק לכפתורי "חזור" בדפדפן. הסרגל מעוצב בצורה נקייה ומינימליסטית, ותומך באופן מלא בפריסה ימין-לשמאל (RTL), כך שזרימת הקריאה והפעולות טבעית למשתמש.
- רכיבי ממשק משתמש (UI Elements) ותפקידם:
 - (הערה: הרכיבים מתוארים מימין לשמאל, בהתאם לפריסת המסך)
 - חיווי סביבת עבודה (System Identity): בפינה הימנית ביותר מופיע אייקון של כובע אקדמי בצבע אדום/בורדו לצד הכיתוב "איזור ניהול מערכת".

רכיב זה אינו לחיץ, ותפקידו להעניק חיווי סטטוס ברור המבהיר למשתמש כי הוא מחובר תחת הרשאות ניהול (להבדיל מאזור הסטודנטים בו האייקון היה שונה).

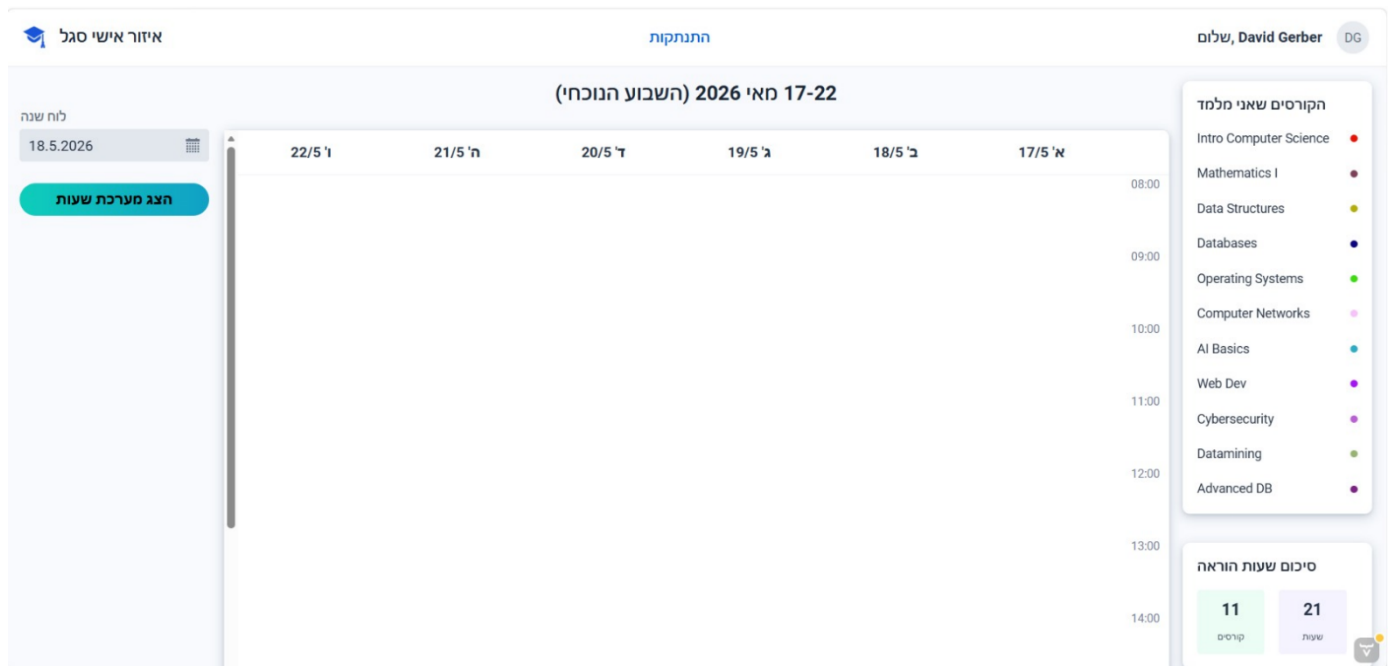
➤ פרטי משתמש (User Profile/Avatar): תצוגת ברכה אישית (לדוגמה: "שלום, kfir") המלווה בסמל פרופיל עגול (Avatar) המכיל את האות הראשונה של שם המשתמש. רכיב זה מאשר ויזואלית מי הוא המשתמש המחובר כעת ל-Session.

➤ תפריט קישורי ניווט (Navigation Links): סדרה של טקסטים לחיצים בצבע כחול. לחיצה על כל אחד מהם מנתבת את המשתמש של המערכת לעמוד המתאים:

- מסך הצגת הנתונים: מעביר את המנהל למסך הצגת הנתונים (טבלאות הנתונים הדינמיות לקריאה בלבד).
- מסך הוספת הנתונים: מעביר את המנהל למסך הטפסים (הוספת קורסים, מרצים, תלויות וכו').
- מסך הבית: מחזיר את המנהל למסך הבית הראשי (לוח הזמנים הכללי וכפתור חישוב המערכת).
- כפתור התנתקות (Logout): הקישור השמאלי ביותר בסרגל. מאפשר סיום יזום ומאובטח של הגישה למערכת, מחיקת ה-Token/Session, וחזרה למסך ההתחברות הראשוני.

• הודעות ודיאלוגים (Messages & Dialogs):

➤ חיווי מצב פעיל (Active State / Hover Effect): למרות שזהו אינו דיאלוג קופץ, נהוג שממשק הניווט מספק משוב ויזואלי מיידי. בעת ריחוף עם העכבר (Hover) על אחד הקישורים, או כאשר המנהל נמצא במסך מסוים (למשל, כשהוא ב"מסך הבית"), הקישור הרלוונטי צפוי להיות מודגש (Bold), או לקבל קו תחתון, כדי לאותת למשתמש היכן הוא נמצא כעת.

אזור אישי סגל / מסך בית למרצה:

- מטרת המסך (תפקידו):

מסך זה מהווה את מרכז הבקרה האישי של איש הסגל (המרצה) לאחר התחברות מוצלחת למערכת. מטרתו היא להציג למרצה ריכוז של כלל הקורסים אותם הוא משובץ ללמד, את היקף שעות ההוראה שלו, ולאפשר לו לשלוף ולצפות במערכת השעות השבועית שלו. המסך המוצג מתאר את המצב ההתחלתי (Initial State) בעת הכניסה למערכת, טרם הפעלת הפונקציה המציגה את הלו"ז בטבלה. עיצוב המסך ממשיך את הקו הנקי של המערכת תוך הקפדה על פריסת Right-to-Left (RTL) לזרימת עבודה טבעית בעברית, כך שפאנלי המידע ממוקמים בצידו הימני של המסך והיומן במרכז.

- רכיבי ממשק משתמש (UI Elements) ותפקידם:

➤ סרגל ניווט עליון (Top Navigation Bar):

- חיווי זהות (Identity): כיתוב "איזור אישי סגל" מלווה באייקון כובע אקדמי כחול, המבדל סביבה זו מסביבת הסטודנטים או ההנהלה.

- כפתור התנתקות (Logout Link): לסיום הגישה למערכת.

- פרטי המרצה (User Profile): שם המרצה (בדוגמה: David Gerber) מלווה באווטאר (Avatar) אישי עם ראשי התיבות שלו.

➤ פאנל ימני - מידע אקדמי (Right Info Panel): רשימת "הקורסים שאני מלמד", רשימה דינמית הנשלפת ממסד הנתונים, המציגה אך ורק את הקורסים ששויכו למרצה זה (למשל על ידי המנהל במסך "הוספת נתונים"). לכל קורס מוצמד קוד צבע (Color Dot) שימשם לזיהוי מהיר שלו על גבי מערכת השעות שתוצג בהמשך.

➤ חלונית "סיכום שעות הוראה" (Teaching Hours Summary): ווידג'ט המרכז נתונים סטטיסטיים אודות העומס האקדמי של המרצה: סך כל הקורסים שהוא מעביר (למשל 11 קורסים) וסך שעות ההוראה הכולל שלו (למשל 21 שעות).

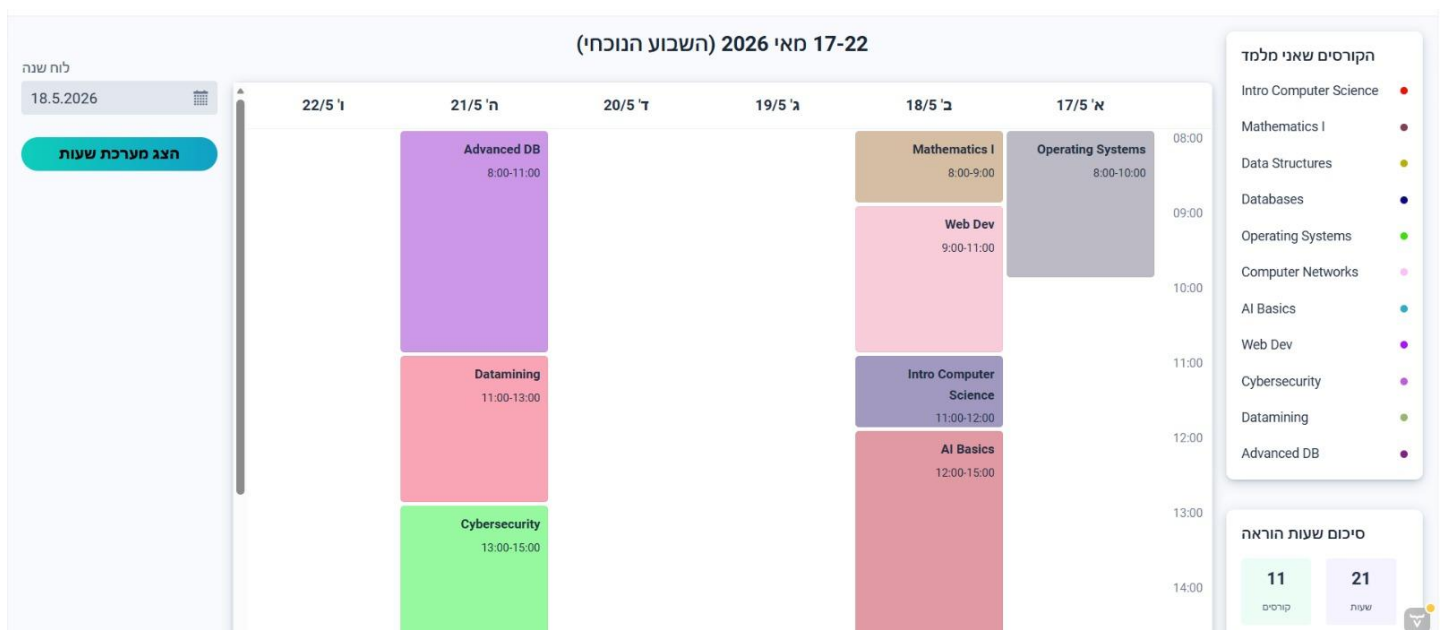
➤ פאנל שמאלי - פקדים (Left Control Panel):

- בחירת תאריך (Date Picker / "לוח שנה"): מאפשר למרצה לנווט בין השבועות השונים של הסמסטר כדי לראות את הלוח העתידי או בעבר.
- כפתור פעולה ראשי - "הצג מערכת שעות" (Primary Button): כפתור בולט בצבע טורקיז. בניגוד לכפתור במסך המנהל שמחשב את המערכת מאפס, כפתור זה מבצע קריאת נתונים (Fetch) ממסד הנתונים כדי למשוך את השיבוץ הקיים שכבר חושב עבור המרצה הספציפי, ולהציג אותו על גבי הטבלה המרכזית.
- רכיב מרכזי - טבלת מערכת שעות ריקה (Empty Calendar Grid): מבנה הרשת (Grid) הסטנדרטי של ימי השבוע ושעות היום. במצב התחלתי זה, הרשת ריקה וממתינה לפעולת המשתמש (לחיצה על כפתור התצוגה).

• הודעות ודיאלוגים (Messages & Dialogs):

➤ חיווי טעינת נתונים (Loading State): בעת לחיצה על כפתור "הצג מערכת שעות", יופיע חיווי ויזואלי המציין שהמערכת טוענת את נתוני השיבוץ מהשרת (לפני שהם מרונדרים אל תוך הטבלה).

אזור אישי סגל (לאחר טעינת מערכת השעות):



• מטרת המסך (תפקידו):

מסך זה מציג את התוצאה הישירה של פעולת שליפת הנתונים, לאחר שהמרצה לחץ על הכפתור "הצג מערכת שעות" (במסך הקודם). מטרת המסך היא להציג באופן חזותי, ברור ונוח את לוח

ההוראה השבועי המלא של איש הסגל, כפי שחושב על ידי אלגוריתם השיבוץ של המערכת ונשמר במסד הנתונים. תצוגה זו מאפשרת למרצה לתכנן את שבוע העבודה שלו, לראות את פריסת העומס, ולזהות "חלונות" (זמנים פנויים) בין השיעורים.

- רכיבי ממשק משתמש (UI Elements) ותפקידם:
 - (הערה: הפאנלים הצידיים - סיכום השעות, רשימת הקורסים, בחירת התאריך והכפתור - נותרו זהים בתפקידם כפי שתוארו במסך הקודם. התיאור להלן מתמקד בשינוי המרכזי שחל בטבלה).
 - טבלת יומן שבועית מאוכלסת (Populated Calendar Grid): הרכיב המרכזי במסך, אשר כעת עבר רינדור (Rendering) מלא ומציג את הנתונים שנשלפו מהשרת. הטבלה שומרת על מבנה ה-RTL (ימין לשמאל) לנוחות הקריאה בעברית.
 - משבצות שיעורים / אירועי הוראה (Teaching Blocks / Events): רכיבים גרפיים (מלבנים צבעוניים) הממוקמים על גבי רשת היומן, המייצגים את חלונות הזמן המדויקים בהם המרצה אמור ללמד.
 - תוכן המשבצת: כל בלוק מציג בצורה תמציתית את שם הקורס (למשל: "Advanced DB") ואת טווח השעות המדויק של השיעור (למשל: "8:00-11:00").
 - סנכרון ומיפוי צבעים (Color Mapping): זהו מאפיין UX מרכזי במסך - צבע הרקע של כל משבצת תואם במדויק לנקודת הצבע המופיעה לצד שם הקורס ברשימת "הקורסים שאני מלמד" בפאנל הימני (למשל, Advanced DB צבוע בסגול גם ברשימה וגם בטבלה).
 - התאמה זו מאפשרת למרצה לזהות קוגניטיבית את פיזור הקורסים שלו במבט חטוף, מבלי להצטרך לקרוא את הטקסט בכל משבצת.
- הודעות ודיאלוגים (Messages & Dialogs):
 - הודעת הצלחה בטעינה (Success Toast): עם סיום שליפת הנתונים מה-DB (Database) והצגתם על המסך, צפויה להופיע חלונת התראה קטנה או הודעה צפה המאשרת: "מערכת השעות נטענה בהצלחה" או "הנתונים מעודכנים".

נאב בר (סרגל ניווט) עבור המרצה:

- מטרת סרגל הניווט (תפקידו):
 - מטרת הנאב בר הנ"ל הוא סרגל הניווט העליון (Navbar) עבור ה"איזור אישי סגל" במערכת. היא נועדה לספק למרצה המחובר ממשק קבוע הכולל זיהוי אישי (שם ותמונת פרופיל או ראשי תיבות) ואפשרות להתנתק מהמערכת ולחזור למסך ההתחברות.
- רכיבי ממשק משתמש (UI Elements) ותפקידם:
 - תמונה של כובע אקדמאי בצבע כחול.
 - כותרת טקסט המציגה את המילים "איזור אישי סגל".

- כפתור "התנתקות" שבעת לחיצה מנקה את הנתונים מהסשן (VaadinSession) ומעביר את המשתמש למסך ההתחברות (LoginView).
- רכיבים המציגים את תמונת הפרופיל (Avatar) וטקסט ברכה ("שלום, [שם המרצה]") הנשאבים מפרטי המרצה השמורים בסשן.

נאב בר (סרגל ניווט) עבור סטודנט:

שלום, ben

התנתקות

חם היום! אל תשכחו להביא בקבוק מים לשיעור, יש היום 24.84 מעלות

איזור אישי סטודנטים

- מטרת סרגל הניווט (תפקידו)
 - סרגל הניווט העליון נועד לספק התמצאות מהירה במערכת, להציג מידע אישי, קונטקסטואלי ורלוונטי בזמן אמת, ולאפשר שליטה בסיסית בחשבון (כניסה/יציאה). הוא מעוצב בצורה נקייה ומודרנית כדי להעניק חוויית משתמש (UX) מזמינה, צעירה וידידותית.
- אלמנטים של ממשק משתמש (UI Elements) בסרגל
 - כותרת ואייקון התמצאות (מימין): טקסט ברור ("אזור אישי סטודנטים") מלווה באייקון של כובע בוגר. האלמנט נותן אינדיקציה מיידית למשתמש באיזה אזור של המערכת הוא נמצא.
 - בועת התראה קונטקסטואלית (במרכז): רכיב המכיל הודעת מערכת דינמית ("חם היום! אל תשכחו להביא בקבוק מים לשיעור, יש היום 24.84 מעלות"). זהו אלמנט ששובר את "היובש" המערכתי, מוסיף אכפתיות ויוצר חיבור אישי ורענן יותר עם הסטודנט.
 - כפתור פעולה - יציאה (במרכז-שמאל): קישור טקסטואלי פשוט ובולט ("התנתקות") המאפשר לסטודנט לסיים את ה-Session שלו בצורה בטוחה ומהירה.
 - חיווי פרופיל משתמש (משמאל): ברכה אישית ("שלום, ben") לצד סמל משתמש (Avatar) עגול המכיל את האות הראשונה של השם (b). האלמנט מאשר למשתמש שהוא מחובר לחשבון הנכון שלו וסוגר את החוויה האישית של המסך.

12. תיאור התוכנה

סעיף זה מפרט את הכלים, השפות והטכנולוגיות שנבחרו לבניית הפרויקט, כולל נימוק מקצועי מדוע נבחר כל כלי כדי לשרת את מטרות המערכת

12.1 סביבת הפיתוח של התוכנה

12.1.1 מערכת הפעלה

סביבת הפיתוח הוקמה ורצה על גבי מערכת הפעלה מסוג Windows גרסה 11. מערכת זו מספקת תמיכה מלאה ונוחה לכלל כלי הפיתוח שבהם השתמשתי (ג'אווה, סביבת העבודה, הדפדפנים וכו'), והיא סביבת העבודה הנפוצה ביותר למפתחים אישיים, המאפשרת ניהול קבצים ורשת בצורה פשוטה ומהירה.

12.1.2 סביבות הפיתוח (IDEs)

פיתוח הפרויקט בוצע בסביבת הפיתוח המשולבת Visual Studio Code (VS Code) בגרסה 1.107.1.

VS Code היא סביבת פיתוח מודרנית, קלילה ומהירה במיוחד. בחרתי בה מכיוון שהיא מציעה תמיכה מצוינת בג'אווה דרך תוספים ייעודיים, וכן מאפשרת שילוב של כלים נוספים באותו חלון - כמו גישה ישירה למסד הנתונים של MongoDB, וניהול הפרויקט דרך כלי הבנייה Maven, ללא צורך בפתיחת תוכנות כבדות אחרות.

12.1.3 מסגרות לפיתוח התוכנה (Frameworks)

בפרויקט נעשה שימוש בשתי מסגרות עבודה (Frameworks) מרכזיות:

Spring Boot 3.5.8 (עבור ה-Backend): מסגרת העבודה המובילה בתעשייה לפיתוח שרתי ג'אווה. היא חוסכת כתיבה של קוד תשתית רב ומגיעה עם שרת Tomcat מובנה. בחרתי בה כי היא מאפשרת ניהול פשוט מאוד של החיבור למסד הנתונים, יצירת נקודות קצה (REST API) וניהול הלוגיקה והאלגוריתמים בצורה מאורגנת ומאובטחת.

Vaadin 24.9.5 (עבור ה-Frontend): זוהי מסגרת עבודה ייחודית המאפשרת לבנות ממשקי משתמש מתקדמים ועשירים לדפדפן - תוך כתיבת קוד ב-Java בלבד (ללא צורך בכתיבת HTML או JavaScript). בחרתי בה מכיוון שהיא מאפשרת ליצור מסכים מהירים ודינמיים בצורה שמדברת באופן ישיר עם שרת ה-Spring Boot, וחוסכת זמן פיתוח יקר על עיצוב ידני.

12.1.4 שפות תכנות

השפה המרכזית שבה נכתב הפרויקט היא Java 21. ג'אווה היא שפה עילית, חזקה ומונחית עצמים (OOP). בחרתי בגרסה החדשה (21) בגלל היציבות שלה, הביצועים הגבוהים שהיא מציעה, והיכולת שלה להתמודד בקלות עם אלגוריתמים מורכבים הדורשים רקורסיה (כמו ה-Backtracking של מחולל השיבוץ). בנוסף, היא מאפשרת לבנות את כל הפרויקט (לקוח ושרת) באותה שפה בדיוק.

12.1.5 ספריות/מערכות/מנגנוני עזר נוספים

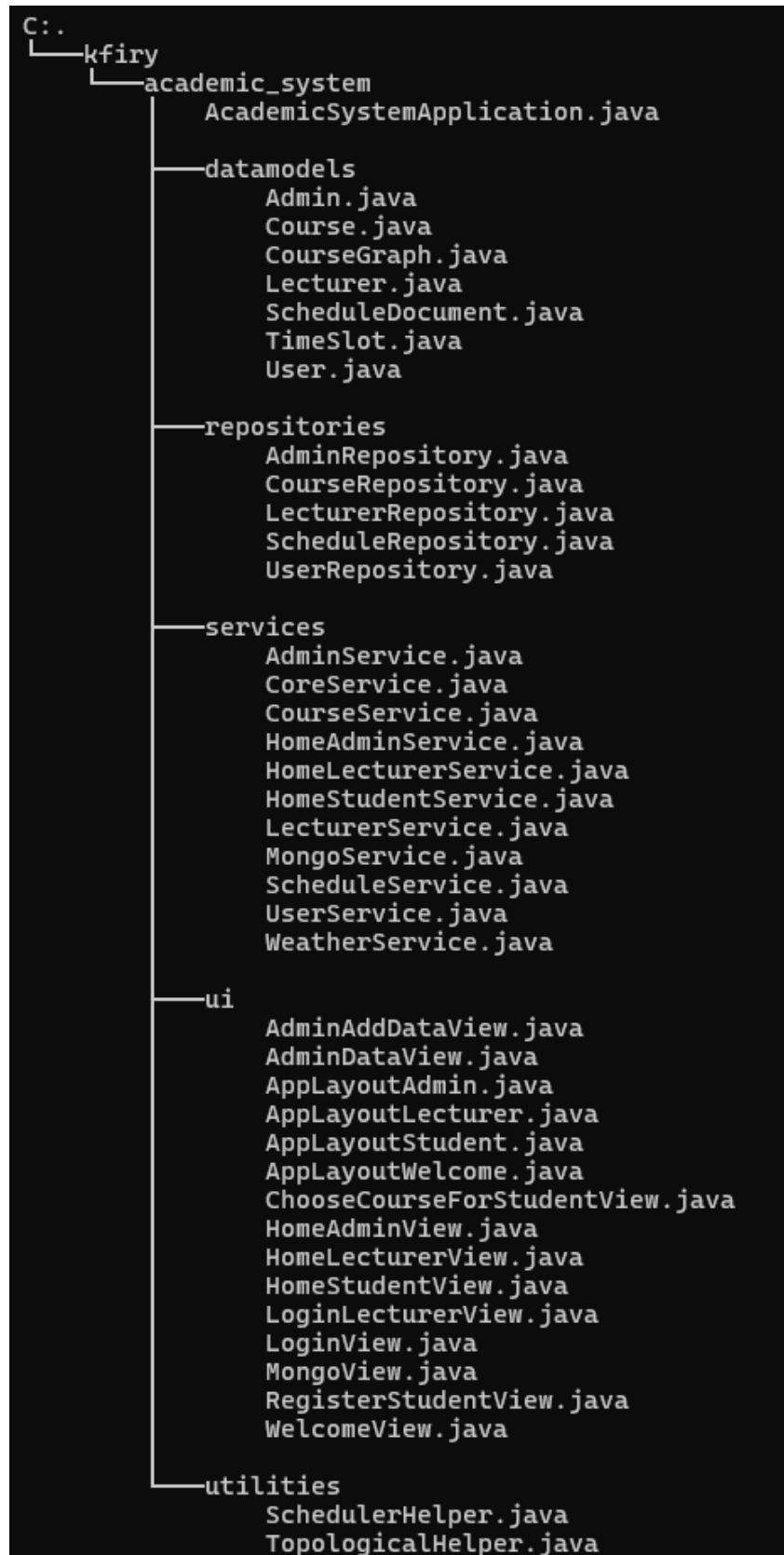
MongoDB Atlas: מערכת ניהול מסד הנתונים בענן (NoSQL). בחרתי בה בגלל הגמישות שלה לשמור מידע כאובייקטים (כמו המסמכים של קורסים ומרצים) ללא צורך בסכמה טבלאית נוקשה, ובגלל האפשרות להתחבר אליה מכל מקום בעולם.

Maven: כלי לניהול ובניית פרויקטים ב-Java. בחרתי בו כי הוא מנהל באופן אוטומטי את כל ספריות העזר שהפרויקט צריך (Dependencies), ודואג שסביבת הפיתוח תעבוד בצורה תקינה.

12.2 תיאור מבנה הפרויקט והמחלקות

12.2.1 עץ המודולים של הפרויקט

להלן תרשים של חבילות (מודולים) המערכת:



פרויקט המערכת האקדמית פותח תוך שימוש בארכיטקטורת שכבות (Layered Architecture) מובנית וקלאסית, המבטיחה הפרדת רשויות מוחלטת בין הלוגיקה העסקית, ניהול הנתונים וממשק המשתמש. חלוקה ארכיטקטונית זו מאפשרת תחזוקה נוחה, קריאות גבוהה של הקוד, ויכולת הרחבה עתידית (Scalability) ללא פגיעה ברכיבים קיימים. בבסיס היישום ממוקמת המחלקה הראשית, AcademicSystemApplication.java, המשמשת כנקודת הכניסה (Entry Point) של המערכת. מחלקה זו אחראית על אתחול השרת, טעינת ההגדרות הראשוניות והפעלת המערכת כולה, לרוב תוך שימוש במנגנוני הזרקת תלויות (Dependency Injection) לחיבור בין השכבות השונות.

שכבת הנתונים המרכזית מאוגדת תחת חבילת datamodels, אשר מהווה את אוסף הישויות (Entities) של המערכת. מחלקות אלו מייצגות את השחקנים השונים הפועלים במערכת, כגון User-Admin, Lecturer ו-Course, TimeSlot, ScheduleDocument. מחלקות אלו נכתבו כמחלקות נתונים פשוטות (POJO) שתפקידן העיקרי הוא לשמש כמכלים (Containers) למידע. הן אינן מכילות לוגיקה עסקית מורכבת אלא מתמקדות בייצוג נקי של מבנה הנתונים, כולל מבנים מתקדמים כמו CourseGraph המשמשים לייצוג קשרים אקדמיים.

הגישה למסד הנתונים מנוהלת באופן בלעדי על ידי חבילת repositories, המהווה את שכבת הגישה לנתונים (Data Access Layer). חבילה זו מפרידה לחלוטין את לוגיקת היישום ממנגנון האחסון הפיזי. לכל ישות מוגדר מאגר נתונים ייעודי משלה, כדוגמת CourseRepository או LecturerRepository. הממשקים והמחלקות בחבילה זו מרכזים את ביצוע פעולות ה-CRUD (יצירה, קריאה, עדכון ומחיקה) מול בסיס הנתונים. הפרדה זו מבטיחה גמישות מקסימלית, שכן כל שינוי עתידי בתשתית מסד הנתונים ידרוש התאמות אך ורק בשכבה מבודדת זו.

ה"מוח" המניע את המערכת והלוגיקה העסקית שלה (Business Logic Layer) מרוכזים בחבילת services. השירותים בחבילה זו פועלים כשכבת תיווך הכרחית: הם מקבלים את הבקשות מממשק המשתמש, אוכפים חוקים עסקיים, מבצעים חישובים ועיבודים, ולאחר מכן פונים לשכבת ה-Repositories לשם תקשורת מול מסד הנתונים. השכבה כוללת שירותים נושאים כמו CourseService ו-ScheduleService המנהלים תהליכי רישום ושיבוץ, לצד שירותים תומכי-מסך כגון HomeAdminService ו-HomeLecturerService המכניסים את הנתונים הייעודיים לתצוגה עבור משתמשים ספציפיים. כמו כן, קיימים שירותים האחראים על תשתיות חיצוניות כגון MongoService ו-WeatherService.

ממשק המשתמש (Presentation Layer) מנוהל במלואו תחת חבילת ui. תפקידה של שכבה זו הוא לתרגם את פעולות המשתמש לבקשות מערכת ולהציג את הנתונים המעובדים באופן ויזואלי. החבילה מורכבת מתבניות עיצוב וסרגלי ניווט כגון AppLayoutAdmin ו-AppLayoutStudent, דרך מסכי אימות כגון LoginView,

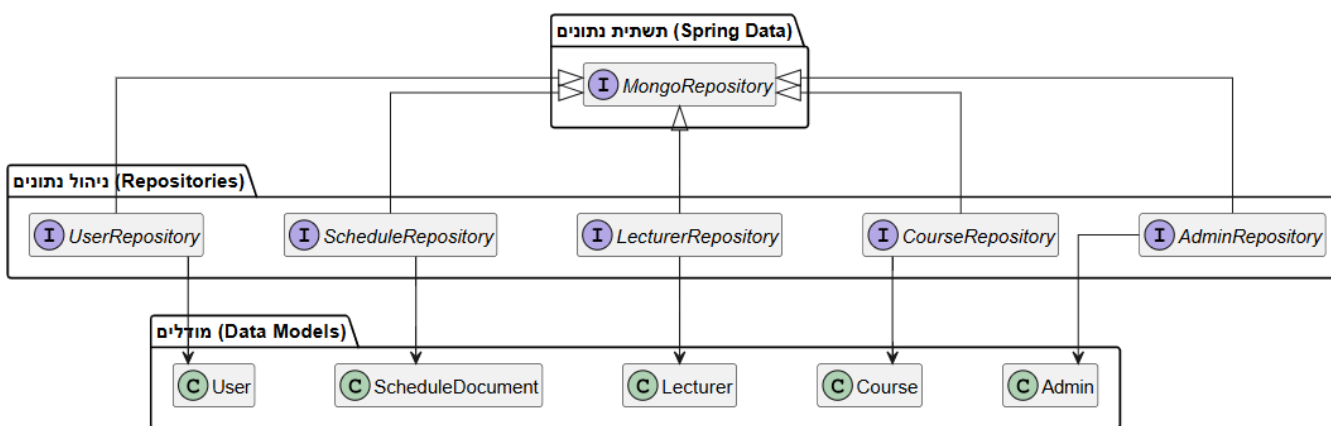
ועד למסכי פעילות מרכזיים דוגמת ChooseCourseForStudentView ו-AdminAddDataView. מסכים אלו תוכננו להיות "רזים" לוגית, והם מסתמכים באופן מלא על חבילת השירותים (Services) לצורך שליפת הנתונים וביצוע הפעולות העסקיות מאחורי הקלעים.

לבסוף, חבילת utilities משמשת כספריית כלי עזר עבור המערכת כולה. חבילה זו מרכזת מחלקות חסרות מצב (Stateless) המכילות אלגוריתמים טהורים או פונקציות עזר עצמאיות. כך למשל, המחלקה TopologicalHelper מכילה אלגוריתמיקה למיון טופולוגי, ככל הנראה לפתרון תלויות של קורסי קדם, והמחלקה SchedulerHelper מספקת לוגיקה מתמטית ואלגוריתמית לסיוע בשיבוץ מערכות שעות ומניעת חפיפות. בידוד הלוגיקה האלגוריתמית לשכבה זו מבטיח קוד נקי, גמיש וניתן לבדיקה (Testing) בקלות מכל מקום במערכת.

12.2.2 תרשים המחלקות UML Class Diagram

בשביל בהירות והבנה יסודית על המחלקות שקיימות לי במערכת החלטתי לחלק את תרשים ה-UML לארבע חלקים. כמובן שכל החלקים חייבים לעבוד ביחד כדי שהמערכת תעבוד כסדרה.

תרשים 1 - ארכיטקטורת שכבת ניהול הנתונים:

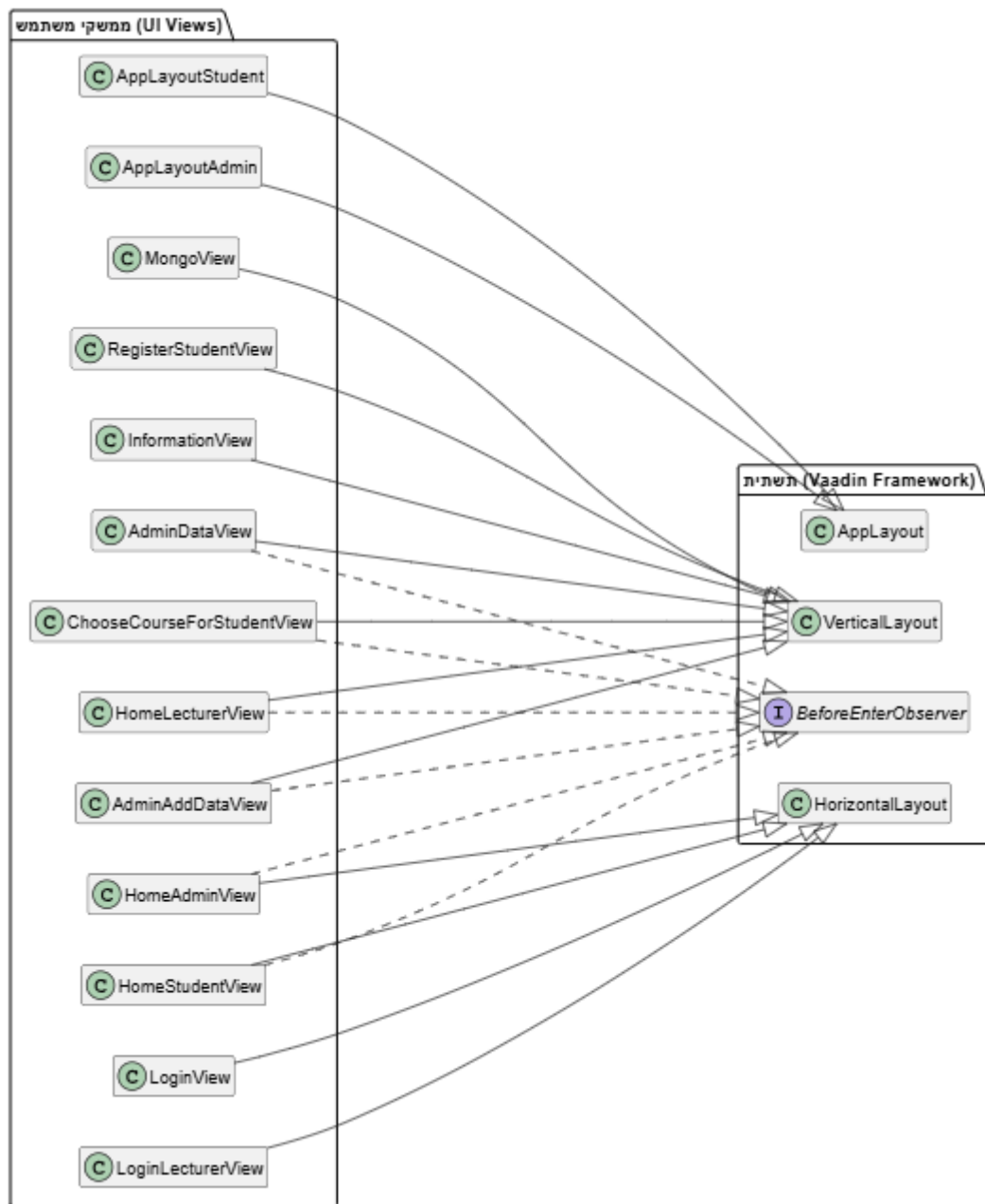


תרשים ה-UML המוצג מתאר את המבנה הארכיטקטוני של שכבת ניהול הנתונים במערכת ואת יחסי הגומלין שלה מול תשתיות האפליקציה ומודלי הנתונים המקומיים. התרשים מורכב משלוש מעטפות (Packages) עיקריות המייצגות את שכבות העבודה השונות: תשתית נתונים המבוססת על Data Models (בחלק העליון), מעטפת ניהול הנתונים הכוללת את ממשקי ה-Repositories של המערכת (במרכז), ומעטפת המודלים המגדירה את ישויות המידע עצמן (בחלק התחתון). חלוקה מובנית זו ממחישה את ההפרדה התקנית והמוחלטת בין הגדרת מבנה הנתונים (הישויות) לבין מנגנוני השליפה, העדכון והאחסון שלהם במסד הנתונים.

במרכז התרשים ממוקמת כאמור מעטפת "ניהול נתונים (Repositories)", אשר מאגדת בתוכה אוסף של ממשקים (Interfaces) כגון UserRepository, ScheduleRepository, CourseRepository, LecturerRepository ו-AdminRepository.

ממשקים אלו משמשים כצינור התקשורת הבלעדי מול בסיס הנתונים עבור כל ישות. מהתרשים עולה בבירור כי כל אחד מממשקים אלו יורש (מסומן באמצעות קו רציף בעל חץ חלול) מהממשק MongoDBRepository, השייך למעטפת העליונה של תשתית Spring Data. ירושה זו מהווה עוגן ארכיטקטוני משמעותי, שכן היא מעניקה למערכת באופן אוטומטי סט רחב של פעולות CRUD (יצירה, קריאה, עדכון ומחיקה) מובנות ומוכנות מראש לעבודה מול מסד נתונים מסוג MongoDB, וזאת מבלי להידרש לכתיבת קוד שאילתות ארוך ומסורבל.

המעטפת התחתונה בתרשים, "מודלים (Data Models)", מכילה את מחלקות הישויות השונות שהמערכת עובדת עמן, כגון Admin, User, Course, Lecturer, ScheduleDocument. התרשים ממחיש קשרי תלות מובהקים (באמצעות חיצים רציפים המצביעים כלפי מטה) בין כל ממשק ניהול נתונים (Repository) לבין מחלקת המודל הספציפית שעליה הוא אחראי. כך לדוגמה, ה-LecturerRepository מקושר ישירות ובלעדית למחלקת Lecturer, וה-ScheduleRepository מקושר למחלקת ScheduleDocument. מיפוי מדויק זה, המחבר בין מאגר נתונים ספציפי לישות ספציפית, שומר על קוד מודולרי בין סוגי נתונים שונים, ומבטיח שמירה על עקרונות תכנות מונחה-עצמים.

תרשים 2 - תלויות וירושה בין ממשק המשתמש לתשתית Vaadin (UI):

תרשים ה-UML המוצג מתאר את מבנה המחלקות בשכבת התצוגה של המערכת (UI Views) ואת יחסי הגומלין שלהן מול רכיבי התשתית של סביבת הפיתוח Vaadin Framework. התרשים בנוי משתי מעטפות (Packages) עיקריות הממחישות את הפרדת האחריות: המעטפת השמאלית מאגדת את כלל מסכי המערכת שפותחו באופן ייעודי, בעוד המעטפת הימנית מרכזת את מחלקות הבסיס והממשקים המובנים שמספקת תשתית Vaadin.

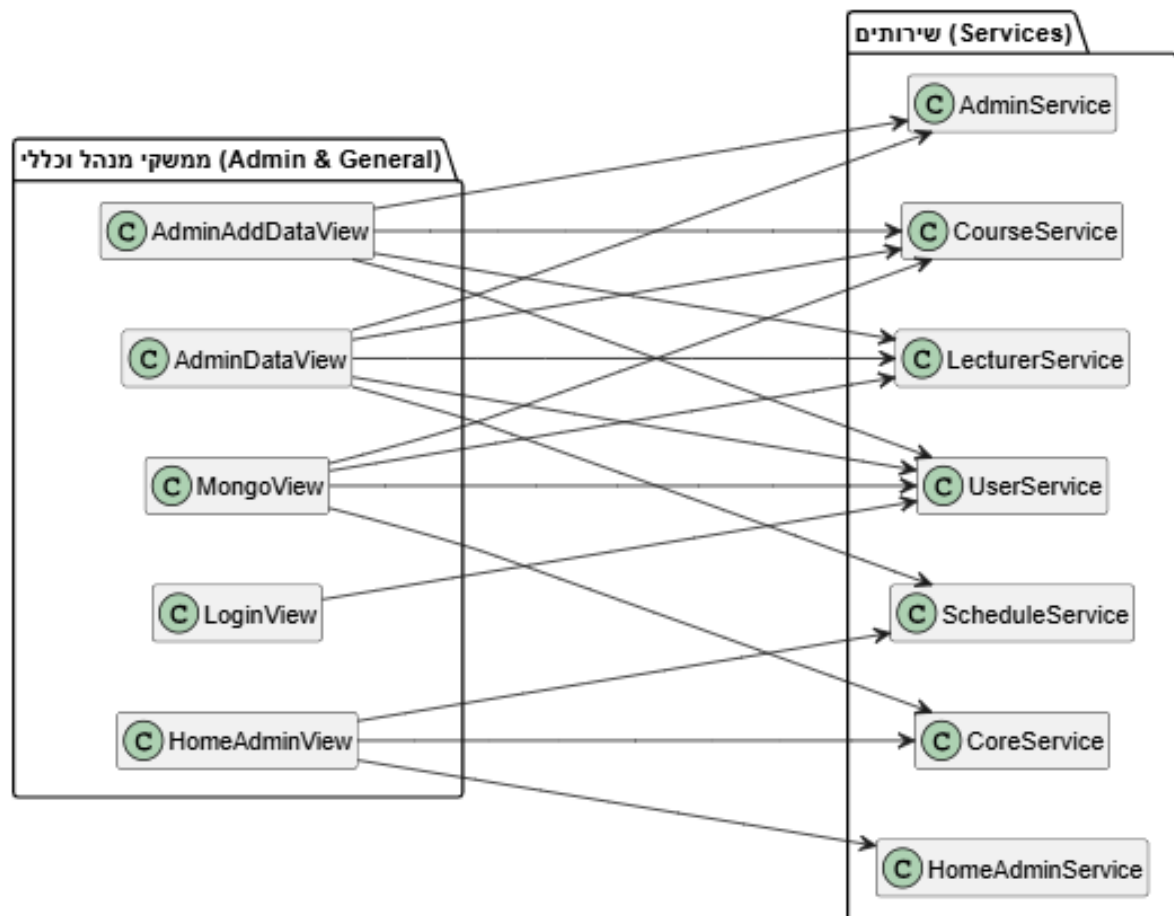
הקווים הרציפים בעלי החץ החלול בתרשים מייצגים יחסי ירושה (Inheritance) ממחלקות בסיס. ניתן לראות כי מסכי התצוגה השונים נחלקים לשני סוגי הורשה עיקריים בהתאם לתפקידם במערכת. קבוצה אחת כוללת מחלקות המשמשות כמעטפות וסרגלי ניווט, כגון AppLayoutStudent ו-AppLayoutAdmin, אשר יורשות ישירות ממחלקת AppLayout של התשתית.

הקבוצה השנייה, הכוללת את מרבית המסכים הפעילים במערכת כדוגמת מסכי הבית (HomeStudentView), מסכי ההתחברות (LoginView, LoginLecturerView) ומסכי ניהול הנתונים (AdminDataView, MongoView), יורשת ממחלקת VerticalLayout. ירושה זו מעניקה למסכים אלו התנהגות מובנית של פריסה אנכית, המאפשרת לסדר את רכיבי המסך (כפתורים, טבלאות, טקסטים) בצורה פשוטה מלמעלה למטה.

בנוסף להורשה הסטנדרטית, התרשים מציג קווים מקווקווים בעלי חץ חלול, אשר מסמלים מימוש של ממשקים (Implements).

מחלקות המייצגות מסכים פנימיים ורגישים, כגון AdminAddDataView, HomeLecturerView, HomeAdminView ו-ChooseCourseForStudentView, מממשות את הממשק BeforeEnterObserver לתשתית Vaadin. ממשק זה הוא חלק חיוני ממנגנון מחזור החיים של המסכים, ותפקידו לאפשר למערכת ליירט את אירוע הכניסה למסך רגע לפני שהוא מוצג למשתמש. מימוש זה משמש בפועל כשכבת אבטחה ובקרת גישה, המאפשרת לוודא שהמשתמש מחובר למערכת ובעל ההרשאות הנדרשות בטרם תינתן לו הגישה לנתונים עצמם.

תרשים 3 - תלויות וזרימת מידע בין שכבת התצוגה של המנהל לשכבת הלוגיקה:



תרשים ה-UML המוצג מתאר את מערכת התלויות וזרימת המידע בין שכבת התצוגה (ממשקי מנהל וכללי) לבין שכבת הלוגיקה העסקית (שירותים) במערכת. התרשים מחולק לשתי מעטפות (Packages) מרכזיות: המעטפת השמאלית מאגדת את מסכי המשתמש המיועדים למנהל המערכת ולפעולות תשתיות, בעוד המעטפת הימנית מרכזת את מחלקות השירות (Services) המכילות את הלוגיקה התפעולית העסקית. החיצים בתרשים מופנים מהמסכים אל מחלקות השירות, דבר הממחיש בצורה ויזואלית את עקרון הפרדת הרשויות בארכיטקטורה: המסכים (Views) אינם מבצעים פעולות חישוב מורכבות או פונים למסד הנתונים בעצמם, אלא מסתמכים לחלוטין על השירותים לצורך משיכת נתונים, עיבודם, וביצוע פעולות המערכת.

ניתן לראות כי המסכים המיועדים לניהול מקיף של נתונים, כגון AdminAddDataView (מסך הוספת נתונים) ו-AdminDataView (מסך תצוגת נתונים), מקיימים קשרי תלות מסועפים עם מספר רב של שירותים, ביניהם UserService, AdminService, LecturerService, CourseService. תלות מרובה זו נובעת מאופיים של מסכי הניהול, אשר נדרשים לאסוף, להציג ולערוך מידע המשלב מספר ישויות במקביל. למשל, כאשר מנהל מזין נתונים למערכת, המסך פונה תחילה ל-UserService כדי לנהל את המידע האישי ולאכוף את השימוש בכתובת הדואר האלקטרוני כמזהה ייחודי ראשי, ובמקביל פונה לשירותים האקדמיים כגון CourseService לצורך קישור המשתמשים להקשרים הלימודיים שלהם. באופן דומה, מחלקת MongoView, המשמשת ככל הנראה לניהול טכני מול מסד הנתונים, מקושרת גם היא לשירותי הליבה השונים כדי לגשת לאוספי הנתונים.

חלקה התחתון של מעטפת הממשקים מציג מסכים בעלי אחריות ממוקדת יותר, אשר התלויות שלהם מצומצמות בהתאמה. מסך ההתחברות LoginView, למשל, תלוי באופן בלעדי ב-UserService לצורך אימות פרטי ההזדהות של המשתמשים מול המערכת. מנגד, מסך הבית של המנהל, HomeAdminView, מתוכנן כלוח בקרה (Dashboard) ולכן מתקשר עם מחלקת שירות ייעודית בשם HomeAdminService האחראית על ריכוז והכנת נתוני התצוגה הראשוניים. בנוסף, מסך זה נעזר בשירותים רוחביים כמו CoreService ו-ScheduleService כדי להציג למנהל תמונת מצב עדכנית של בניית המערכות האקדמיות ופעילות המערכת עם כניסתו.

תרשים 4 - תלויות בין ממשקי סטודנט ומרצה (UI) לשכבת הלוגיקה:

תרשים ה-UML הנוכחי ממשיך את פירוט הארכיטקטורה של שכבת התצוגה, והפעם מתמקד בממשקי המשתמש של הסטודנטים והמרצים (Student & Lecturer) ובקשרי התלות שלהם מול שכבת השירותים (Services). בדומה לתכנון ממשקי המנהל, גם כאן נשמרת בקפידה חלוקת האחריות: המעטפת השמאלית מאגדת את מסכי המערכת (Views) והמעטפת הימנית את השירותים שמכילים את הלוגיקה, כאשר החיצים מראים שזרימת הבקשות עוברת תמיד מהתצוגה אל המנוע העסקי ולא להפך.

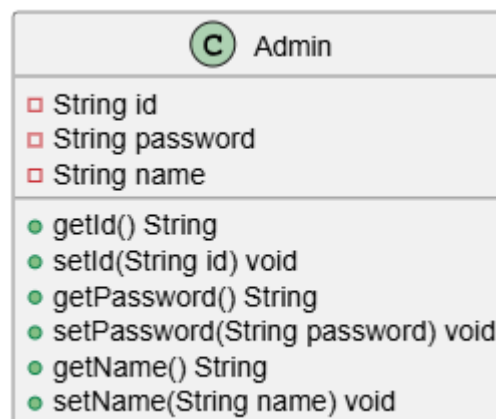
מבט על מסכי הבית של המשתמשים מעלה כי הן HomeLecturerView (מסך הבית למרצה) והן HomeStudentView (מסך הבית לסטודנט) מקיימים תלות כפולה ודומה במבנה שלה. כל מסך תלוי בשירות ייעודי המותאם לו אישית (HomeLecturerService ו-HomeStudentService בהתאמה) לצורך משיכת נתונים הנגזרים מתפקיד המשתמש, אך במקביל שניהם פונים גם לשירות מרכזי משותף בשם CoreService. תכנון זה מצביע על כך שה-CoreService מספק לוגיקת ליבה רוחבית ונתונים בסיסיים הנדרשים לכלל המשתמשים המחוברים במערכת, ובכך תורם למניעת שכפול קוד.

התרשים חושף תלויות נוספות המעידות על השילוב של פונקציונליות ייחודית במערכת. כך למשל, תבנית העיצוב והניווט הכללית של הסטודנט, `AppLayoutStudent`, מקיימת תלות ישירה ב-`WeatherService`. קישור זה מאפשר את שילובו של שירות מזג אוויר בזמן אמת ישירות אל תוך המעטפת של המערכת, מה שמספק לסטודנטים הודעות והתראות קונטקסטואליות חכמות המבוססות על חציית ספי טמפרטורה. בנוסף, ניתן לראות כי מסך ההרשמה לסטודנטים חדשים, `RegisterStudentView`, תלוי באופן בלעדי ב-`UserService`. שירות זה מנהל את תהליך הרישום, מפעיל את הלוגיקה לשמירת הנתונים, ומוודא את שלמותם ואמינותם – למשל על ידי אכיפת השימוש בכתובת דואר אלקטרוני כמשתנה זיהוי ייחודי (Primary Key) למניעת התנגשויות וכפילויות חשבוניות. לבסוף, מסך בחירת הקורסים, `ChooseCourseForStudentView`, המהווה נדבך מרכזי עבור הסטודנט, מתקשר עם `UserService` לצורך זיהוי הסטודנט ונתוניו, ובמקביל נעזר ב-`MongoService` כדי לנהל את מבני הנתונים הייעודיים הנדרשים עבור תהליך הרישום לקורסים. מסך התחברות המרצים, `LoginLecturerView`, מקושר כצפוי ל-`LecturerService` לצורך אימות נתוני המרצה מול מאגר המידע, אך מסתייע גם ב-`AdminService`, דבר העשוי להצביע על בדיקת סטטוס ניהולי, הרשאות מיוחדות הניתנות לחלק מהסגל, או מעורבות של מנהל תוכנית הלימודים בתהליך הכניסה שלהם.

12.2.3 תיאור מחלקות מפורט

נתחיל במחלקות הנמצאות ב `dataModels`:

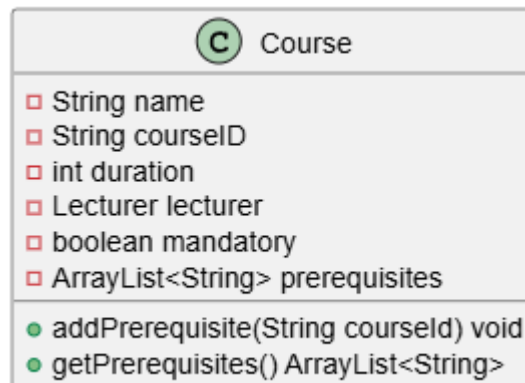
:Admin



מחלקת `Admin` מייצגת את ישות מנהל המערכת בפרויקט וממופה לאוסף ייעודי במסד הנתונים הלא-רלציוני. תפקידה המרכזי הוא לאפשר זיהוי, אימות והרשאות גישה למשתמשים בעלי סמכויות ניהוליות, כגון הזנת נתונים ראשוניים למערכת וחישוב מחדש של מערכת השעות הגלובלית. המחלקה כוללת את השדה `id`, שהוא שדה מחרוזת המשמש כמזהה הייחודי של המנהל במערכת ומוגדר באמצעות האנוטציה `@Id` לצורך ניהול מפתחות במסד הנתונים.

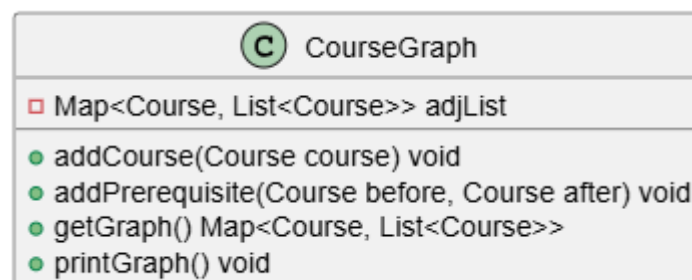
בנוסף, המחלקה שומרת את סיסמת הגישה של המנהל תחת השדה password לצורך תהליך האימות. פעולות הגישה השונות (Setters ו-Getters) הכלולות במחלקה מהוות את הממשק הבלעדי להגדרת ושליפת נתוני הישות, תוך שמירה על עקרון הכימוס (Encapsulation).

:Course



מחלקת Course היא ישות ליבה במערכת המייצגת קורס אקדמי בודד. היא אוגרת את כל המידע הפדגוגי והמנהלתי הנחוץ לניהול הקורס, שיוכו לסגל המרצים והגדרת התלויות האקדמיות שלו שישמשו את אלגוריתמי השיבוץ במערכת. המחלקה מכילה את המזהה החד-ערכי courseID, המשמש כעוגן לקשרי הגומלין, ואת השדה duration המציין את אורך הקורס או את סמסטר הפתיחה שלו. תכונה מרכזית נוספת היא lecturer, המהווה קשר הכלה (Composition) לאובייקט מסוג Lecturer שמגדיר את המרצה האחראי. רשימת קורסי הקדם מנוהלת על ידי המערך הדינאמי prerequisites, המכיל מזחים של קורסים שחובה לסיים לפני ההרשמה לקורס זה. המחלקה כוללת את הפעולה addPrerequisite, אשר מאפשרת להוסיף קורס קדם לרשימה תוך ביצוע בדיקת תקינות מובנית המונעת הכנסת כפילויות למערך.

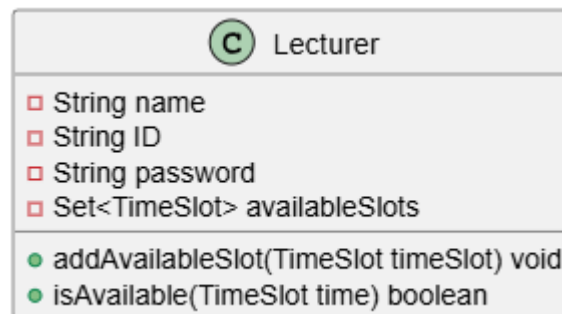
:CourseGraph



מחלקת CourseGraph היא מחלקה מבנית האחראית על ייצוג והרכבת גרף הקורסים האקדמי של המערכת. הגרף ממומש כגרף מכון נטול לולאות (DAG - Directed Acyclic Graph), בו הקודקודים הם קורסים והקשתות מייצגות קשרי קדימות ותנאי קדם לימודיים. מבנה זה הוא חיוני ומקדים לצורך הרצת אלגוריתם המיון הטופולוגי אשר מחשב את סדר הלמידה התקין עבור הסטודנט.

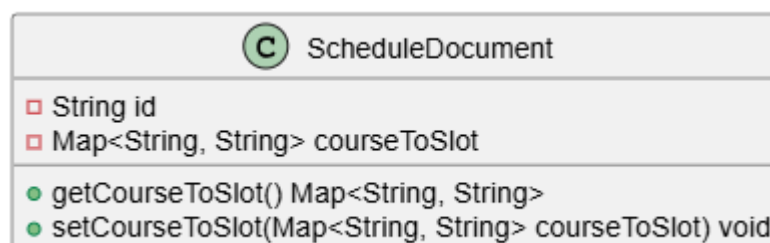
בבסיס המחלקה ניצב המילון `adjList`, הממומש כ-`LinkedHashMap`, שבו המפתח הוא קורס והערך הוא רשימת הקורסים התלויים בו. השימוש במבנה נתונים זה מבטיח שמירה על סדר ההכנסה לצורך יציבות האלגוריתם. המחלקה מספקת את הפעולה `addCourse` להוספת קודקוד חדש לגרף, ואת הפעולה `addPrerequisite` המייצרת קשת מכוונת מקורס הקדם אל הקורס העוקב, ובכך בונה את שרשרת התלויות בפועל.

Lecturer:



מחלקת `Lecturer` מייצגת את ישות המרצה במערכת האקדמית, ותפקידה העיקרי הוא לנהל את פרטי אימות הזהות של חבר הסגל ואת אילוצי הזמן שלו. מידע מערכתי זה מהווה תנאי הכרחי בתהליך שיבוץ הקורסים האוטומטי ומונע שיבוצים חופפים. המחלקה מכילה אוסף בשם `availableSlots` הממומש כקבוצה (`Set`) ונועד להחזיק אובייקטים של משבצות זמן (`TimeSlot`) שבהן המרצה הצהיר על פנייתו להוראה. השימוש במבנה נתונים מסוג זה מונע לחלוטין שמירה של משבצות זמן כפולות עבור אותו מרצה. ניהול יומן זה מתבצע באמצעות הפעולה `addAvailableSlot` המאפשרת הוספת חלון זמן פנוי. פעולה קריטית ומרכזית נוספת היא `isAvailable`, אשר מבצעת איטרציה על פני יומן הפניות של המרצה ובודקת האם קיימת חפיפה עם משבצת זמן מבוקשת, תהליך המשמש את אלגוריתם השיבוץ לגיזום אפשרויות לא חוקיות (`Pruning`).

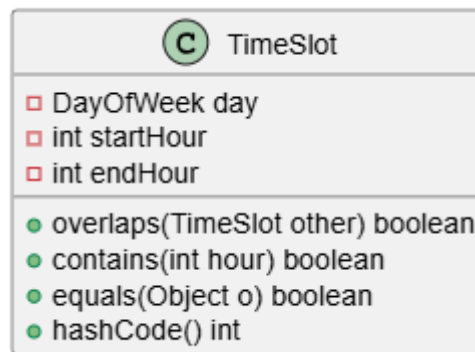
ScheduleDocument:



מחלקת `ScheduleDocument` משמשת כישות נתונים קבועה (`Persistent Entity`) המייצגת את מסמך מערכת השעות הסופי והגלובלי של המוסד. מסמך זה נשמר ישירות בבסיס הנתונים הלא-רלציוני ומאפשר למערכת כולה לשלוף במהירות את השיבוץ המוגמר, מבלי הצורך להריץ ולחשב את אלגוריתם השיבוץ מחדש בכל טעינת מסך.

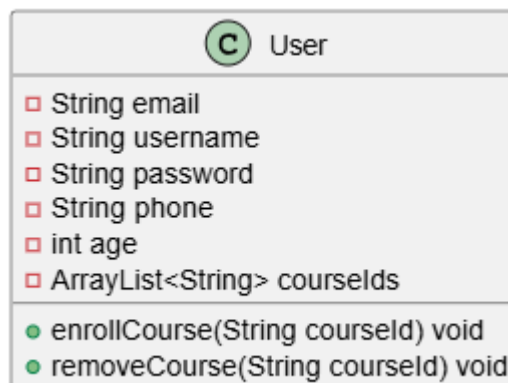
במרכז המחלקה נמצאת מפת הגיבוב `courseToSlot`, הממפה בין מזהה הקורס כמחרוזת לבין ייצוג מחרוזתי של משבצת הזמן שנקבעה לו, כגון "SUNDAY 8:00-10:00". מבנה זה מאפשר גישה מהירה ויעילה בזמן ריצה קבוע ($O(1)$) עבור כל רכיבי ממשק המשתמש שניגשים להציג את מערכות השעות עבור המשתמשים השונים במערכת.

:TimeSlot



מחלקת `TimeSlot` היא מחלקה מבנית בסיסית המגדירה משבצת זמן קלנדרית אטומית במערכת, התחומה בשעות מוגדרות עבור יום ספציפי בשבוע. מחלקה זו משמשת כבסיס הלוגי והמתמטי לחישוב התנגשויות וניהול יומני המרצים והקורסים. היא כוללת את התכונה `day`, המשתמשת בטיפוס המנוהל `DayOfWeek` לייצוג יום העבודה, ואת משתני השעות `startHour` ו-`endHour` הנגזרים כמספרים שלמים. הפעולה הלוגית המרכזית במחלקה היא `overlaps`, אשר מקבלת משבצת זמן אחרת ומוודאת האם שעת ההתחלה של האחת קטנה משעת הסיום של השנייה ולהיפך, ובכך בודקת חפיפה ומונעת התנגשויות תזמון במערכת. בנוסף, המחלקה מכילה דריסה (`Override`) של מתודות הבסיס `equals` ו-`hashCode`, פעולה הכרחית המאפשרת השוואת ערכים מדויקת בין משבצות זמן שונות ואת אחסון התקין במבני נתונים מתקדמים המבוססים על טבלאות גיבוב.

:User



מחלקת `User` מייצגת את ישות הסטודנט במערכת, ומכילה את נתוני הפרופיל האישיים שלו, נתוני ההתחברות, ואת רשימת הקורסים אליהם הוא משובץ או רשום.

רשימת הקורסים הייעודית של כל משתמש מהווה את הקלט המרכזי למערכת לבניית תוכנית הלימודים האוטומטית. המחלקה מגדירה את שדה המחרוזת email כמפתח החד-ערכי של הסטודנט במערכת באמצעות האנוטציה @Id, בחירה הנדסית המבטיחה זיהוי חד-משמעי ומונעת מצבי קצה של רישום כפול. מעקב הרישום עצמו מנוהל תחת המערך הדינמי courseIds האוגר את מזהי הקורסים הנבחרים. על מנת לנהל את הרישום באופן בטיחותי, המחלקה מספקת את הפעולה enrollCourse שמוסיפה מזהה קורס למערך רק לאחר ביצוע של פעולת אימות שמונעת כפילויות זהות ברשימה, וכן את הפעולה removeCourse המאפשרת את הסרתו של קורס ממערך הלימודים.

נמשך במחלקות הנמצאות ב repositories:

:AdminRepository



ממשק AdminRepository מהווה את שכבת הגישה לנתונים עבור ישות המנהל (Admin) במערכת. ממשק זה יורש ישירות מהממשק הגנרי MongoRepository של ספריית Spring Data, ובכך מקבל באופן אוטומטי ובחינם אוסף רחב של פעולות CRUD (יצירה, קריאה, עדכון ומחיקה) מול מסד הנתונים הלא-רלציוני. מכיוון שישות המנהל אינה דורשת בשלב זה שאילתות מורכבות או חיפושים חריגים מעבר לחיפוש לפי המזהה הייחודי, הממשק נותר ריק ונסמך לחלוטין על הפעולות המובנות של ספריית התשתית, כגון שמירת מנהל חדש או משיכת כלל המנהלים.

:CourseRepository

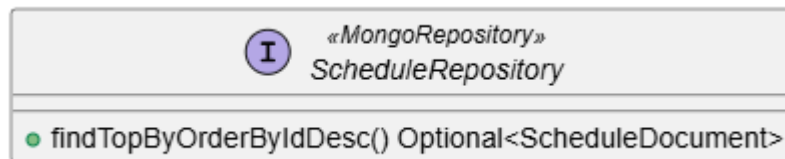


ממשק CourseRepository מספק את מנגנון התקשורת מול אוסף הקורסים במסד הנתונים. בנוסף לפעולות הבסיסיות אותן הוא יורש מ-MongoRepository, ממשק זה מרחיב את יכולות השאילתה באמצעות שתי פעולות מותאמות אישית (Custom Queries) המוגדרות לפי מוסכמות השמות של Spring Data. הפעולות findByIdIn ו-findAllByIdIn מקבלות שתיהן רשימה של מזהי קורסים (Strings) ומחזירות רשימה מתאימה של אובייקטי Course. פעולות אלו משמעותיות ביותר עבור המערכת, שכן הן מאפשרות שליפה יעילה של כלל הקורסים אליהם רשום סטודנט מסוים בשאילתה אחת בודדת, במקום לפנות למסד הנתונים שוב ושוב עבור כל קורס בנפרד.

LecturerRepository:

ממשק `LecturerRepository` אחראי על ניהול התקשורת ושליפת הנתונים מול מסד הנתונים עבור ישויות המרצים במערכת. לצד פעולות המחיקה והשמירה הסטנדרטיות המורשות מ-`MongoRepository`, הממשק מגדיר פעולת חיפוש ייעודית בשם `findById`.

פעולה זו מקבלת מחרוזת המייצגת תעודת זהות של מרצה, ומחזירה את אובייקט ה-`Lecturer` התואם מתוך בסיס הנתונים. קיומה של פעולה זו קריטי לשלב אימות הנתונים (`Authentication`) במסך ההתחברות, שם המערכת נדרשת לאתר במהירות את המרצה לפי תעודת הזהות שהזין טרם בדיקת התאמת הסיסמה.

ScheduleRepository:

ממשק `ScheduleRepository` מנהל את השמירה והשליפה של מסמכי מערכת השעות המוגמרת של המוסד האקדמי מתוך אוסף ה-`Schedule`. תכונתו המשמעותית ביותר של ממשק זה היא שאילתת השליפה המותאמת אישית `findTopByOrderByldDesc`. פעולה זו מנחה את מסד הנתונים למיין את כל מסמכי מערכת השעות הקיימים בסדר יורד על פי המזהה שלהם, ולהחזיר רק את המסמך הראשון והעדכני ביותר (עטוף באובייקט `Optional` כדי למנוע שגיאות של הפניות ריקות). מנגנון זה מבטיח שאלגוריתם הליבה, וכן רכיבי התצוגה של המשתמשים, יעבדו תמיד מול הגרסה העדכנית ביותר של מערכת השעות שחושבה ושמורה במערכת.

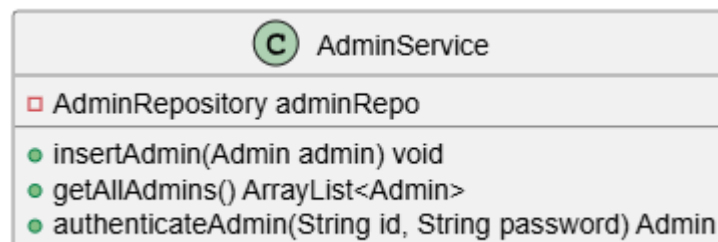
UserRepository:

ממשק `UserRepository` מתפקד כשכבת הגישה לנתונים עבור ישויות הסטודנטים במערכת, ומאפשר את ביצוע הפעולות הנדרשות לניהול משתמשים כגון הרשמה, התחברות ועדכון מערך

הקורסים האישי. הממשק מגדיר שתי פעולות חיפוש מתקדמות: הפעולה `findByUsername` מאפשרת שליפה ישירה של אובייקט סטודנט על בסיס שם המשתמש שלו, והפעולה `findByEmail` המאתרת משתמש על סמך כתובת הדואר האלקטרוני שלו. מכיוון שכתובת המייל משמשת כמפתח הראשי של הסטודנט, השליפה באמצעות פעולה זו מחזירה אובייקט `Optional` מוגן, המסייע למנוע שגיאות ריצה במקרים שבהם ניסיון ההתחברות או ההרשמה נעשה עם כתובת שאינה קיימת במאגר.

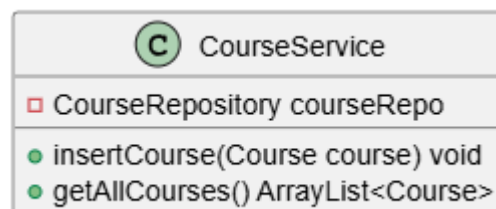
נמשך במחלקות הנמצאות ב `services` :

:AdminService



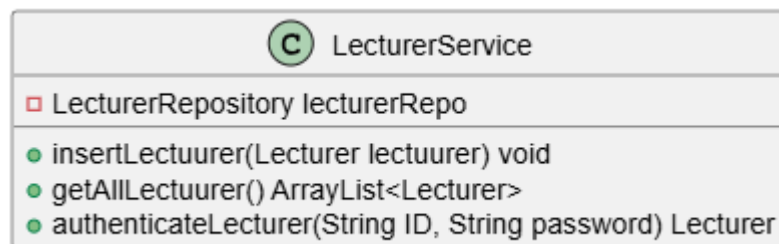
מחלקת `AdminService` משמשת כשכבת הלוגיקה העסקית (Business Logic) עבור ניהול ישויות המנהלים במערכת. המחלקה מקבלת בהזרקת תלויות (Dependency Injection) את `AdminRepository` כדי לתקשר עם מסד הנתונים. הפעולה המרכזית `authenticateAdmin` מקבלת תעודת זהות וסיסמה, שולפת את אובייקט המנהל מהמסד ומוודאת את נכונות הסיסמה, תוך זריקת חריגה מבוקרת (Exception) במקרה של אימות שנכשל. פעולה נוספת היא `insertAdmin` הדואגת לוודא שמנהל בעל תעודת זהות לא קיים כבר במערכת בטרם היא מאשרת את שמירתו.

:CourseService

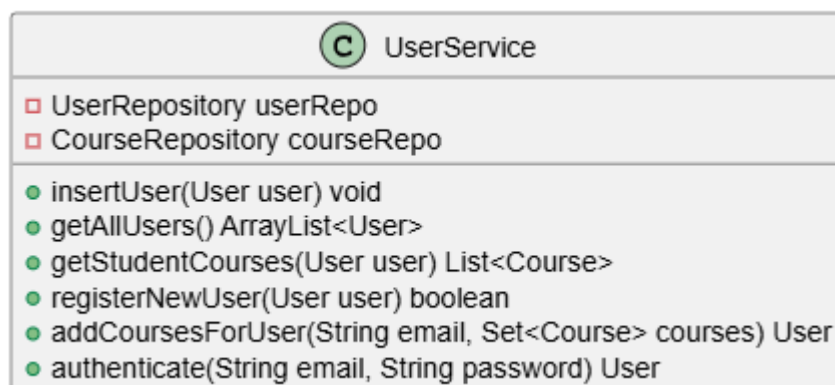


מחלקת `CourseService` מרכזת את הפעולות העסקיות הקשורות לניהול קורסים. תפקידה הוא לוודא את תקינות הנתונים בטרם הם נשלחים לאחסון קבוע דרך `CourseRepository`.

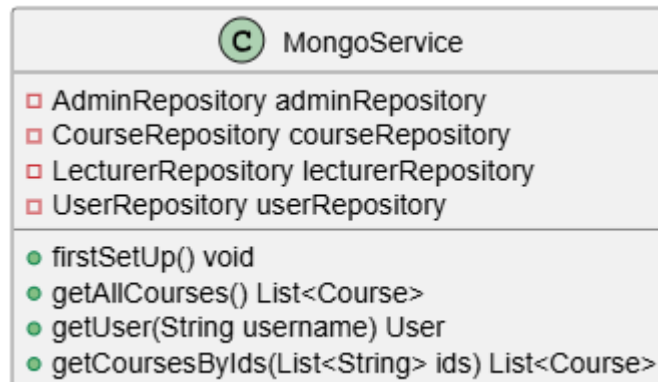
הפעולה `insertCourse` מקבלת אובייקט של קורס חדש ומוודאת כי מזהה הקורס (`CourseID`) הוא ייחודי ואינו בשימוש על ידי קורס אחר במערכת, פעולה החיונית לשמירה על שלמות הנתונים (Data Integrity).

LecturerService:

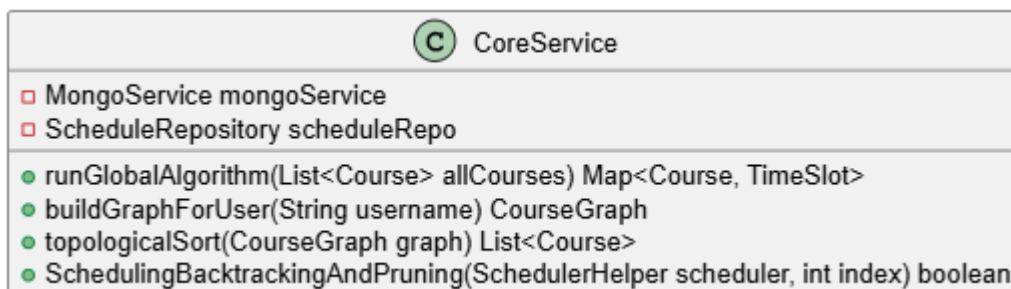
מחלקת LecturerService מנהלת את הלוגיקה הקשורה לחברי הסגל והמרצים. בדומה לשירותי משתמשים אחרים, היא מספקת את מנגנון האימות ההכרחי דרך הפעולה authenticateLecturer, אשר שולפת את נתוני המרצה לפי תעודת הזהות שלו ומשווה את הסיסמה המוצפנת כדי לאפשר גישה מאובטחת לאזור האישי שלו. כמו כן, הפעולה insertLectuurer (נשמרה התחביר המקורי) אחראית על רישום מרצים חדשים תוך מניעת כפילויות.

UserService:

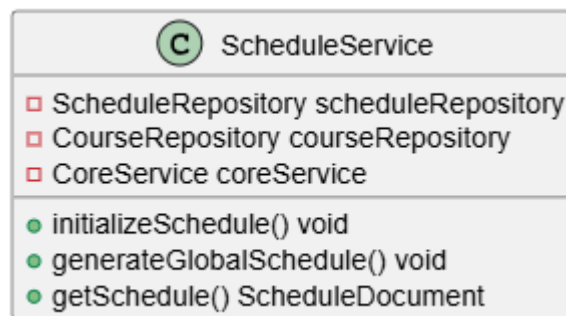
מחלקת UserService היא אחת המחלקות המרכזיות לניהול משתמשי הקצה (הסטודנטים). היא מרכזת תהליכים מורכבים כמו הרשמה רב-שלבית, המנוהלת על ידי הפעולה registerNewUser השומרת נתוני פרופיל בסיסיים, ולאחריה הפעולה addCoursesForUser המעדכנת את רשימת מזהי הקורסים הנבחרים של הסטודנט. בנוסף, המחלקה מספקת את מנגנון האימות המרכזי לסטודנטים באמצעות הפעולה authenticate המבוססת על כתובת הדואר האלקטרוני שלהם המשמשת כמפתח ראשי.

:MongoService

מחלקת MongoService מתפקדת כשכבת שירות גלובלית וראשונית מול מסד הנתונים. פעולתה המשמעותית ביותר היא firstSetUp, האחראית על אתחול מסד הנתונים והזרקת נתוני דמה (Mock Data) מקיפים לצורך בדיקות, כולל יצירת זמני הוראה, מרצים, עץ קורסים ענף המכיל תנאי קדם מורכבים, הרשמת סטודנטים לדוגמה ויצירת משתמש מנהל מוגדר מראש. המחלקה מאפשרת למערכת לעלות באופן עצמאי עם נתונים מוכנים להרצת אלגוריתמי השיבוץ.

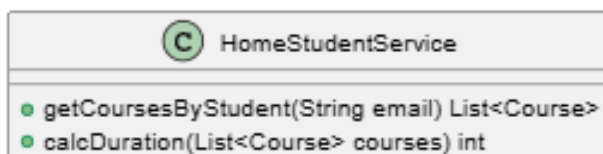
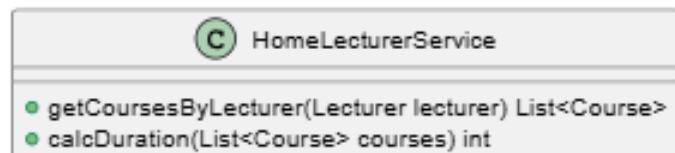
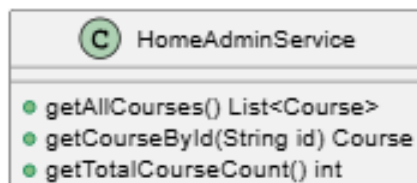
:CoreService

מחלקת CoreService מהווה את הליבה האלגוריתמית והמתמטית של המערכת כולה. היא מרכזת את בניית מערכת השעות האוטומטית באמצעות רצף פעולות מורכב: הפעולה runGlobalAlgorithm מהווה את נקודת הכניסה לאלגוריתם הכללי, אשר בתורו נעזר ב-buildGraphForUser ליצירת תרשים תלויות גרפי של כלל הקורסים הרלוונטיים. לאחר מכן, הפעולה topologicalSort סורקת את הגרף באמצעות סריקת עומק (DFS) כדי לחלץ סדר למידה חוקי המכבד את דרישות הקדם. לבסוף, הפעולה הרקורסיבית SchedulingBacktrackingAndPruning משתמשת בטכניקות של נסיגה לאחור (Backtracking) וגיוזום ענפים (Pruning) כדי לשבץ כל קורס למשבצת זמן מתאימה התואמת לאילוצי המרצים וללא חפיפות, תוך שהיא נעזרת במחלקת העזר SchedulerHelper.

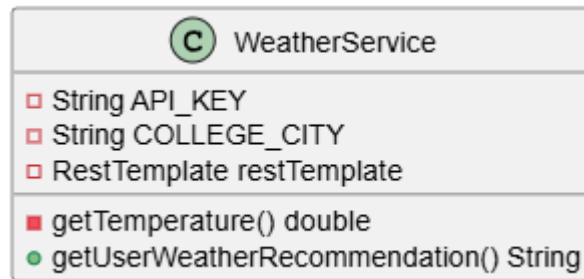
:ScheduleService

מחלקת ScheduleService מופקדת על ניהול התוצר הסופי של המערכת – מערכת השעות המוגמרת. המחלקה מכילה את הפעולה initializeSchedule המופעלת באופן אוטומטי בעת עליית השרת באמצעות אנוטציית @PostConstruct. מתודה זו קוראת לפעולה המרכזית generateGlobalSchedule, אשר שואבת את כל הקורסים ממסד הנתונים, מפעילה את שירותי הליבה (CoreService) לחישוב השיבוץ המושלם, ממירה את תוצאות האלגוריתם למבנה של מילון מזהים, ושומרת את המסמך הסופי באופן קבוע במסד הנתונים דרך שכבת ה-Repository, תוך מחיקת גרסאות קודמות כדי להבטיח עדכניות.

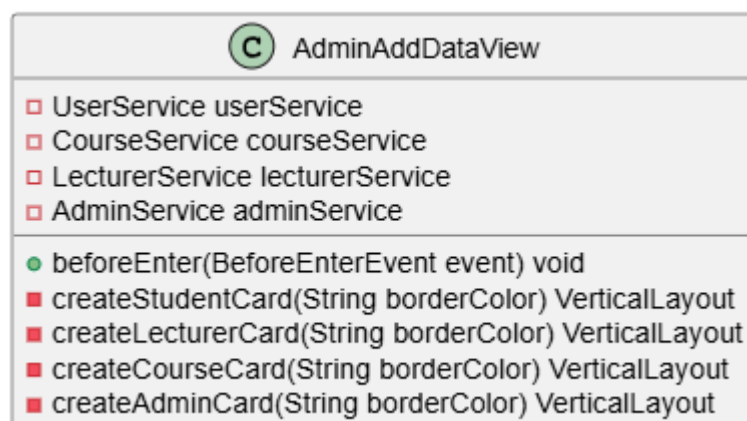
:HOME (HomeAdminService, HomeLecturerService, HomeStudentService)



מחלקות אלו משמשות כשירותי מעטפת (Facade) ייעודיים התומכים באופן ישיר בתצוגת מסכי הבית של המשתמשים השונים. מחלקת HomeAdminService מספקת פעולות מהירות למנהל, כגון ספירה יעילה של כמות הקורסים המוחלטת במערכת באמצעות הפונקציה המובנית getTotalCourseCount. מחלקת HomeLecturerService מסננת ומושכת אך ורק את הקורסים המשייכים למרצה ספציפי באמצעות הפעולה getCoursesByLecturer. באופן דומה, מחלקת HomeStudentService אחראית לשליפת מסלול הקורסים האישי של הסטודנט. שתי מחלקות אלו כוללות גם את הפעולה המשותפת רעיונית calcDuration, אשר רצה על מערך הקורסים המסונן ומסכמת את סך שעות ההוראה או נקודות הזכות עבור תצוגת הסטטיסטיקות במסך המשתמש.

:WeatherService

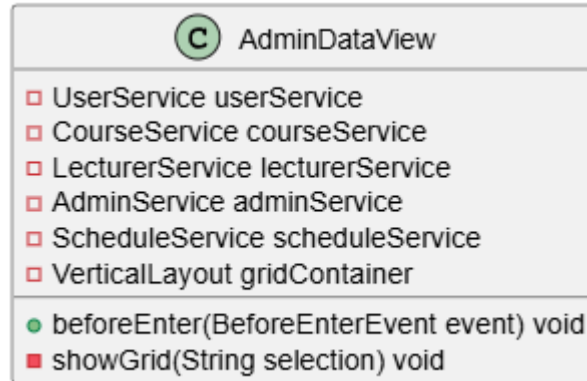
מחלקת WeatherService היא שירות אינטגרציה חיצוני המעשיר את חווית הסטודנט על ידי חיבור ל-API חיצוני של מזג אוויר (OpenWeatherMap). הפעולה הפרטית getTemperature מבצעת קריאת רשת (HTTP GET) באמצעות אובייקט RestTemplate ומחלצת את נתוני הטמפרטורה הנוכחית של העיר המוגדרת מתוך קובץ ה-JSON המוחזר, תוך מתן טמפרטורת גיבוי במקרה של כשל תקשורת. הפעולה הפומבית getUserWeatherRecommendation מעבדת את הטמפרטורה שנשלפה ומייצרת מחרוזת המלצה טקסטואלית ידידותית ומותאמת אישית לסטודנט, המייצעת לו כיצד להתלבש או להיערך לקראת הגעתו לקמפוס.

: נעבור במחלקות הנמצאות ב UI ::AdminAddDataView

מחלקת AdminAddDataView משמשת כמסך הניהול המרכזי של האדמין המיועד להוספת ישויות חדשות למאגר הנתונים של המערכת. המסך מעוצב בצורה מודרנית וצבעונית המורכבת מארבעה כרטיסי קלט נפרדים הנוצרים על ידי פונקציות העזר הפרטיות createStudentCard, createLecturerCard, createCourseCard, ו-createAdminCard, כאשר כל כרטיס מכיל טופס ייעודי עם שדות קלט מותאמים (כמו בחירת קורסים מרובה או הוספת חלונות זמן דינמיים למרצה). המחלקה מממשת את הממשק BeforeEnterObserver וכוללת את הפעולה beforeEnter המבצעת בדיקת אבטחה קריטית בסשן, ומוודאת כי המשתמש הנוכחי הוא אכן מנהל מערכת מורשה טרם אישור הצגת התוכן.

בעת לחיצה על כפתורי השמירה, המסך מפעיל לוגיקת ולדיציה קפדנית הכוללת בדיקת אורך סיסמה, פורמט אימייל תקין, בדיקות גיל ומניעת כפילויות של מזהים מול השירותים השונים.

:AdminDataView



מחלקת AdminDataView משמשת כמסך התצוגה הכולל של המנהל, ותפקידה להציג בצורה מרוכזת ונוחה את כל הנתונים הקיימים במערכת בתוך טבלאות נתונים מתקדמות (Grids). המסך מכיל רכיב ComboBox מרכזי המאפשר למנהל לבחור את קטגוריית המידע המבוקשת (סטודנטים, מרצים, קורסים, מנהלים או את מערכת השעות הגלובלית שחושבה). בחירה זו מפעילה את מתודת העזר, showGrid, אשר מנקה דינמית את מיכל הטבלאות gridContainer ומייצרת כרטיס תצוגה מותאם המושך את המידע העדכני ביותר משכבות השירות והריפוזיטורי ומציג אותו בגריד מרובה עמודות, כולל תצוגה מרוכזת של מפת השיבוצים של מערכת השעות הגלובלית.

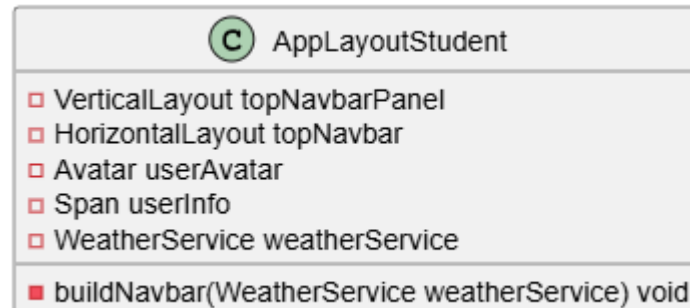
:AppLayoutAdmin



מחלקת AppLayoutAdmin מהווה את תבנית העיצוב והשלד המרכזי (Layout) עבור כל מסכי הניהול של האדמין במערכת. המחלקה יורשת מ-AppLayout של Vaadin ומגדירה סרגל ניווט עליון קבוע (Top Navbar) המלווה את המנהל בכל דפי הניהול. פעולת הבנייה buildNavbar מעצבת את הסרגל בצורה מובנית המורכבת משלושה חלקים: בצד ימין מוצג לוגו המערכת בגוון אדום המאפיין את אזור הניהול, במרכז מוצבים כפתורי ניווט מהירים למעבר בין מסך הבית, מסך הוספת הנתונים ומסך הצגת הטבלאות, ובצד שמאל מופיעים פרטי המנהל המחובר לצד רכיב האוואטר הוויזואלי שלו השלופ

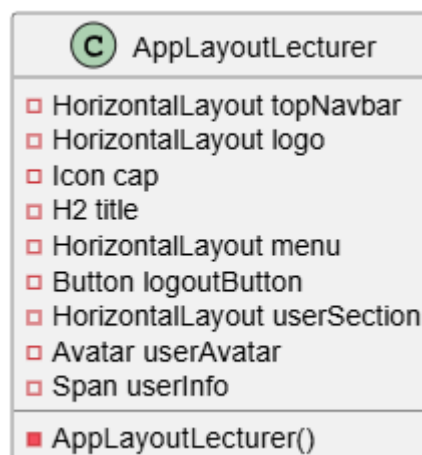
מתוך ה-VaadinSession. הסרגל כולל גם כפתור התנתקות המבצע אינולידציה מלאה לסשן ומפנה את המשתמש חזרה למסך הלוגין.

:AppLayoutStudent



מחלקת AppLayoutStudent משמשת כשלד העיצובי והניווטי הראשי עבור אזור הסטודנטים במערכת. בדומה לגרסת המנהל, היא מייצרת בר ניווט עליון קבוע, אך מותאמת באופן ייחודי לחוויית הסטודנט. תכונתה הייחודית של מחלקה זו היא השילוב של WeatherService ישירות בתוך רכיבי התפריט העליון. במהלך ריצת הפעולה buildNavbar, המערכת פונה לשירות מזג האוויר, שולפת את ההמלצה הלוגית המעודכנת ומציגה אותה כתווית ייעודית (weatherBadge) מעוצבת בגוון תכלת נעים בצמוד ללוגו האקדמי, דבר המאפשר לסטודנט לקבל התרעה על תנאי האקלים בקמפוס מיד עם כניסתו.

:AppLayoutLecturer



AppLayoutLecturer מהווה את תבנית העיצוב והשלד המרכזי (Layout) עבור מסכי ה"איזור אישי סגל" במערכת. המחלקה יורשת מ-AppLayout של Vaadin ומגדירה סרגל ניווט עליון קבוע (Top Navbar) המלווה את המרצה בכל דפי האזור האישי שלו. הבנאי של המחלקה מעצב את הסרגל בצורה מובנית המורכבת משלושה חלקים: בצד ימין מוצג לוגו הכולל כובע אקדמי כחול וכותרת המאפיינת את אזור הסגל, במרכז מוצב כפתור התנתקות, ובצד שמאל מופיעים פרטי המרצה המחובר

לצד רכיב האוואטר הוויזואלי שלו הנשלפים מתוך ה-VaadinSession. לחיצה על כפתור ההתנתקות מבצעת ניקוי ואינוולידציה מלאה לסשן (Session) ומפנה את המשתמש חזרה למסך הלוגין.

:ChooseCourseForStudentView

C ChooseCourseForStudentView
○ MongoService mongoService ○ UserService userService ○ List<Course> allCourses ○ VerticalLayout selectedCoursesList ○ Span creditsLabel ○ Map<Integer, CheckboxGroup<Course>> semesterGroups ○ Button finishChooseBtn ○ Set<Course> selectedCourses
■ createSemesterColumn(HorizontalLayout parent, String title, List<Course> courses, int semesterIdx) CheckboxGroup ■ createSummaryColumn() VerticalLayout ■ moveToHome() void ■ refreshTotalSummary() void

מחלקת ChooseCourseForStudentView מנהלת את מסך בחירת הקורסים האינטראקטיבי, המהווה את השלב השני והקריטי בתהליך הרישום של סטודנט חדש במערכת. המסך מעוצב במבנה של לוח עבודה מתוחכם המציג את כל קורסי המוסד מחולקים לעמודות לפי סמסטרים (א', ב' או קיץ) באמצעות פונקציית העזר createSemesterColumn, כאשר כל סמסטר מקבל ייצוג צבעוני ייחודי וקבוצת צ'קבוקסים עצמאית המנוהלת במפת השורות semesterGroups. בצד המסך ניצב טור סיכום דינמי (summaryColumn) המופעל על ידי הפעולה refreshTotalSummary בכל פעם שהסטודנט מסמן או מסיר קורס. טור זה מציג בזמן אמת את שמות הקורסים שנבחרו ומעדכן את כמותם. כפתור הסיום finishChooseBtn נותר חסום כל עוד לא נבחר דבר, ובלחיצתו מופעלת המתודה moveToHome השולפת את המייל של המשתמש מהסשן, מעדכנת את רשימת הקורסים האישיים שלו במסד הנתונים דרך UserService ומעבירה אותו אל מסך הבית האישי.

:HomeAdminView

C HomeAdminView
○ CoreService coreService ○ HomeAdminService homeAdminService ○ ScheduleService scheduleService ○ Div calendarGrid
● beforeEnter(BeforeEnterEvent event) void ■ showGlobalSchedule() void ■ createEventCard(String title, String time, String subTitle, String color, int col, int startRow, int rowSpan) Div

מחלקת HomeAdminView משמשת כמסך הבית הראשי של מנהל המערכת ומציגה את לוח השנה המוסדי הגלובלי המורכב מגריד שעות וימים מורכב. במרכז המסך נמצא רכיב calendarGrid המעוצב באמצעות תכונות CSS Grid מתקדמות המגדירות שורות עבור שעות הפעילות (08:00 עד 18:00) ועמודות עבור ימי הלימוד (א'-ו').

הפעולה המרכזית `showGlobalSchedule` שולפת את מסמך השיבוץ המערכתי מ-`CoreService`, מפרקת את מחרוזות הזמנים הכלליות ומציגה אותן בצורה גרפית על גבי הלוח באמצעות כרטיסי אירוע צבעוניים הנוצרים דינמית במתודה `createEventCard` ומציגים את שם הקורס, השעה ושם המרצה השיבוץ. בצד המסך קיים כפתור אדום בולט המקושר לשירות `ScheduleService`, המאפשר למנהל להפעיל בלחיצת כפתור את אלגוריתם ה-`Backtracking` והמיון הטופולוגי מחדש על כלל הקורסים ולעדכן את בסיס הנתונים בזמן אמת.

:HomeLecturerView

HomeLecturerView
- HomeLecturerService homeServiceLecturer - List<Course> lecturerCourses - CoreService coreService - Div calendarGrid
- beforeEnter(BeforeEnterEvent event) void - privateSchedule() void - createEventCard(String title, String time, String room, String color, int col, int startRow, int rowSpan) Div

מחלקת `HomeLecturerView` מגדירה את מרכז העבודה והאזור האישי של המרצה במערכת. עם כניסת המרצה המורשה, המערכת שולפת את רשימת הקורסים הספציפית שהוא מלמד באמצעות `HomeLecturerService` ומציגה אותם בסרגל הצידי לצד כרטיסי סטטיסטיקה המציגים את סך שעות ההוראה המצטברות שלו. הפעולה המשמעותית ביותר במסך היא `privateSchedule`, המופעלת בלחיצה על כפתור תצוגת המערכת. מתודה זו מושכת את המערכת הגלובלית השמורה, מסננת מתוכה אך ורק את הקורסים המשויכים למזהה המרצה הנוכחי, ומציגה אותם על גבי הגריד `calendarGrid`, ובכך מייצרת עבור המרצה לוח שעות אישי המונע ממנו את הצורך לחפש את שעותיו בלוח המוסדי הכללי.

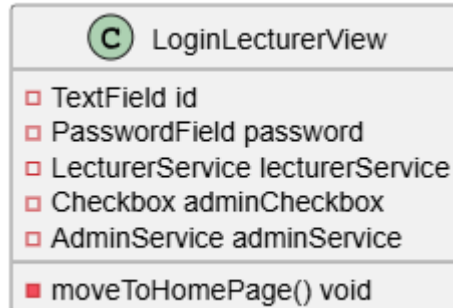
:HomeStudentView

HomeStudentView
- HomeStudentService homeServiceStudent - List<Course> studentCourses - CoreService coreService - Div calendarGrid
- beforeEnter(BeforeEnterEvent event) void - privateSchedule() void - createEventCard(String title, String time, String room, String color, int col, int startRow, int rowSpan) Div

מחלקת `HomeStudentView` אמונה על הצגת אזור העבודה האישי של הסטודנט, המשקף לו את תוכנית הלימודים שנבנתה עבורו. המסך מציג ברשת שעות שבועית את מערכת השעות המותאמת שלו. בעת לחיצה על כפתור הצגת המערכת, מופעלת הפעולה `privateSchedule`, אשר פונה לשירות הליבה ומושכת את מסמך מערכת השעות המוסדית העדכנית ביותר. הלוגיקה של הפעולה מבצעת התאמה בין רשימת הקורסים אליהם רשום הסטודנט (הנשלפת באמצעות `HomeStudentService`)

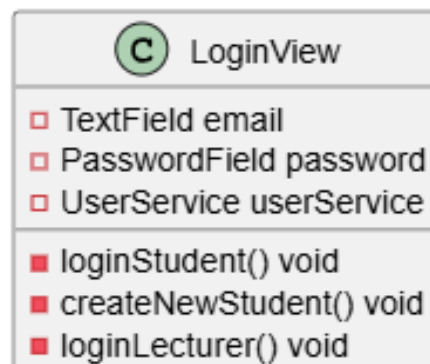
לבין השיבוצים הקיימים במערכת הכללית, ומציגה עבורו באופן בלעדי את השעות, הימים ושמות המרצים של הקורסים שלו על גבי לוח השנה, לצד סיכום נקודות הזכות המצטברות שלו בסרגל הימני.

:LoginLecturerView



מחלקת LoginLecturerView מספקת את שער הכניסה המאובטח עבור סגל המרצים ומנהלי המערכת בפרויקט. המסך מחולק ויזואלית בצורה אופקית לשני אזורים, כאשר הצד השמאלי הרחב מציג תמונת אווירה מקצועית והצד הימני מכיל את טופס הזנת הנתונים המעוצב באפקט זכוכית (Glassmorphism). רכיב המפתח במסך הוא תיבת הסימון adminCheckbox, המאפשרת למשתמש להצהיר על סוג ההתחברות המבוקש. בעת לחיצה על כפתור ההתחברות, מופעלת המתודה moveToHomePage אשר בודקת את מצב תיבת הסימון ומנתבת את הליך האימות בהתאם: במידה והתיבה מסומנת, היא פונה ל-AdminService לאימות מנהל, ובמידה ולא, היא פונה ל-LecturerService לאימות מרצה, תוך שמירת הישות המתאימה בסשן ומעבר למסך הבית התואם.

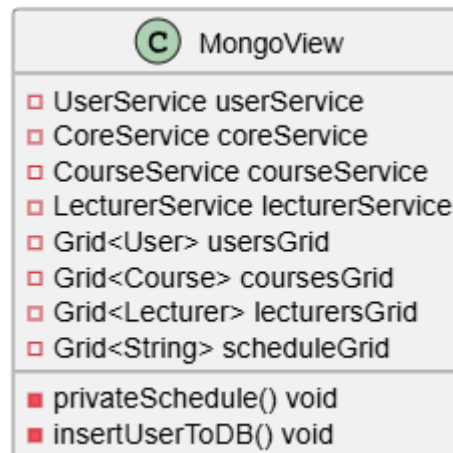
:LoginView



מחלקת LoginView משמשת כטופס כניסה וזיהוי. הטופס כולל שדות להזנת כתובת דואר אלקטרוני וסיסמה, לצד מערכת כפתורים מעוצבת בעלת מעברי צבעים (Gradients) המנוהלת על ידי שלוש פעולות ניתוב מרכזיות. הלחיצה על כפתור הרישום מפעילה את createNewStudent ומעבירה את הסטודנט למסך הרישום, וכפתור הסגל מפעיל את loginLecturer ומפנה למסך הלוגין של המרצים והמנהלים.

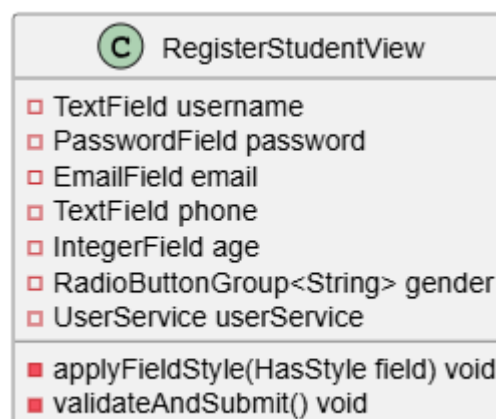
הפעולה הלוגית המרכזית היא loginStudent, אשר מבצעת בדיקת תקינות ראשונית לפורמט המייל, ומעבירה את הנתונים לאימות מול UserService, תוך שמירת אובייקט המשתמש והמייל שלו בסשן לצורך זיהוי עתידי ומעבר למערכת השעות האישית.

:MongoView



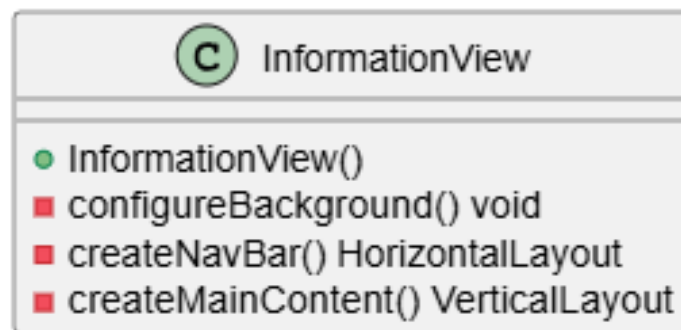
מחלקת MongoView היא מחלקת ממשק ייעודית שפותחה כלוח בקרה (Developer Dashboard) המאפשרת למפתח המערכת לצפות ישירות ובזמן אמת בכל האוספים והטבלאות השמורות במסד הנתונים של MongoDB. המסך מרכז ארבע טבלאות נתונים נפרדות (usersGrid, coursesGrid, lecturersGrid, ו-scheduleGrid) המציגות את נתוני הסטודנטים, הקורסים, המרצים והמערכת המשובצת. המחלקה כוללת כפתור הפעלה ישיר המקושר למתודה firstSetUp של MongoService, המאפשר לאתחל לחלוטין את בסיס הנתונים ולהזריק אליו את כל נתוני הדמה הראשוניים בלחיצה אחת, וכן את הפעולה privateSchedule המריצה את גרסת הבדיקה הטקסטואלית הראשונה של אלגוריתם השיבוץ ומציגה את פלט המחרוזות ישירות בגריד התואם.

:RegisterStudentView



מחלקת RegisterStudentView מנהלת את מסך ההרשמה של סטודנטים חדשים במערכת. המסך מציג טופס רישום מקיף המעוצב בתוך בועת זכויות מרכזית על גבי רקע קמפוס מלא, כאשר כל שדה קלט עובר דרך פונקציית העזר applyFieldStyle המקנה לו מראה חצי שקוף וצל עדין אחיד. הפעולה המרכזית והמשמעותית ביותר במסך היא validateAndSubmit, המופעלת בעת לחיצה על כפתור ההתקדמות הירוק. פעולה זו מבצעת סדרת בדיקות ולדיציה מתקדמות בזמן אמת: היא מוודאת ששם שדה אינו ריק, בודקת פורמט אימייל חוקי באמצעות תווים מוגדרים, מוודאת ששדה הטלפון מכיל אך ורק מספרים ומוגבל לעשר ספרות, ומאמתת שהגיל שהוזן הנו 18 ומעלה. במידה והבדיקות עברו בהצלחה, היא מייצרת ישות משתמש חדשה, פונה ל-UserService לרישום במסד הנתונים, ושומרת את המייל והשם בסשן לצורך העברה אוטומטית לשלב בחירת הקורסים.

:WelcomeView



מחלקת WelcomeView משמשת כתצוגת עמוד הבית (Landing Page) הראשית של המערכת, ומוגדרת להופיע בנתיב הראשי של האפליקציה. מחלקה זו יורשת מ-VerticalLayout של תשתית Vaadin ואחראית על הרכבת ממשק המשתמש הראשוני והוויזואלי שפוגש המשתמש טרם ההתחברות למערכת.

המחלקה מתמקדת אך ורק בלוגיקת תצוגה (UI) ואינה שומרת נתונים (Stateless). כדי לשמור על קוד מודולרי וקריא, תהליך בניית העמוד מחולק למספר מתודות עזר פרטיות: המתודה configureBackground אחראית על עיצוב הרקע הגרפי ויצירת אפקט טשטוש (Glassmorphism), המתודה createNavBar מרכיבה את סרגל הניווט העליון הכולל את לוגו המערכת ואת כפתורי הניתוב למסכי ההתחברות וההרשמה, והמתודה createMainContent בונה את התוכן המרכזי הכולל את כותרת המערכת, תיאורה ופרטי ההגשה האקדמיים. חלוקה זו מאפשרת שליטה נוחה וגמישה בכל אחד מרכיבי התצוגה בנפרד.

:AppLayoutWelcome

C AppLayoutWelcome
□ HorizontalLayout navBar □ HorizontalLayout logoLayout □ Icon capIcon □ Span logoText □ HorizontalLayout buttons □ Button loginBtn □ Button registerBtn
● AppLayoutWelcome()

AppLayoutWelcome מהווה את תבנית העיצוב והשלד המרכזי (Layout) עבור מסך הבית וקבלת הפנים של המערכת. המחלקה יורשת מ-AppLayout של Vaadin ומגדירה סרגל ניווט עליון מעוצב המקבל את פני המשתמשים טרם כניסתם למערכת. הבנאי של המחלקה מעצב את הסרגל, המוגדר טכנית בכיווניות מימין לשמאל (RTL), ומחלק אותו לשני מוקדים עיקריים: בצד ימין מוצג הלוגו הכולל כובע אקדמי חום ואת שם המערכת "Academic System", ובצד שמאל ממוקמים כפתורי ניווט מהירים למעבר למסך ההתחברות ולמסך ההרשמה למערכת. עיצוב הסרגל כולל גם אפקטים של רקע מעט שקוף וטשטוש (backdrop-filter) המעניקים למסך הנחיתה מראה מודרני.

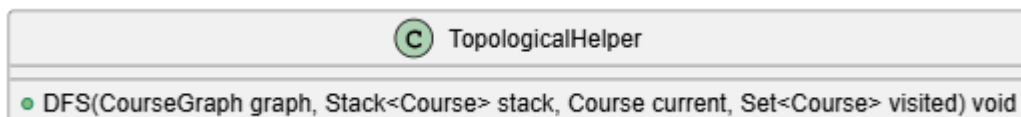
נעבור למחלקות העזר::SchedulerHelper

C SchedulerHelper
□ Map<Course, TimeSlot> assignments □ List<Course> topologicCourses
● setTopologicCourses(List<Course> topologicCourses) void ● canAssign(Course course, TimeSlot timeSlot) boolean ● assign(Course course, TimeSlot timeSlot) void ● unassign(Course course) void ● getPossibleTimeSlots(Course course) List<TimeSlot> ● isComplete() boolean

מחלקת SchedulerHelper משמשת כמחלקת עזר משמעותית המנהלת את המצב (State) של אלגוריתם השיבוץ האוטומטי המבוסס על נסיגה לאחור (Backtracking). המחלקה מחזיקה את רשימת הקורסים הממוינת טופולוגית במערך topologicCourses, ומנהלת את השיבוצים בפועל בתוך מילון הנתונים assignments הממפה כל קורס למשבצת הזמן שנקבעה לו.

התכונה החשובה ביותר שלה היא הפעולה `canAssign`, אשר מוודאת עמידה בכלל האילוצים המערכתיים בטרם אישור שיבוץ: היא בודקת האם המרצה פנוי בשעה המבוקשת, האם אין התנגשות עם קורסים אחרים שכבר שובצו באותו הזמן, ומוודאת שהקורס טרם שובץ. בנוסף, המחלקה מספקת פעולות ליבה לאלגוריתם כמו `assign` לקיבוע שיבוץ, `unassign` לביטולו בעת חזרה לאחור (Backtrack), והפעולה `getPossibleTimeSlots` אשר מייצרת בזמן אמת את כל חלונות הזמן האפשריים לקורס נתון על פני שבוע הלימודים, בהתחשב באורך הקורס.

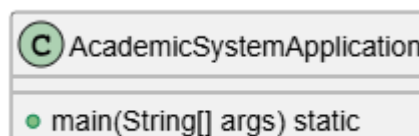
:TopologicalHelper



מחלקת `TopologicalHelper` היא מחלקת עזר אלגוריתמית ייעודית, אשר נועדה להפריד ולנהל את לוגיקת סריקת העומק (DFS) על גבי גרף הקורסים. המחלקה מכילה פעולה בודדת אך קריטית בשם `DFS`, אשר מקבלת את גרף הקורסים, מחסנית (`Stack`), קודקוד נוכחי וקבוצת קודקודים שכבר בוקרו (`visited`). הפעולה מבצעת קריאות רקורסיביות כדי לסרוק לעומק את כל הקורסים העוקבים (השכנים) של הקורס הנוכחי, ורק לאחר שסיימה לעבור על כל תלויותיו, היא דוחפת את הקורס הנוכחי אל תוך המחסנית. מנגנון מתמטי זה מבטיח שהקורסים יישלפו מהמחסנית בסדר למידה חוקי העונה על כל דרישות הקדם, המהווה את שלב העיבוד המקדים וההכרחי להצלחת אלגוריתם השיבוץ.

התוכנית הראשית:

:AcademicSystemApplication



מחלקת `AcademicSystemApplication` מהווה את נקודת הכניסה (Entry Point) הראשית של הפרויקט כולו. מחלקה זו מעוטרת באנוטציה `@SpringBootApplication`, אשר מסמנת לשלד התוכנה של `Spring Boot` להתחיל בסריקת כלל המחלקות בפרויקט, בניית מאגר התלויות (`Dependency Injection`) והפעלת השרת המובנה. המחלקה מכילה את הפעולה הסטטית `main`, אשר מפעילה את מתודת `SpringApplication.run` ובכך מאתחלת בפועל את המערכת, יוצרת חיבור למסד הנתונים של `MongoDB`, ומתחילה להאזין לבקשות הלקוח בממשק המשתמש.

12.4 מבני נתונים בהם נעשה שימוש

מילון (Map) הממומש באמצעות `HashMap` או `LinkedHashMap`. מבנה נתונים זה מופיע במספר מחלקות מרכזיות בפרויקט. במחלקת המודל `CourseGraph`, המילון `adjList` משמש לייצוג רשימת השכנויות של גרף הקורסים, שבו המפתח הוא אובייקט מסוג `Course` והערך הוא רשימת קורסים העוקבים לו, לצורך ניהול קשרי הקדימות בין הקורסים במערכת. במחלקת המודל `ScheduleDocument`, המילון `courseToSlot` משמש למיפוי בין מזהה הקורס כתו מחרוזת לבין משבצת הזמן שנקבעה לו, לצורך שמירת מערכת השעות הגלובלית במסד הנתונים. במחלקת העזר `SchedulerHelper`, המילון `assignments` משמש למעקב בזמן אמת אחר קורסים ששובצו למשבצות זמן במהלך ריצת אלגוריתם ה-`Backtracking`. במחלקות השירות `CoreService` ו-`ScheduleService`, המילון משמש ככלי קיבול להחזרת תוצאות השיבוץ הסופיות של האלגוריתם והכנתן לייצוג במערכת השעות. הצורך המרכזי במבנה זה בכל המחלקות הללו הוא לאפשר גישה מהירה ויעילה בזמן קבוע של $O(1)$ לשליפה של נתונים על בסיס מפתח ייחודי, כמו התאמה מדויקת בין קורס למשבצת הזמן שלו.

רשימה דינמית (List) הממומש באמצעות `ArrayList`. מבנה נתונים זה משמש במספר רב של מחלקות לניהול רשימות משתנות. במחלקת המודל `Course`, המערך `prerequisites` שומר את מזהי קורסי הקדם הנדרשים עבור הקורס הנוכחי. במחלקת המודל `User`, המערך `courseIds` שומר את רשימת מזהי הקורסים אליהם נרשם הסטודנט במערכת. במחלקת השירות `CoreService`, המערך `orderedCourses` משמש לאחסון רשימת הקורסים לאחר שעברו מיון טופולוגי, ומערכים נוספים במחלקה זו משמשים להחזרת פלט מחרוזות של מערכת השעות המתוכננת.

במחלקות הממשק והשירות, כדוגמת `AdminAddDataView`, `UserService` ו-`HomeLecturerService`, מערכים אלו משמשים לאיסוף זמני של קורסים שנבחרו או נשלפו ממסד הנתונים לצורך הצגתם בגריד המערכת. הצורך במבנה זה נובע מהצורך בניהול רשימות אלמנטים דינמיות שגודלן אינו ידוע מראש, תוך שמירה על סדר ההכנסה ואפשרות גישה ישירה לפי אינדקס.

קבוצה (Set) הממומשת באמצעות `HashSet`. מבנה זה מופיע במחלקות המודל, השירות והממשק כדי להבטיח ייחודיות של נתונים ומניעת כפילויות. במחלקת המודל `Lecturer`, הקבוצה `availableSlots` משמשת לאחסון משבצות הזמן שבהן המרצה הגדיר שהוא פנוי להוראה. במחלקת השירות `CoreService`, הקבוצה `visited` משמשת במהלך אלגוריתם סריקת העומק (DFS) כדי לעקוב אחר קורסים שכבר בוקרו, ובכך למנוע מעגליות ולולאות אינסופיות בגרף הקורסים, בעוד הקבוצה `userCourseIds` משמשת לבדיקת הכלה מהירה של קורסי הסטודנט. במחלקת הממשק `AdminAddDataView`, קבוצה דינמית שומרת את משבצות הזמן שנבחרו עבור המרצה החדש טרם שמירתו.

במחלקת השירותים `MongoService`, קבוצות אלו משמשות לאתחול הנתונים הראשוניים של המרצים במערכת. השימוש בקבוצה במחלקות אלו חיוני בשל תכונתה המונעת כפילויות באופן מובנה ומאפשרת בדיקת שייכות של איבר בזמן יעיל של $O(1)$.

מחסנית (Stack) מבנה נתונים זה מוגדר ומנוהל בתוך מחלקת השירותים `CoreService` ומועבר כפרמטר למחלקת העזר `TopologicalHelper` לצורך ביצוע המיון הטופולוגי של קורסי הלימוד. במהלך סריקת העומק (DFS) המבוצעת על גרף הקורסים, כל קורס שסורק את כל שכניו נדחף אל תוך המחסנית `stack`. הצורך במחסנית נובע ישירות מאופן פעולתה בגישת "אחרון נכנס – ראשון יוצא" (LIFO), המאפשרת להפוך את סדר הקודקודים המסתיימים בסריקה, ובכך לקבל את סדר הקורסים הנכון והתקין כפי שנדרש לבניית מסלול הלימודים האקדמי המשמש את הסטודנט במערכת.

12.5 קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים

- אלגוריתם בניית גרף הקורסים המכונן:

```
public CourseGraph buildGraphForUser(String username) {
    User user = mongoService.getUser(username);
    if (user == null) {
        throw new RuntimeException("User not found: " + username);
    }
    List<Course> userCourses = getCoursesForUser(user);
    Set<String> userCourseIds = new HashSet<>(user.getCourseIds());
    CourseGraph graph = new CourseGraph();
    // הוספת קורסים לגרף
    for (Course course : userCourses) {
        graph.addCourse(course);
    }
    // הוספת קשרי prerequisite
    for (Course course : userCourses) {
        if (course.getPrerequisites() == null)
            continue;
        for (String prereqId : course.getPrerequisites()) {
            if (userCourseIds.contains(prereqId)) {
                Course prereqCourse = findCourseById(userCourses, prereqId);
                if (prereqCourse != null) {
                    graph.addPrerequisite(prereqCourse, course);
                }
            }
        }
    }
    return graph;
}
```

תיעוד טענת כניסה: הפעולה המרכזית מקבלת גרף מאותחל מסוג CourseGraph. פעולת העזר הרקורסיבית מקבלת את הגרף, מבנה נתונים מחסנית (Stack<Course>), קורס נוכחי (Course), וקבוצת קורסים שבוקרו (Set<Course>).

תיעוד טענת יציאה: הפעולה המרכזית מחזירה List<Course> - רשימה דינמית של הקורסים המסודרים בסדר הלמידה הנכון.

מה מבצע: האלגוריתם מסדר את קודקודי הגרף כך שכל קורס קדם יופיע תמיד לפני הקורס שתלוי בו, מה שמבטיח מסלול לימודים תקין וללא סתירות פדגוגיות.

אופן פעולתו בצורה מפורטת: האלגוריתם מאתחל קבוצת מעקב (visited) כדי להימנע ממעגליות ולולאות אינסופיות בגרף, יחד עם מחסנית חדשה. הוא עובר על כל קודקוד בגרף ברצף, ואם הקודקוד טרם בוקר, מופעל עליו אלגוריתם סריקת העומק (DFS) באופן רקורסיבי. אלגוריתם ה-DFS מסמן את הקורס הנוכחי כ"בוקר", ויורד לעומק אל כל השכנים שלו (הקורסים העוקבים שדורשים אותו) תוך קריאה לעצמו. הכלל האלגוריתמי החשוב מופעל רק בסוף הסריקה: אך ורק לאחר שקודקוד סיים לסרוק לחלוטין את כל שכניו, הוא "נדחף" (Push) אל תוך המחסנית. עם סיום כלל הסריקות, הפונקציה הראשית שולפת (Pop) את כל האיברים מהמחסנית אל תוך הרשימה. בזכות עקרון LIFO של המחסנית (אחרון נכנס ראשון יוצא), הקורסים יוצאים בסדר הטופולוגי ההפוך, שהוא בדיוק הסדר הליניארי והתקין. סיבוכיות זמן ריצה: $O(V + E)$ מכיוון שסריקת ה-DFS מבקרת בכל קודקוד (V) פעם אחת בלבד לאורך כל הריצה, ועוברת על כל הקשתות (E) היוצאות ממנו פעם אחת בדיוק.

• אלגוריתם סריקת עומק ומיון טופולוגי (DFS & Topological Sort)

הפעולה Topological Sort מתוך המחלקה CoreService

```
public static List<Course> topologicalSort(CourseGraph graph) {
    Set<Course> visited = new HashSet<>();
    Stack<Course> stack = new Stack<>();
    TopologicalHelper helper = new TopologicalHelper();

    for (Course course : graph.getGraph().keySet()) {
        if (!visited.contains(course))
            helper.DFS(graph, stack, course, visited);
    }

    List<Course> topologicalList = new ArrayList<Course>();
    Course outStack;
    while (!stack.empty()) {
        outStack = stack.pop();
        topologicalList.add(outStack);
    }
    return topologicalList;
}
```

הפעולה DFS מתוך המחלקה TopologicalHelper:

```

public void DFS(CourseGraph graph, Stack<Course> stack, Course current,
Set<Course> visited){
    visited.add(current); // mark the current course
    for (Course neighbor : graph.getGraph().get(current)) {
        if(!visited.contains(neighbor)) // check if we checked this course
            DFS(graph,stack,neighbor,visited); // recursive
    }
    stack.push(current);
}

```

תיעוד טענת כניסה: הפעולה המרכזית מקבלת גרף מאותחל מסוג CourseGraph. פעולת העזר הרקורסיבית מקבלת את הגרף, מבנה נתונים מחסנית (Stack<Course>), קורס נוכחי (Course), וקבוצת קורסים שבוקרו (Set<Course>).

תיעוד טענת יציאה: הפעולה המרכזית מחזירה List<Course> - רשימה דינמית של הקורסים המסודרים בסדר הלמידה הנכון.

מה מבצע: האלגוריתם מסדר את קודקודי הגרף כך שכל קורס קדם יופיע תמיד לפני הקורס שתלוי בו, מה שמבטיח מסלול לימודים תקין וללא סתירות פדגוגיות.

אופן פעולתו בצורה מפורטת: האלגוריתם מאתחל קבוצת מעקב (visited) כדי להימנע ממעגליות ולולאות אינסופיות בגרף, יחד עם מחסנית חדשה. הוא עובר על כל קודקוד בגרף ברצף, ואם הקודקוד טרם בוקר, מופעל עליו אלגוריתם סריקת העומק (DFS) באופן רקורסיבי. אלגוריתם ה-DFS מסמן את הקורס הנוכחי כ"בוקר", ויורד לעומק אל כל השכנים שלו (הקורסים העוקבים שדורשים אותו) תוך קריאה לעצמו. הכלל האלגוריתמי החשוב מופעל רק בסוף הסריקה: אך ורק לאחר שקודקוד סיים לסרוק לחלוטין את כל שכניו, הוא "נדחף" (Push) אל תוך המחסנית. עם סיום כלל הסריקות, הפונקציה הראשית שולפת (Pop) את כל האיברים מהמחסנית אל תוך הרשימה. בזכות עקרון LIFO של המחסנית (אחרון נכנס ראשון יוצא), הקורסים יוצאים בסדר הטופולוגי ההפוך, שהוא בדיוק הסדר הליניארי והתקין.

סיבוכיות זמן ריצה: $O(V + E)$ מכיוון שסריקת ה-DFS מבקרת בכל קודקוד (V) פעם אחת בלבד לאורך כל הריצה, ועוברת על כל הקשתות (E) היוצאות ממנו פעם אחת בדיוק.

• אלגוריתם השיבוץ - נסיגה לאחור וגזום (Backtracking and Pruning):

```
public static boolean SchedulingBacktrackingAndPruning(SchedulerHelper scheduler, int
index) {
    if (scheduler.isComplete())
        return true;
    Course currentCourse = scheduler.getNextCourse(index);
    for (TimeSlot slot : scheduler.getPossibleTimeSlots(currentCourse)) {
        if (!scheduler.canAssign(currentCourse, slot)) // Pruning
            continue;

        scheduler.assign(currentCourse, slot);
        if (SchedulingBacktrackingAndPruning(scheduler, index + 1))
            return true;

        scheduler.unassign(currentCourse);
    }
    return false;
}
```

תיעוד טענת כניסה: הפעולה מקבלת אובייקט SchedulerHelper המכיל את מצב השיבוצים המעודכן באותו רגע, ומשתנה שלם (int) בשם index המייצג את מיקום הקורס מתוך הרשימה הטופולוגית שאותו מנסים לשבץ כעת.

תיעוד טענת יציאה: מחזירה ערך בוליאני true (boolean). אם נמצא שיבוץ חוקי וסופי לכלל הקורסים, ו-false אם ניסיון השיבוץ הנוכחי מוביל למבוי סתום ואין פתרון במערכת.

מה מבצע: זהו מנוע הליבה האלגוריתמי של הפרויקט. הוא משבץ באופן אוטומטי את כל הקורסים למשבצות זמן על גבי השבוע באופן אופטימלי, תוך עמידה מלאה באילוצי זמינות המרצים וללא חפיפת כיתות, בעזרת טכניקת של ניסוי וטעייה.

אופן פעולתו בצורה מפורטת: מדובר באלגוריתם רקורסיבי חכם. תנאי העצירה (Base Case) מתרחש מיד בתחילת הפעולה: אם כמות הקורסים ששובצו שווה לכמות הקורסים המלאה (מתודת isComplete), משמע מצאנו פתרון מלא והפעולה מחזירה true. במידה ולא, האלגוריתם שולף את הקורס הנוכחי לפי האינדקס, ומייצר דינמית רשימה של כל משבצות הזמן האפשריות לשבוע הלימודים. עבור כל משבצת זמן במערך, מבוצע שלב "גזום הענפים" (Pruning), כלומר נבדקת חוקיות המשבצת דרך הפעולה canAssign. אם המשבצת מתנגשת עם אילוצי כלשהו, האלגוריתם "גוזם" את המשך החישוב עליה ומדלג מיד להבאה באמצעות הפקודה continue. אם היא נמצאה חוקית, הקורס משובץ זמנית למשבצת זו (assign), ומתבצעת קריאה רקורסיבית מתוך ה-lf לניסיון שיבוץ הקורס הבא בתור (index + 1).

היתרון הגדול של ה-Backtracking מתרחש כאן: אם הקריאה הרקורסיבית הבאה החזירה false (משמע נוצר מבוי סתום בהמשך העץ), האלגוריתם מבצע "נסיגה לאחור" ומפעיל את הפקודה unassign המבטלת את השיבוץ האחרון שבוצע, ועובר לבחון את משבצת הזמן הבאה עבור הקורס. סיבוכיות זמן ריצה: במקרה הגרוע ביותר התיאורטי (Worst Case), הסיבוכיות הינה מעריכית $O(T^N)$ כאשר T הוא מספר חלונות הזמן האפשריים על פני השבוע ו-N הוא מספר הקורסים. אולם, הודות למנגנון החכם של "גיזום הענפים" המיושם בקוד, המערכת פוסלת תתי-עצים שלמים בשלב מוקדם מאוד והאלגוריתם רץ בפועל בזמן יעיל וקצר.

- פונקציית האימות וגיזום האילוצים (canAssign):

```
// check if course can be assigned to this timeSlot
public boolean canAssign(Course course, TimeSlot timeSlot) {
    Lecturer lecturer = course.getLecturer();

    // 1. בדיקת זמינות מרצה
    boolean available = false;
    for (TimeSlot slot : lecturer.getAvailableSlots()) {
        if (slot.overlaps(timeSlot)) {
            available = true;
            break;
        }
    }
    if (!available)
        return false;

    // 2. בדיקת התנגשויות מרצה אחרים
    for (TimeSlot assigned : assignments.values()) {
        if (assigned.overlaps(timeSlot)) {
            return false;
        }
    }

    // 3. לא משבצים פעמיים קורס
    if (assignments.containsKey(course)) {
        return false;
    }

    return true;
}
```


תיעוד טענת כניסה: מקבלת אובייקט מסוג Course (הקורס שאותו מנסים לשבץ כעת) ואובייקט TimeSlot (חלון הזמן הנבדק לשיבוץ)

תיעוד טענת יציאה: מחזירה ערך מסוג boolean. התשובה תהיה true אם ניתן לשבץ את הקורס בחלון הזמן הזה ללא הפרת אילוצי המערכת, ו-false אם קיים קונפליקט מכל סוג שהוא.

מה מבצע: הפעולה מהווה את "מנוע החוקים" החכם שמדריך את אלגוריתם ה-Backtracking. היא אחראית באופן בלעדי לגזום ולפסול מסלולי חישוב שגויים, ובכך למנוע התנגשויות של כפילויות, אילוצי מרצים וחפיפות במידה ויש.

אופן פעולתו בצורה מפורטת: הפעולה מבצעת סדרה אגרסיבית של 3 בדיקות לוגיות קפדניות:

- בדיקת זמינות המרצה: הפעולה שולפת את המרצה המשוך לקורס ועוברת איטרטיבית על כל חלונות הזמן שהגדיר כפנויים (availableSlots). היא מבצעת הפעלה של מתודת החפיפה (overlaps) אל מול חלון הזמן המבוקש. במידה ואין אף חלון זמן פנוי שחופף למשבצת המבוקשת, המרצה אינו זמין והפעולה מחזירה false.
- מניעת התנגשויות תזמון קורסים: המערכת רצה על פני כלל המשבצות (TimeSlots) של הקורסים האחרים שכבר שובצו בהצלחה במערכת עד לאותו רגע (מתוך מילון ה-assignments). במידה והיא מוצאת שקורס אחר כבר משובץ במשבצת זמן שחופפת לזמן המבוקש, משמע תיווצר התנגשות שעות במערכת השעות הסופית, ולכן היא תעצור ותחזיר false.
- מניעת כפילות קורס: בשלב האחרון, מתבצעת שאילתת בדיקת הכלה מול מילון השיבוצים הנוכחי כדי לוודא שמשתמשים לא ינסו לשבץ בטעות פעמיים את אותו הקורס. במידה וכל הבדיקות עברו ללא הכשלה, מוחזר true לגיטימי.

סיבוכיות זמן ריצה: $O(S + C)$ כאשר S הוא מספר חלונות הזמן הזמינים של המרצה הנבדק ו-C הוא מספר הקורסים הכללי שכבר שובצו בהצלחה באותה ריצה. מכיוון שהגבלת המשתנים קטנה מאוד וקבועה ביחס לשבוע קלנדרי רגיל (ללא אלפי שיבוצים בו זמנית).

13. תיאור מסד הנתונים

בפרויקט זה נעשה שימוש במסד נתונים מסוג NoSQL, ספציפית MongoDB, המשתלב עם שרת ה-Spring Boot של המערכת (כפי שניתן לראות ממיפוי המחלקות). בחירה זו מאפשרת גמישות בשמירת הנתונים, עבודה חלקה עם אובייקטים מקוננים (Embedded Documents) כגון זמינות מרצים, וייצוג טבעי של מבני הנתונים הנדרשים לאלגוריתם שיבוץ המערכת.

מסד הנתונים מורכב מהאוספים (Collections) המרכזיים הבאים, המייצגים את הישויות במערכת:

ישות משתמש / סטודנט (User)

```

{
  _id: 'user@gmail.com',
  username: 'firstUser',
  password: '123456',
  phone: '0548765234',
  age: NumberInt('22'),
  courseIds: [
    'J64Cw',
    'C1LL10',
    'C1CS1',
    'C1ALG4',
    'C1MATH1',
    'C1DS5',
    'C1C++2'
  ],
  _class: 'kfiry.academic_system.datamodels.User'
}

```

- אוסף המנהל את נתוני הסטודנטים במערכת.
- מזהה ראשי (id): מוגדר ככתובת הדוא"ל של המשתמש. בחירה עיצובית זו נועדה להבטיח זיהוי חד-ערכי (Uniqueness) מוחלט של כל סטודנט בעת ההתחברות למערכת, ומניעת כפילויות שעלולות להיווצר משימוש בשמות רגילים.
- שדות נוספים: שדה username (שם תצוגה), password (סיסמה), phone (מספר טלפון), ו-age (גיל הסטודנט, הנשמר כערך מספרי שלם).
- קשרים: השדה courseIds מכיל מערך של מחרוזות (מזהי קורסים) המייצג את רשימת הקורסים אליהם הסטודנט רשום או אותם הוא נדרש להשלים.

ישות קורס (Course)

```

{
  _id: ObjectId('6a0610e9d2c854c3a665fd2b'),
  name: 'C++ Programming',
  courseID: 'C1C++2',
  duration: NumberInt('1'),
  lecturer: {
    name: 'Yotam Eli',
    ID: '462829473',
    password: 'yotam222',
    availableSlots: [
      {
        day: 'WEDNESDAY',
        startHour: NumberInt('11'),
        endHour: NumberInt('18')
      },
      {
        day: 'TUESDAY',
        startHour: NumberInt('8'),
        endHour: NumberInt('18')
      }
    ],
    mandatory: true,
    prerequisites: [
      'C1CS1'
    ],
    _class: 'kfiry.academic_system.datamodels.Course'
  }
}

```

- אוסף המגדיר את הקורסים השונים הנלמדים במוסד.
- מזהה ראשי courseID
- שדות נוספים name (שם הקורס במלואו, למשל "C++ Programming"), מזהה קורס פנימי courseID (למשל "C1C++2"), duration (משך הקורס עבור מנהל או נקודות זכות עבור סטודנט), ודגל בוליאני mandatory המציין האם הקורס הינו חובה.
- אובייקטים מקוננים וקשרים:
 - lecturer, אובייקט מקונן המכיל את פרטי המרצה המשויך לקורס (כולל שמו, תעודת הזהות שלו, ורשימת חלונות הזמן הפנויים שלו).
 - prerequisites מערך של מזהי קורסים המהווים דרישות קדם לקורס הנוכחי

ישות מרצה (Lecturer)

```

{
  _id: ObjectId('6a0610e8d2c854c3a665fd28'),
  name: 'David Gerber',
  ID: '232784029',
  password: '123456',
  availableSlots: [
    {
      day: 'SUNDAY',
      startHour: NumberInt('8'),
      endHour: NumberInt('18')
    },
    {
      day: 'MONDAY',
      startHour: NumberInt('8'),
      endHour: NumberInt('18')
    },
    {
      day: 'THURSDAY',
      startHour: NumberInt('8'),
      endHour: NumberInt('18')
    }
  ],
  _class: 'kfiry.academic_system.datamodels.Lecturer'
}

```

- אוסף המנהל את צוות ההוראה ואת אילוצי המערכת שלהם.
- מזהה ראשי id
- שדות נוספים: name (שם המרצה), ID (תעודת זהות, משמשת לעיתים קרובות גם לצורכי זיהוי/התחברות), ו-password.
- חלונות זמינות (availableSlots): מערך של אובייקטים המייצג את זמינות המרצה. כל אובייקט מכיל את היום בשבוע (day), שעת התחלה (startHour) ושעת סיום (endHour). נתון זה הינו קריטי לפעולתו של מנגנון שיבוץ המערכת האוטומטי.

ישות שיבוץ מערכת שעות (ScheduleDocument)

```

✓{
  _id: ObjectId('6a0b31e92099aeed00ff6be5'),
  ✓ courseToSlot: {
    C1WEB13: 'MONDAY 9:00-11:00',
    C1OS8: 'SUNDAY 8:00-10:00',
    C1MATH1: 'MONDAY 8:00-9:00',
    C1ALG4: 'WEDNESDAY 12:00-13:00',
    C1ML12: 'TUESDAY 13:00-15:00',
    C1LL10: 'TUESDAY 8:00-9:00',
    C1CN9: 'THURSDAY 15:00-17:00',
    C1ADB19: 'THURSDAY 8:00-11:00',
    C1DB6: 'MONDAY 15:00-17:00',
    C1CAP20: 'WEDNESDAY 16:00-18:00',
    J64Cw: 'TUESDAY 11:00-12:00',
    C1SE7: 'WEDNESDAY 13:00-16:00',
    C1AI11: 'MONDAY 12:00-15:00',
    C1CS1: 'MONDAY 11:00-12:00',
    C1MOB14: 'TUESDAY 9:00-11:00',
    C1SEC15: 'THURSDAY 13:00-15:00',
    C1CLOUD16: 'TUESDAY 15:00-18:00',
    C1DM17: 'THURSDAY 11:00-13:00',
    'C1C++2': 'TUESDAY 12:00-13:00',
    C1NET18: 'WEDNESDAY 9:00-12:00',
    C1DS5: 'MONDAY 17:00-18:00'
  },
  _class: 'kfiry.academic_system.datamodels.ScheduleDocument'
}

```

- אוסף השומר את התוצר הסופי של אלגוריתם שיבוץ הקורסים.
- מיפוי קורסים לחלונות זמן (courseToSlot): מבנה נתונים מסוג מילון (Object/Map) המקשר בין מזהה קורס ספציפי (Key) למחרוזת המייצגת את חלון הזמן שנקבע לו בלוח השעות (Value). לדוגמה: הקורס "C1WEB13" משובץ ל-MONDAY 9:00-11:00.

ישות מנהל מערכת (Admin)

```

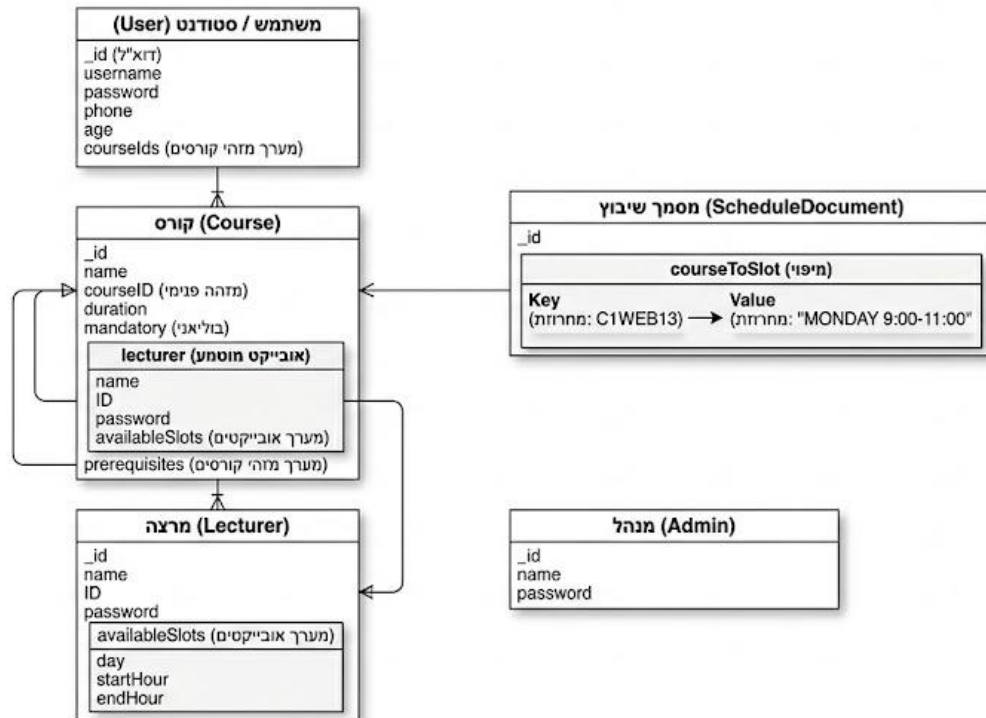
{
  _id: '216232470',
  password: '123',
  name: 'kfiry',
  _class: 'kfiry.academic_system.datamodels.Admin'
}

```

- אוסף המנהל את המשתמשים בעלי הרשאות הניהול.
- מזהה ראשי id מוגדר על בסיס מספר זיהוי ייחודי או תעודת זהות (נשמר כמחרוזת).
- שדות נוספים: name (שם המנהל) ו-password (סימת ההתחברות).

להלן תרשים המתאר את קשרי הגומלין שבמערכת:

תרשים מודל נתונים (ERD) - Academic System



התרשים לעיל מציג את מודל הנתונים של המערכת ואת הקשרים הלוגיים בין הישויות השונות במסד הנתונים. מכיוון שמדובר במסד נתונים שאינו רלציוני, התרשים מדגים כיצד ממומשים קשרים באמצעות הפניות והטמעת מסמכים במקום טבלאות מסורתיות. המודל בנוי ממספר ישויות מרכזיות המנהלות את המידע האקדמי והניהולי של המערכת.

ישות הסטודנט שומרת את פרטי המשתמשים במערכת. המזהה הראשי שלה הוא כתובת הדואר האלקטרוני, על מנת להבטיח זיהוי ייחודי לכל משתמש. בנוסף, ישות זו מכילה מערך השומר את מזהי הקורסים אליהם הסטודנט קשור או רשום. ישות הקורס היא המרכזית במערכת ומייצגת את מקצועות הלימוד. היא כוללת נתונים כמו שם הקורס, מזהה פנימי, משך הקורס וציון האם מדובר בקורס חובה. בתוך מסמך הקורס מוטמע אובייקט שלם המכיל את פרטי המרצה המעביר את הקורס, גישה המאפשרת שליפת נתונים מהירה ויעילה. כמו כן, קיים מערך דרישות קדם המכיל את מזהי הקורסים שיש להשלים לפני הרישום.

ישות המרצה מנהלת את צוות ההוראה ואת אילוצי הזמן שלהם, באמצעות מערך המגדיר את הימים והשעות הפנויות של כל מרצה. נתונים אלו הם קריטיים לפעולת התקינה של אלגוריתם שיבוץ המערכת. התוצר הסופי של אלגוריתם זה נשמר בישות מסמך השיבוץ, המכילה מבנה נתונים מסוג מילון. במילון זה, כל מפתח מייצג מזהה של קורס, והערך שלו הוא חלון הזמן המדויק שנקבע לאותו

קורס במערכת השעות. לבסוף, ישות המנהל פועלת כישות עצמאית המיועדת לניהול המערכת מאחורי הקלעים, והיא כוללת נתוני זיהוי בסיסיים כמו תעודת זהות, שם וסיסמה.

הקשרים הלוגיים בין הישויות מיוצגים בתרשים באמצעות חצים. הקשר בין סטודנט לקורס, שהוא קשר של רבים לרבים, מיוצג באמצעות חץ היוצא ממערך הקורסים של הסטודנט ומצביע על ישות הקורס. דרישות הקדם מתוארות באמצעות חץ היוצא ממערך קורסי הקדם וחוזר אל ישות הקורס עצמה, כהפניה עצמית המסמלת תלות בין קורסים. קשר נוסף מתקיים בין הקורס למרצה, ומוצג כחץ היוצא מהמרצה המוטמע בתוך הקורס לעבר ישות המרצה המקורית. לסיום, הקשר בין מסמך השיבוץ לקורס מומחש על ידי חץ היוצא ממפתח הקורס במסמך השיבוץ ומצביע על הקורס הספציפי, ומסביר כיצד מערכת השעות מקשרת בין חלונות הזמן לקורסים המוגדרים במערכת.

14. מדריך למשתמש

מערכת שיבוץ הקורסים האקדמית תוכננה לספק חוויית שימוש אינטואיטיבית וידידותית, תוך הפרדה ברורה בין סוגי המשתמשים השונים: סטודנטים, אנשי סגל (מרצים) ומנהלי מערכת. להלן פירוט תהליכי העבודה המרכזיים עבור כל אחד מהתפקידים.

כניסה ראשונית והרשמה למערכת:

- עבור סטודנטים: בכניסה למערכת, יוצג מסך ההתחברות הראשי (Login). סטודנט רשום יזין את כתובת האימייל והסיסמה שלו ויקליק על Sign In.
- סטודנט חדש יקליק על הקישור Don't have an account? Sign up וימלא את טופס ההרשמה (שם, אימייל, סיסמה, טלפון, גיל ומגדר).
- עבור סגל ומנהלים: יש ללחוץ על הכפתור הירוק Login Lecturer/Admin במסך הראשי.
- במסך הייעודי שייפתח, יש להזין תעודת זהות (ID) וסיסמה. מנהל מערכת נדרש גם לסמן את תיבת הסימון "התחבר כמנהל מערכת" טרם הלחיצה על Login.

מדריך למשתמש - סטודנט:

לאחר הרשמה או התחברות מוצלחת, תהליך העבודה של הסטודנט כולל שני שלבים עיקריים:

שלב א': הרכבת סל הקורסים: מיד לאחר ההרשמה, המערכת תציג את מסך "בחירת קורסים". הקורסים מחולקים לעמודות לפי סמסטרים (א', ב', קיץ). יש לעבור על הרשימה ולסמן בתיבת הסימון (Checkbox) את הקורסים הרצויים. ניתן לעקוב אחר הקורסים שנבחרו בזמן אמת בפאנל "הסיכום שלי" בצד שמאל. בסיום הבחירה, יש ללחוץ על הכפתור "סיים בחירה ועבור למסך הבית".

שלב ב': הפקת מערכת שעות (שיבוץ אישי): במסך הבית האישי, הסטודנט יראה את סיכום הקורסים שבחר ואת סך נקודות הזכות (נק"ז) שלו. להפקת לוח הזמנים, יש ללחוץ על הכפתור

"runAlgoritemButton" בפאנל הימני. המערכת תחשב את המסלול האופטימלי, ולאחר מספר שניות, טבלת היומן המרכזית תתמלא במשבצות צבעוניות המייצגות את שעות השיעורים במדויק.

מדריך למשתמש - איש סגל (מרצה)

מטרת הממשק למרצה היא הצגת נתונים ולוחות זמנים (תצוגה בלבד):

- כניסה לאזור האישי: לאחר התחברות ממסך הסגל, המרצה יועבר לדף הבית ("איזור אישי סגל").
- צפייה בנתונים: בפאנל הימני ניתן לראות את סיכום שעות ההוראה השבועיות ואת רשימת הקורסים ("הקורסים שאני מלמד") ששויכו למרצה על ידי ההנהלה.
- הצגת הלו"ז השבועי: יש ללחוץ על הכפתור "הצג מערכת שעות" בפאנל השמאלי. פעולה זו תמשוך את הנתונים מהשרת ותציג את פריסת השיעורים השבועית של המרצה על גבי טבלת היומן המרכזית.

מדריך למשתמש – מנהל מערכת (Admin):

מנהל המערכת שולט בנתוני התשתית ואחראי על הפעלת מנוע השיבוץ המרכזי. הניווט מתבצע דרך סרגל הניווט העליון.

- ניהול והזנת נתונים (מסך "הוספת הנתונים"): יש לנווט ללשונית "מסך הוספת הנתונים" בסרגל העליון. המסך מחולק לעמודות יעודיות (הוספת קורס, מרצה, סטודנט, מנהל). לדוגמא הוא יכול להוסיף:
 - הוספת קורס: חובה להזין את שם הקורס, מזהה (ID), משך הקורס, ולהגדיר קורסי קדם (קדימויות) מתוך הרשימה הנפתחת במידה וישנם.
 - הוספת מרצה: יש להזין פרטים אישיים, וחשוב מכך - להגדיר את זמני ההוראה (הימים והשעות בהם המרצה זמין) ולשייך לו קורסים מרשימת הקורסים הקיימת.

בסיום מילוי כל טופס, יש ללחוץ על כפתור השמירה בתחתית העמודה.

- צפייה ובקרה (מסך "הצגת הנתונים"): יש לנווט ללשונית "מסך הצגת הנתונים". יש לבחור מהרשימה הנפתחת העליונה את סוג הנתונים המבוקש (קורסים, מרצים, סטודנטים וכו'). הטבלה המרכזית תתעדכן ותציג את כלל הרשומות הקיימות במסד הנתונים לקריאה וביקורת. יצירת לוח זמנים מוסדי (מסך "מסך הבית"): במסך הבית של המנהל, מוצגים כלל הקורסים הפעילים במערכת לצד טבלת "לו"ז כללי" ריקה. לבניית מערכת שעות אופטימלית לכלל המוסד, יש ללחוץ על הכפתור האדום "חשב מערכת מחדש".

שימו לב: פעולה זו מפעילה את אלגוריתם השיבוץ המרכזי, הסורק את כלל האילוצים, התלויות וזמינות המרצים. בסיום החישוב, תאוכלס הטבלה הכללית בשיבוצים החדשים עבור כלל הקורסים והמרצים במערכת.

15. בדיקות והערכה

במהלך ולאחר פיתוח המערכת, בוצעה סדרת בדיקות מקיפה במטרה לוודא שהמערכת עומדת בדרישות שהוגדרו באפיון, מטפלת במקרי קיצון ולא קורסת תחת עומס. הבדיקות התחלקו לשלושה מישורים:

בדיקות אלגוריתם ולוגיקה (Unit Testing): לב המערכת הוא אלגוריתם השיבוץ. נבדק שהמיון הטופולוגי אכן מסדר קורסי מבוא לפני קורסים מתקדמים ללא שגיאות. כמו כן, בוצעו בדיקות מאמץ שהוכיחו כי בזכות החסם העליון שהוגדר (עד 20 קורסים בסך הכל למערכת) ובזכות מנגנון הגיזום (Pruning), האלגוריתם מוצא פתרון שלם וחוקי בתוך שניות בודדות ולא גורם לקריסת הזיכרון של השרת. בנוסף, הזנו בכוונה אילוצים סותרים (למשל, מרצה שנדרש להיות בשני קורסים באותה שעה בדיוק) כדי לוודא שהמערכת קוטעת את הפעולה ומחזירה הודעת שגיאה על חוסר היתכנות במקום לייצר לוח זמנים שגוי.

בדיקות אבטחת נתונים ומניעת כפילויות: נבדק מנגנון ההזדהות של המערכת. ניסינו לרשום שני משתמשים עם אותה כתובת אימייל או להשתמש בשם משתמש רגיל, והמערכת דחתה את הפעולה כנדרש, מה שמוכיח שמנגנון ההתחברות מבוסס האימייל הייחודי פועל באופן מלא ומונע כפילויות במסד הנתונים.

בדיקות ממשק והרשאות משתמש (UAT): נבדק שכל משתמש רואה אך ורק את המסכים המותרים לו. מנהל המערכת הצליח להגיע למסכי ההפקה ולהזין אילוצי מרצים, המרצים ראו רק את לוח ההוראה שלהם, והסטודנטים קיבלו למסך שלהם אך ורק את טבלת מערכת השעות הסופית שלהם לצפייה, ללא יכולת לשנות נתונים כלשהם.

16. מסקנות

פרויקט זה הוכיח כי ניתן לפתור את בעיית שיבוץ הקורסים המורכבת (שהיא בעיית NP-Hard קלאסית של סיפוק אילוצים) על ידי תחום גבולות גזרה חכמים ושילוב אלגוריתמי נכון. המסקנה המרכזית היא שאין צורך להשתמש באלגוריתמים אקראיים (כמו אלגוריתמים גנטיים) שאינם מבטיחים דיוק, כל עוד מגבילים את נפח הנתונים לרמה נשלטת (עד 20 קורסים).

מסקנה נוספת היא שמנגנון הנסיגה לאחור (Backtracking) יחד עם גיזום ענפים לא חוקיים (Pruning) מצליח להתכנס לפתרון חוקי ב-100% (אפס התנגשויות בין קורסים או בין זמני מרצים),

תוך עמידה מלאה בתנאי הקדם האקדמיים הודות למיון הטופולוגי המקדים. יתרה מזאת, הפיתוח הוכיח שהשימוש במסגרות עבודה מודרניות כמו Spring Boot לפיתוח השרת ו-Vaadin ליצירת ממשק המשתמש, מייעל דרמטית את תהליך הפיתוח ומאפשר יצירת אפליקציית Web יציבה, מהירה ונוחה לתחזוקה. המערכת שנבנתה מהווה תשתית מצוינת למוסדות אקדמיים השואפים להחליף את העבודה הלוגיסטית הידנית והמסורבלת.

17. פיתוחים עתידיים

ניתן להרחיב ולשפר את הפרויקט במספר דרכים בעתיד:

- שילוב מערכת הודעות פנימית (צ'אט): פיתוח רכיב תקשורת בזמן אמת (מבוסס WebSockets) שיאפשר למנהל המערכת ולמרצים להתכתב בתוך המערכת לטובת בירורים מהירים על חלונות זמן או שינויים נקודתיים.
- הצגת סטטיסטיקות מתקדמות (Dashboard למנהלים): הוספת מסך גרפי מיוחד למנהל המערכת שיוציג נתונים סטטיסטיים, כגון: אחוזי הניצול של שעות המרצים ומומזעי ציונים של קורסים אקדמיים.
- פיתוח אפליקציית מובייל ייעודית: יצירת אפליקציית טלפון משלימה או הפיכת אתר ה-Web הקיים ל-PWA (Progressive Web App) כך שסטודנטים יוכלו לגשת למערכת השעות שלהם ולקבל התראות ישירות למכשיר הנייד בצורה נוחה יותר מאשר דרך דפדפן רגיל.

18. ביבליוגרפיה

1. Java SE 21 Documentation Oracle. התיעוד הרשמי של שפת Java בגרסה 21. המקור מספק את המידע המלא על תחביר השפה, פיתוח מונחה עצמים, ומבני הנתונים שבהם נעשה שימוש בבניית המחלקות בפרויקט.

[\[https://docs.oracle.com/en/java/javase/21/\]](https://docs.oracle.com/en/java/javase/21/)

2. Spring Boot Documentation Spring. התיעוד הרשמי של מסגרת העבודה Spring Boot. המקור שימש כבסיס להבנת ארכיטקטורת ה-Backend, חיבור מהיר למסד הנתונים בענן, וניהול תקין של בקשות ה-HTTP בשרת התוכנה.

[\[https://docs.spring.io/spring-boot/documentation.html\]](https://docs.spring.io/spring-boot/documentation.html)

3. Vaadin 24 Documentation Vaadin התייעוד הרשמי של מסגרת העבודה Vaadin ליצירת ממשק המשתמש (UI). מקור עיקרי להבנת יצירת רכיבים ויזואליים וסנכרון נתונים ישיר בין השרת ללקוח.

<https://vaadin.com/docs/latest/>

4. MongoDB Documentation MongoDB התייעוד הרשמי של מסד הנתונים מסוג NoSQL. כולל את המידע הנדרש על ניהול אבטחה, הגדרת אוספים (Collections) לנתונים גמישים של מרצים וקורסים, והרשאות גישה בסביבת הענן של MongoDB Atlas.

<https://www.mongodb.com/docs/>

5. הוכחה לשימוש ב-Backtracking לפתרון בעיות CSP (שיבוץ קורסים) - זהו מסמך רשמי מקורס בינה מלאכותית של אוניברסיטת טורונטו (University of Toronto). המסמך מסביר בדיוק כיצד בעיית שיבוץ הקורסים (Course Scheduling) היא בעיית "סיפוק אילוצים" (CSP), וכיצד אלגוריתם הנסיגה לאחור (Backtracking) מיושם כדי לפתור אותה.

http://www.cs.utoronto.ca/~strider/docs/D84_CSPs.pdf

6. מאמר אקדמי על בניית מערכת שעות אוניברסיטאית באמצעות CSP - מאמר מהז'ורנל South African Journal of Industrial Engineering. המאמר מסביר שבעיית בניית מערכת שעות במוסדות לימוד היא בעיית NP-Complete קשה, ומדגים כיצד פותרים אותה באמצעות הגדרת אילוצים קשיחים (Constraints) ושימוש בטכניקות חיפוש עם Backtracking.

<https://sajie.journals.ac.za/pub/article/download/314/259/277>

7. מיון טופולוגי (Topological Sort) וסידור קורסי קדם - זהו פרק מתוך ספר לימוד פתוח על מבני נתונים ואלגוריתמים (Data Structures and Algorithms) של אוניברסיטת צ'אלמס. הפרק עוסק במפורש בגרפים מכוונים ללא מעגלים (DAG), ומדגים כיצד משתמשים במיון טופולוגי בדיוק כדי לפתור את הבעיה של שיבוץ קורסים או משימות על סמך תנאי קדם (Prerequisites).

<https://chalmersgu-data-structure-courses.github.io/OpenDSA/Published/ChalmersGU-DSABook/html/GraphTopSort.html>

8. מחקר עדכני על אופטימיזציה של מערכות שעות (2024)

מחקר טרי שמסביר בצורה יפה כיצד משתמשים בתורת הגרפים ובמודל CSP (Constraint Satisfaction Problem) כדי למנוע התנגשויות (כמו מרצה שמשוּבָּץ לאותו זמן) ולצמצם את האפשרויות הלא חוקיות בעץ האפשרויות.

[https://informatika.stei.itb.ac.id/~rinaldi.munir/Matdis/2024-2025/Makalah/Makalah-IF1220-Matdis-2024%20\(113\).pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Matdis/2024-2025/Makalah/Makalah-IF1220-Matdis-2024%20(113).pdf)